



A Thesis for the

Master in Telecommunication Engineering

Electrocardiogram Denoising Using Generative Adversarial Network

by
Luis Geovanny Triviño Macías

Supervisor: José López Vicario

Telecomunicació i Enginyeria de Sistemes

**Escola Tècnica Superior d'Enginyeria (ETSE)
Universitat Autònoma de Barcelona (UAB)**

June 2022

Resum:

Aquest projecte ha investigat l'estat de l'art de les xarxes neuronals per eliminar el soroll dels senyals d'electrocardiograma (ECG). Hem analitzat i processat les dades de la base de dades disponible a la plataforma PhysioNet per tal de formar una Xarxa Adversarial Generativa (GAN per les seves sigles en anglès). El GAN s'ha dissenyat i implementat utilitzant xarxes neuronals convolucionals, concretament una xarxa de codificador-descodificador per al generador i un PathcGAN per al discriminador, que formen un GAN tipus Pix2Pix. També s'ha implementat una Xarxa Neuronal Convolcuional (CNN) per classificar tipus de batecs de cor.

Resumen:

En este proyecto se ha investigado sobre el estado del arte de las Redes Neuronales para eliminar el ruido de señales de electrocardiogramas ECGs. Se ha analizado y procesado los datos de la base de datos disponibles en la plataforma PhysioNet para poder entrenar una Red Generativa Adversaria (GAN, por sus siglas en inglés). Se ha diseñado e implementado la GAN mediante Redes Neuronales Convolucionales, en concreto una red Encoder-Decoder para el Generador y una PathcGAN para el Discriminador, las cuales conforman una GAN de tipo Pix2Pix. También se ha implementado una Red Neuronal Convolcuional (CNN) para clasificar tipos de latidos de corazón..

Summary:

This project has investigated the state of the art of Neural Networks to remove noise from electrocardiogram ECG signals. We have analysed and processed the database data available in the PhysioNet platform in order to train a Generative Adversarial Network (GAN). The GAN has been designed and implemented using Convolutional Neural Networks, specifically an Encoder-Decoder network for the Generator and a PathcGAN for the Discriminator, which form a Pix2Pix type GAN. A Convolutional Neural Network (CNN) has also been implemented to classify heartbeat types.

Table of contents

1. Introduction.....	1
1.2. State of the art.....	2
1.3. Objectives.....	3
1.4. Tools.....	3
1.4.1. Tensorflow.....	3
1.4.2. Google Colaboratory	4
2. Artificial Intelligence	5
2.1. Machine Learning (ML)	5
2.2. Deep Learning	6
2.2.1. Deep Neural Networks (DNN)	6
2.2.2. Activation Fucntions	6
2.2.3. Loss Functions	9
2.2.4. Backpropagation.....	11
2.2.5. Optimizer.....	12
2.3. Convolutional Neural Networks (CNN).....	13
2.3.1. CNN Architecture.....	14
2.3.2. Classification of objects in the image.....	16
3. Generative Adversarial Networks (GANs)	19
3.1. Objective functions	20
3.2. Types of GANs.....	21
3.2.1. Convolutional Generative Adversarial Networks (CGANs).....	21
3.2.2. Conditional Generative Adversarial Networks	21
3.2.3. Cycle GAN	22
3.2.3. Pix2Pix	23
3.3. Applications of GANs.....	23
3.4. Disadvantages of the GANs.....	24
4. Dataset	26
4.1. MIT-BIH Arrhythmia Database	26
4.2. MIT-BIH Noise Stress Test Database.....	26
4.3. Database processing.....	28
5. GAN Implementation.....	33
5.1. Implemented GAN structure	33
5.1.1. Generator	34
5.1.2. Discriminator	37
5.2. Training Objective Functions	39

5.2.1.	Generator Loss	39
5.2.2.	Discriminator Loss	40
5.3.	Training step.....	41
5.4.	Results	42
6.	Classifier Network.....	58
6.1.	Network architecture	60
6.2.	Optimiser and loss function	61
6.3.	Training	61
6.4.	Results	62
7.	Conclusions	65
	Appendices.....	68
	Appendix 1	68

List of Figures/Tables

Figure 1: Heart beat period	1
Figure 2: AI fields.....	5
Figure 3: Single Neural Network and Deep Neural Network	6
Figure 4: Sigmoid Activation Function.....	7
Figure 5: Hyperbolic Tangent Activation Function	8
Figure 6: ReLU Activation Function	8
Figure 7: Leaky ReLU Activation Function	9
Figure 8: Softmax Activation Function.....	9
Figure 9: Gradient Descent Method.....	11
Figure 10: Backpropagation Diagram	12
Figure 11: CNN Basic Architecture	14
Figure 12: Color Image with three channels (RGB)	14
Figure 13: Convolution Process Schema	15
Figure 14: Max-Pooling	15
Figure 15: Max Pooling Real Case	16
Figure 16: Concatenation of two convolution modules (Convolution + ReLu + pooling)	16

Figure 17: Pre-feed neural network model with only one hidden layer.....	17
Figure 18: GAN Architecture	19
Figure 19: DCGAN Architecture	21
Figure 20: Conditional GAN Simple Architecture	22
Figure 21: Cycle GAN Structure	22
Figure 22: pix2pix Generator.....	23
Figure 23: Signal with BW noise. Extract of the database.....	27
Figure 24: Signal with MA noise. Extract of the database.....	27
Figure 25: Signal with EM noise. Extract of the database.....	27
Figure 26: Heartbeat sample.....	28
Figure 27: Sample ECG signal	28
Figure 28: Signal corrupted with MA noise.....	29
Figure 29: Signal corrupted with EM noise.....	29
Figure 30: Signal corrupted with BW noise	30
Figure 31: Signal corrupted with MA + EM noises	30
Figure 32: Signal corrupted with MA + BW noises.....	30
Figure 33: Signal corrupted with BW + EM noises.....	31
Figure 34: Signal corrupted with MA + EM + BW noises	31
Figure 35: Overall GAN Structure	33
Figure 36: Encoder-decoder architecture.....	34
Figure 37: Encoder.....	35
Figure 38: Decoder.....	36
Figure 39: Generated output without training	37
Figure 40: Discriminator Structure	38
Figure 41: Evolution of the Generator and Discriminator Losses.....	41
Figure 42: Denoising signal corrupted by MA noise at level 0.2.....	43
Figure 43: Denoising signal corrupted by a real sample of MA noise.....	44
Figure 44: Denoising of signal corrupted by BW noise at 0.2 level of noise	45
Figure 45: Denoising signal corrupted by a real BW noise sample	46
Figure 46: Denoising signal corrupted by EM noise at 0.2 level	47
Figure 47: Denoising signal corrupted by a real sample of EM noise	48
Figure 48: Denoising signal corrupted by MA + BW at 0.2 level of noises.....	49
Figure 49: Denoising signal corrupted by a real sample of MA + BW noises.....	50
Figure 50: Denoising signal corrupted by EM + MA at 0.2 level of noises	51
Figure 51: Denoising signal corrupted by a real sample of EM + MA noises	52
Figure 52: Denoising signal corrupted by EM + BW at 0.2 level of noises	53
Figure 53: Denoising signal corrupted by a real sample of EM + BW noises	54
Figure 54: Denoising signal corrupted by MA + EM + BW at 0.2 level noises	55
Figure 55: Denoising signal corrupted by a real sample of MA + EM + BW noises	56
Figure 56: Structure of the system to be achieved	58
Figure 57: Samples initial distribution	59
Figure 58: Samples distribution after resample.....	59
Figure 59: Shape of the Heartbeats by types	60
Figure 60: Classifier Network Structure.....	61
Figure 61: Model accuracy evolution.....	62
Figure 62: Model Loss evolution	62
Figure 63: Model Predictions I.....	63
Figure 64: Model Predictions II.....	64

1. Introduction

According to the World Health Organisation (WHO), cardiovascular diseases (CVD) are the leading cause of death worldwide. Among them are heart diseases that affect a person's normal life, in some cases leading to death. [1]

The main method of diagnosing these diseases is the electrocardiogram (ECG), which measures the electrical activity of the heart. Every time the heart beats, an electrical signal flows through it. An ECG shows whether the heart is beating at a normal rate and strength. An abnormal ECG can be a sign of heart damage or disease. This method, because of its simplicity and non-invasiveness to the patient, is the most widely used method to detect, classify and treat heart disease.

Figure 1 shows the complete period of a heartbeat. Each heartbeat has 6 peaks: P, Q, R, S, T and U, which, in practice, the ECG signal is mixed with a lot of noise, which makes it difficult for a professional to diagnose by making it impossible or difficult to distinguish the different pulses that make up the ECG signal. For this reason, the need arises to remove the noise from the signal.

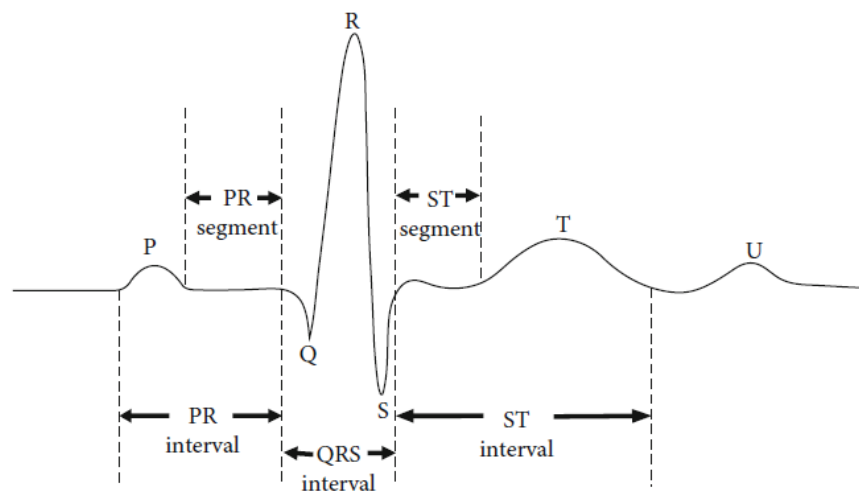


Figure 1: Heart beat period

Generally, ECG signals include three types of noise:

- Baseline deviation (BW)
- Muscle artefact (MA)
- Electrode motion artefact (EM)

P wave: Represents the depolarisation or contraction of the atria pushing blood into the ventricles [3] Its normal duration is 0.1s. The shape depends on the location of the electrodes (lead). An increase in the voltage or duration of this wave indicates an atrial abnormality.

QRS complex: Represents the depolarisation of the ventricles. It consists of the Q, R and S waves. Its normal duration is approximately 100ms.

T wave: Represents the repolarisation of the ventricles. The normal T wave is asymmetrical in its branches and rounded at its apex. The slope of the initial branch is smoother than that of the terminal branch. Abnormalities of this wave may indicate primary cardiac disease, although there are cases of healthy individuals with the same abnormalities.

U-wave: Of unclear physiological origin, this wave is thought to be produced by repolarisation of the intraventricular conduction system.

1.2. State of the art

Several methods for denoising ECG signals have been studied and can be divided into two main groups, traditional methods and methods using Deep Learning (DL). Traditional ECG denoising methods include Fourier decomposition [2], spectral decomposition [3], empirical modal decomposition (EMD) [4], wavelet transform (WT) [5], filtering [6][7] among others. These methods have limitations of adaptability when analysing different types of noise, i.e., they can be effective for one type of noise, whereas, if the signal to be analysed is mixed with another type of noise, the result may not be satisfactory, causing the ECG signal to lose value.

On the other hand, advances in the field of artificial intelligence, specifically in the field of deep learning (we will explain these terms in more detail at a later point), have led to an increase in the study of new methods to eliminate noise in ECGs. The most representative are the functional link neural network (FLNN) [8], obtaining good results for BW and EM denoising, but separately; the Wavelet neural network (WNN) [9]; the automatic denoising encoder (DAE) [10]. Among these methods, FLNN and WNN are the most widely used, but they can only remove one type of noise in the ECG signal, although in their improved versions they were able to reduce all three types of noise, the ECG signals ended up losing value due to the poor adaptability of these networks.

The model chosen for this thesis, Generative Adversarial Networks (GAN) which were presented by Goodfellow, J et al. [11], and it will be discussed in more detail later, has been very popular recently in the field of machine learning (ML). Researchers have made great advances with variants of GAN, such as CycleGAN, Conditional GAN, Convolutional GANs or pix2pix, but all of them have been used for the generation of realistic data (text, images, audio, etc.) from random noise. However, in the field of noise reduction, the use of GANs is limited to image data because of their great effectiveness in image processing, and little research has been done on the possibility of using this type of architecture for noise removal for other types of data.

1.3. Objectives

The aim of this work is to remove the most common types of noise from ECG signals using the GAN neural network model.

We also want to emphasise that GANs are a very powerful architecture for tackling other problems not related to image processing.

To achieve the objective of the project we have studied and processed the MIT-BIH Arrhythmia Database and the MIT-BIH Noise Stress Test Database, understanding the nature of these, and formatting them for use in our model. Once the analysis and formatting of the data was done, we proceeded to create the structure of the GAN to solve the problem and concluded with the analysis of the results.

A Convolutional Neural Network (CNN) has also been implemented to classify different types of heartbeats.

1.4. Tools

This section will introduce the two main tools and platforms that have facilitated the implementation of the project. Both tools are owned by Google, which allows a high compatibility between them.

1.4.1. Tensorflow

Tensorflow is an open-source platform for numerical computing and machine learning developed by Google. It is one of the most popular frameworks for performing Natural Language Processing (NLP) and Machine Learning tasks.

The name TensorFlow is mainly since it works by the flow of tensors in the form of a computational graph. A tensor is an n-dimensional vector or matrix representing all types of data.

Its architecture allows for easy implementation of computations on multiple platforms such as CPU, GPU and others.

It is also worth noting that Tensorflow includes the Keras API, which allows access to various machine learning frameworks and to develop them, accelerating the creation of neural networks.

Thus, Tensorflow facilitates the main processes when designing and implementing Machine Learning models such as data acquisition, model training and model analysis.

1.4.2. Google Colaboratory

The main tool used to carry out the project is Google Colaboratory or Google Colab, which allows writing and running Python code in the browser. It is a hosted notebook service from Jupyter that requires no configuration and allows access to computing resources (GPUs, RAM) suitable for machine learning, data analysis and education tasks. This allows you to simply have a Google account, open the Google Colab application in your browser and start implementing the model.

The files that are generated in the project are stored in Google Drive, these include the scripts or notebooks, the resulting images, save points, etc. Also, the files needed to run the project, such as datasets or external scripts, must be preloaded in Google Drive, although it is also possible to import datasets from other compatible platforms such as Kaggle, but this requires a bit of configuration.

So, using Google Colab it is only necessary to import the Tensorflow and Keras frameworks and libraries without having to install anything locally, speeding up the work a lot.

It should be noted that Google Colaboratory offers two subscription plans costing 10 and 50 euros per month. During the neural network training stage, the 10€ subscription had to be taken out, as this stage required a lot of time to complete. At times, GPUs were not available in the free plan to perform the training and the CPU was used, which in terms of time took up to 20 times longer to complete the process.

It has actually been a good idea to sign up for the service over the last few months, as it has improved greatly in terms of execution times, becoming at least twice as fast.

2. Artificial Intelligence

Nowadays, Artificial Intelligence (AI) plays a very important role in research and industry because of its high performance in many practical applications. AI is intended to develop an intelligent system to automate routine work, to process audio or images coherently, to make diagnoses in medicine, and so on.

There is no agreed way to define IA. One could define the concept as a discipline of computer science where autonomous and adaptive systems are designed that can perform specific tasks by mimicking or even improving human behaviour. To achieve this, large amounts of data are required.

In AI we can find subfields such as Machine Learning (ML) or Deep Learning (DL), which are very interesting as they can solve problems in which prediction or classification are present.

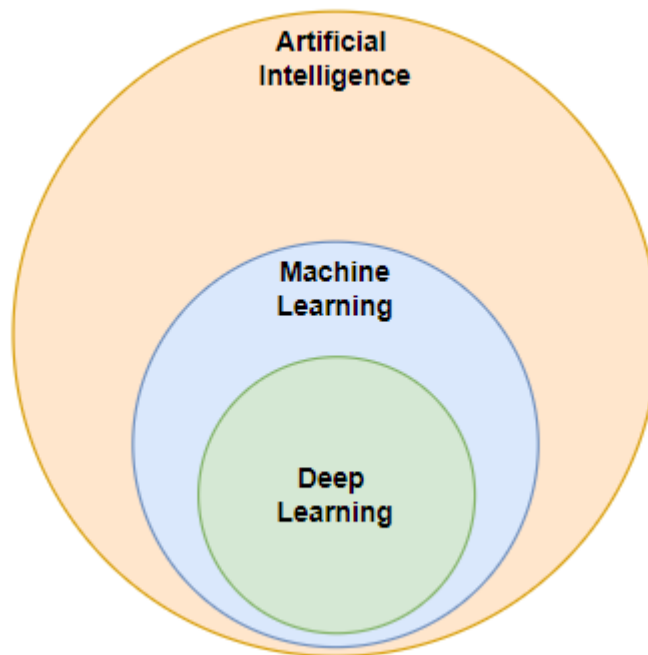


Figure 2: AI fields

2.1. Machine Learning (ML)

Machine Learning (ML) is a subfield of AI that aims to mimic human behaviour by means of algorithms and statistics, extracting information from data and making the task to be performed time effective. ML could be categorised as follows:

- Supervised learning: these are models that learn relationships that associate inputs with outputs, i.e., these models are trained from a set of labelled data. These models are used in classification (classifying specific data) or regression applications (a relationship is established between the input data and its output).

- Unsupervised learning: these models do not have labelled data, so it must learn on its own by finding a relationship between the input data. These models are commonly used in descriptive models (pattern detection, understanding, communication).

2.2. Deep Learning

Deep learning (DL) is a subfield of ML, which consists of the use of deep models or Deep Neural Networks i.e. systems that mimic the human brain, adapting and learning from large amounts of data. The word deep refers to the number of layers used in the model.

2.2.1. Deep Neural Networks (DNN)

In general, the term DL refers to deep neural networks. A deep neural network (DNN) is a neural network (NN) with multiple layers between the input and output layers, which are called hidden layers. Each neural network is a system with n inputs and m outputs consisting of elements called perceptrons or artificial neurons interconnected with each other. These generate an output from an input according to their parameters or weights. The perceptrons in turn, a certain function called Activation Function is applied to them. It is the weights that will be modified and adjusted throughout the training to obtain the desired result. The algorithm that modifies the value of the weights of the neurons is called backpropagation and is what gives the network its automatic learning capacity.

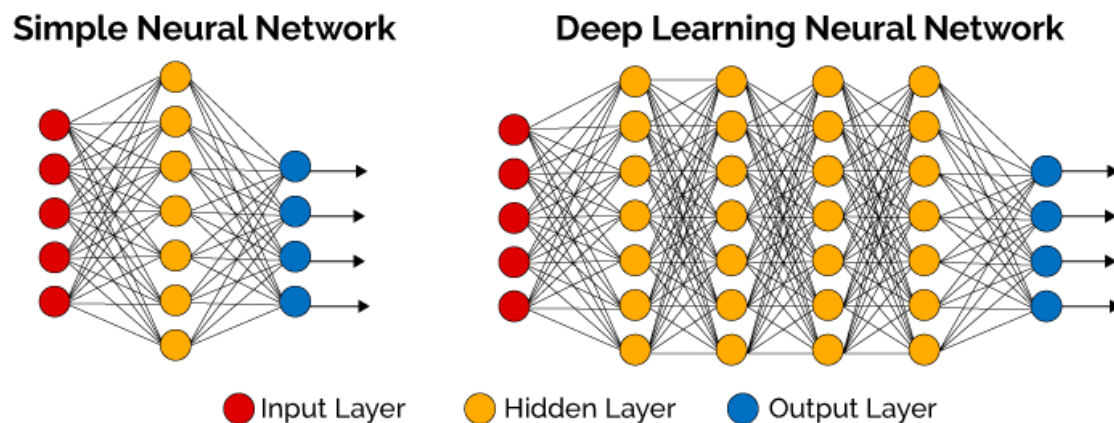


Figure 3: Single Neural Network and Deep Neural Network

2.2.2. Activation Functions

At the output of the neuron, a filter or limiting function may be added, which modifies the result value or imposes a limit that must be exceeded to proceed to another neuron. This function is known as an activation function. An activation function transmits the information generated by the linear combination of weights and inputs, i.e., they are the means of transmitting the information via the output connections. The information can be transmitted without modification, identity function, or it cannot transmit the information at all. Since the goal is for the neural network to be able to solve increasingly

complex problems, activation functions will generally make the models non-linear. Among the best known or most used activation functions are:

- **Sigmoid:** this function generates an output within the range $[0, 1]$, so its output is interpreted as a probability. If the function is evaluated with very negative input values, i.e. $x < 0$ the function will be equal to zero, if it is evaluated at zero the function will give 0.5 and at high values its value is approximately 1 as it can be seen in the Figure 4. This function has one drawback and that is that its range is not centred on the origin. However, this last problem can be solved by using the hyperbolic tangent function, which is very similar. Equation (1) describes its nature.

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (1)$$

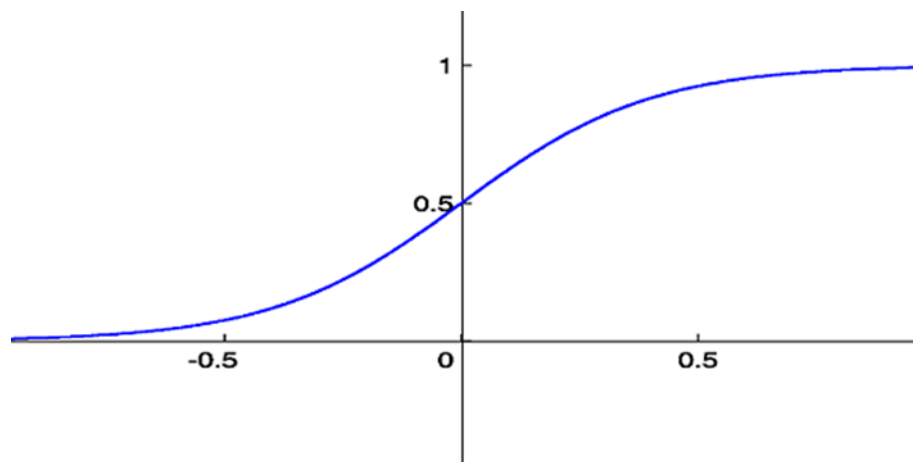


Figure 4: Sigmoid Activation Function

- **Tanh:** the shape of this function is like the sigmoid function, but in this case, it has a range of output values between $[-1, 1]$ (see figure 5). In (2) we can see its description. The most common use of this function is for sorting between two categories.

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (2)$$

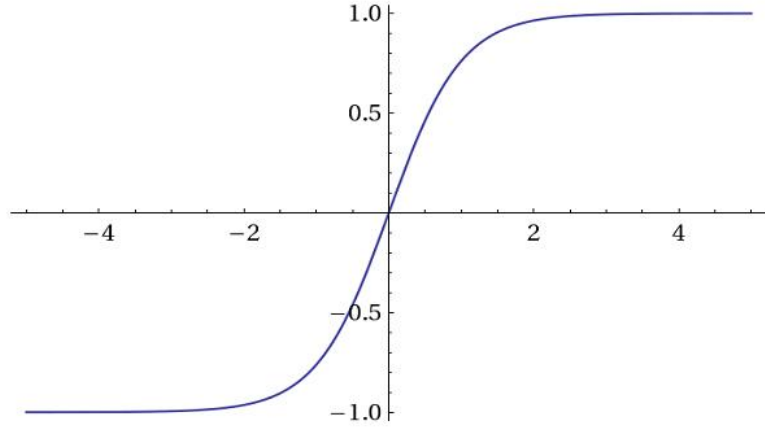


Figure 5: Hyperbolic Tangent Activation Function

- **ReLU (Rectified Linear Unit) Activation Function:** For negative values results in zero while for positive values in the identity. Its range is $[0, \infty]$ (see figure 6). It is currently the most widely used activation in the intermediate layers of a network. Among its main characteristics we find that it has a very low computational cost, which improves the performance of the network. In addition, it helps to quickly converge neural networks. Equation (3) describes the ReLU function.

$$ReLU(x) = \max(0, x) \quad (3)$$

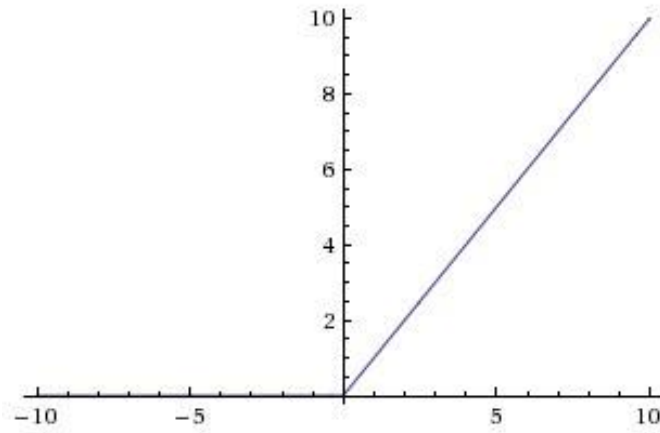


Figure 6: ReLU Activation Function

- **Leaky ReLU:** The problem with the ReLU function is that it returns negative values to 0, decreasing the model's ability to train. To solve this problem, the Leaky ReLU activation function was proposed, which consists of adding negative slope to the negative part of the function, increasing the range of the ReLU function to $[-\infty, \infty]$ (see figure 7). The slope is usually of value 0.01. Equation (4) describes its behaviour.

$$LeakyReLU(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases} \quad (4)$$

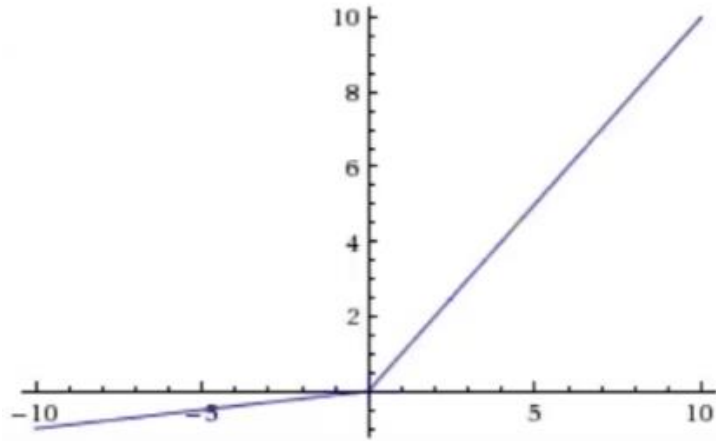


Figure 7: Leaky ReLU Activation Function

- **Softmax:** This function is used to classify data, for example if we give as input the image of a fruit and we are asked to know the type of fruit it belongs to, applying softmax the network will give us the probability that it can be 0.3 or 30% melon, 0.2 or 20% watermelon and 0.5 or 50% papaya, so the result will be the one with the highest probability and it should be noted that the sum of these probabilities will be equal to 1. In other words, Softmax is used for multiple classes and when assigning probabilities to each class that belongs to multiple classes. This function is described by (5).

$$\text{Softmax}(x_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (5)$$

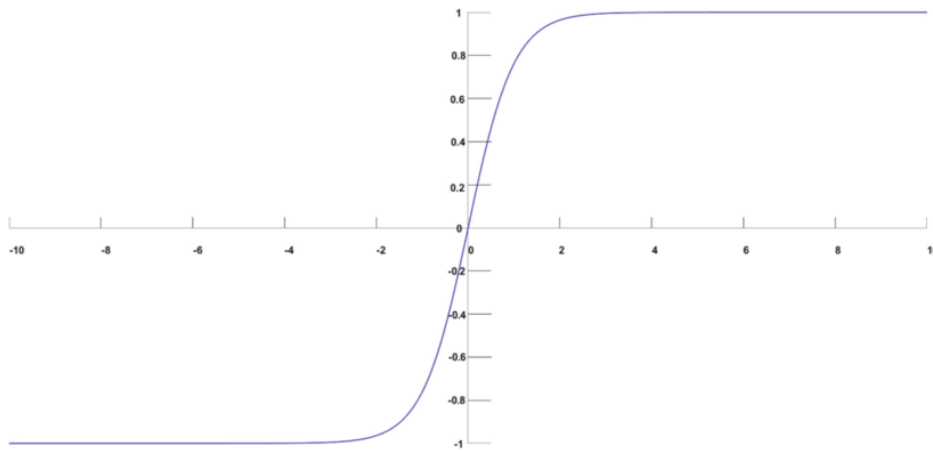


Figure 8: Softmax Activation Function

2.2.3. Loss Functions

Loss functions allow us to determine the error with which the neural network is responding. There are several types of loss functions, and this section will explain the most common ones.

- **Mean Absolute Error (MAE) or L1 Loss:** The mean absolute error is measured as the average of the sum of the absolute differences between predictions and actual

observations. Like MSE, it also measures the magnitude of the error without considering its direction, but it is more robust to outliers as it does not use squaring. Its description is given by equation (6).

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (6)$$

Where:

- N is the number of samples
- y_i is the actual label of sample i
- $\hat{y}_i$ is the predicted label for sample i

- **Mean square error (MSE) or L2 Loss:** The mean squared error is measured as the average of the squared difference between predictions and actual observations regardless of their direction. Due to the quadratic term, predictions that are far away from the actual values are heavily penalised compared to less deviated predictions. Its description is given by equation (7).

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (7)$$

Where:

- N is the number of samples
- y_i is the actual label of sample i
- $\hat{y}_i$ is the predicted label for sample i

- **Cross Entropy (CE):** This is the most common loss function for classification problems. The cross-entropy loss increases as the predicted probability diverges from the actual label, i.e., it only penalises correct predictions. The value of the probability is in the range $[0,1]$, where 0 would refer to a perfect model. Its description is given by (8).

$$CE = -\frac{1}{N} \sum_{j=1}^N \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (8)$$

Where:

- N is the number of samples
- c is the number of classes
- y_i is the actual class of sample i
- $\hat{y}_i$ is the predicted class for sample i

- **Binary Cross Entropy (BCE):** It is normally used in binary classification problems, although it can also be used in problems where the variables to be predicted take values between zero and one. Its description is given by (9), where depending on the value of the class to be predicted, one of the two logarithmic terms cancels out.

$$BCE = -\frac{1}{N} \sum_{i=1}^N y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \quad (9)$$

Where:

- y is the class to predict
- p is the predicted probability for the class
- c is the number of classes
- N is the number of samples

2.2.4. Backpropagation

This algorithm emerged in 1986, when Rumelhart, Hinton and Williams wrote the paper Learning representations by back-propagating errors [12] and until this time the machine learning algorithm using the perceptron was not extensible to complex neural networks. With the advent of this algorithm, a neural network was able to self-adjust its parameters to learn an internal representation of what it was processing.

The algorithm is based on the gradient descent method, an iterative algorithm by which the error can be minimised until its minimum is found (see figure 9) and employs a two-phase propagation cycle. In the first phase, once input values have been applied, these are propagated through the layers of the network until an output is generated, which is compared with the desired output and, if these do not match, an error signal is obtained. In the second phase, it is the error signal that propagates from the output to each of the neurons in the hidden layers, as can be seen in figure 10. The gradient can be calculated using partial derivatives of the loss functions.

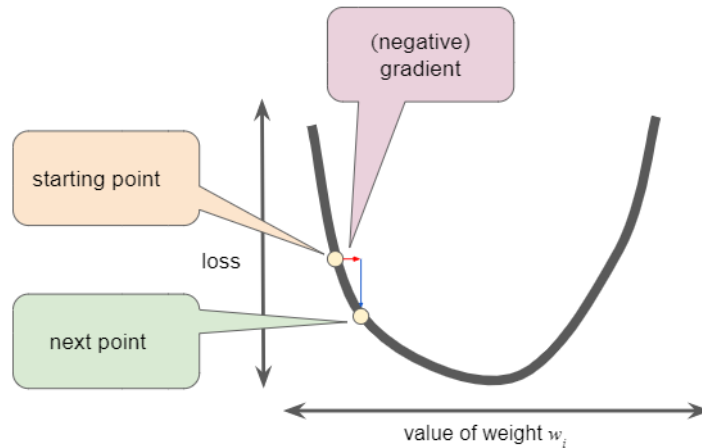


Figure 9: Gradient Descent Method

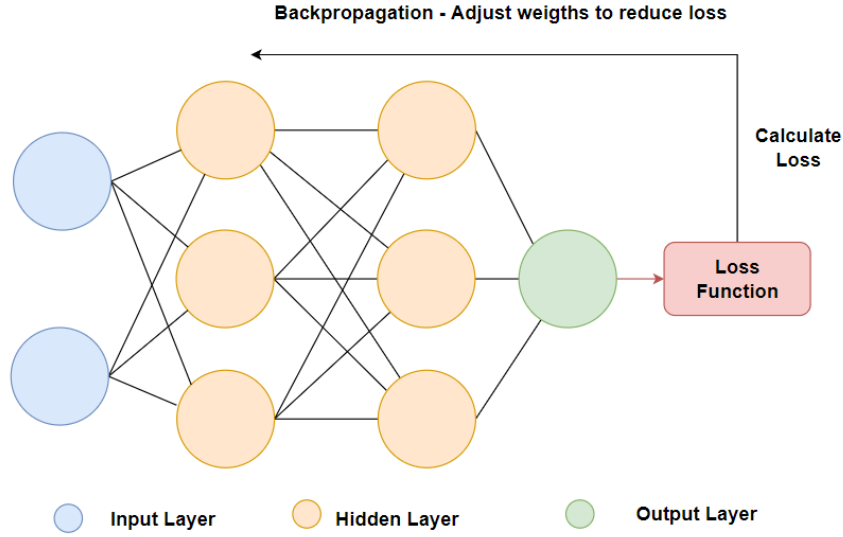


Figure 10: Backpropagation Diagram

2.2.5. Optimizer

The optimiser is responsible for modifying the neural network parameters or weights according to the loss function. The update occurs every batch. There are several types of optimisers, we will focus on the two most important ones:

- Stochastic Gradient Descent (SGD): It is the basic optimisation method of neural networks and other machine learning algorithms. The updating of the weights occurs iteratively according to the equation 10.

$$w = w - \alpha \nabla W \quad (10)$$

Where W is the weight matrix or parameters, ∇W is the derivative of the weight matrix, α is the learning rate.

- Adaptive Gradient Algorithm (Adagrad): This adaptive method adjusts the learning rate to the network parameters, where larger updates are carried out on parameters that infrequently change, while those that vary more frequently receive a smaller adjustment. The defining equations are:

$$C = C + \nabla W \quad (11)$$

$$W = W - \frac{\alpha \nabla W}{\sqrt{C + \epsilon}} \quad (12)$$

Where C is a cache that holds the sum of the squares of the gradients (derivatives), for each parameter. C allows us to update each parameter respectively to the magnitude of its change. ϵ is used to avoid divisions by 0.

- **Adadelta:** This is an improvement to Adagrad, which seeks to reduce the monotonic decay of the learning rate due to the nature of the cache, C . The equations are the same, but while Adagrad calculates the cache for each mini-batch, taking into account the entire history of squared gradients, Adadelta does so only for a small number of previous squared gradients. Thus, what this optimiser does is to calculate the decaying average of the gradient history.
- **RMSprop:** seeks to correct the negative effects of the global accumulation of caches by converting it to an exponential moving average with weights.

$$C = C + dC(1 - d)\nabla W^2 \quad (13)$$

$$W = W - \frac{\alpha \nabla W}{\sqrt{C + \epsilon}} \quad (14)$$

The new parameter, d , known as the decaying rate, typically set at 0.9, which makes previous entries in the cache considerably less influential than new entries.

- **Adaptive Moment Estimation (Adam):** This optimiser is faster than SGD and converges more easily. This optimiser is based on RMSprop with momentum. Adam is updated as follows:

$$m = \beta_1 m + (1 - \beta_1) \nabla W \quad (15)$$

$$v = \beta_2 v + (1 - \beta_2) \nabla W^2 \quad (16)$$

$$W = W - \frac{\alpha m}{\sqrt{v + \epsilon}} \quad (17)$$

Where m and v represent the two momentums, with m modelling the mean of the gradients over time, while v does the same for the variance. β_1 is equal, in most cases, to 0.9, while β_2 is almost always fixed at 0.99.

2.3. Convolutional Neural Networks (CNN)

In this section, Convolutional Neural Networks will be introduced to explain the nature of this type of networks as they will be used to implement the final system.

CNNs are a type of Artificial Neural Network (ANN), which can process a two-dimensional structured input as an image, i.e., it uses two-dimensional arrays to process the information. For this reason, they are the most popular type of ANN for tasks such as computer vision, image classification, among others.

2.3.1. CNN Architecture

Like all neural networks, CNNs are made up of layers, between which the processed information is passed. The layers can be seen in figure 11, where three main blocks can be highlighted: the input, the feature obtaining stage and, finally, the classification stage based on the image features.

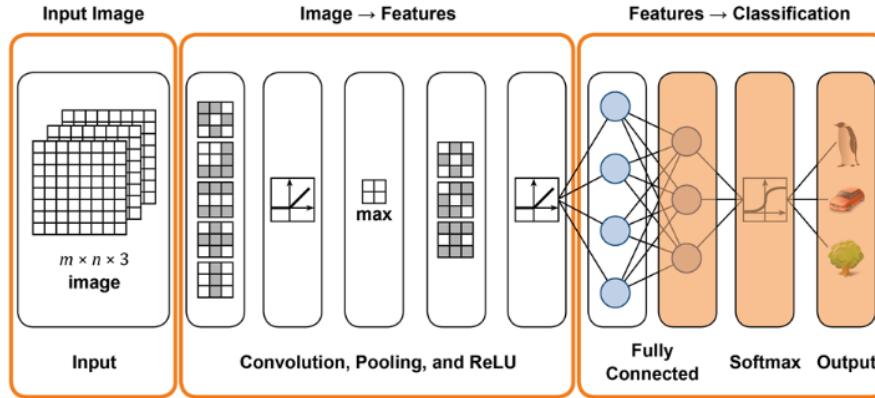


Figure 11: CNN Basic Architecture

Input

The input layer takes the pixels of an image. The number of neurons is given by the pixels and the colours of the image, for example, if an image has $7 \times 7 \times 1$ pixels (width x height x colour channels where 1 is greyscale) it would be equivalent to having 49 neurons, if the image had colour, we would need 3 channels to represent it, red, blue and green (RGB), so the same image would go from having 49 neurons with the image in greyscale to $7 \times 7 \times 3 = 147$ neurons with the image in colour.

The pixels have values ranging from 0 to 255 but are normalised for the neural network from 0 to 1. An example can be seen in figure 12.

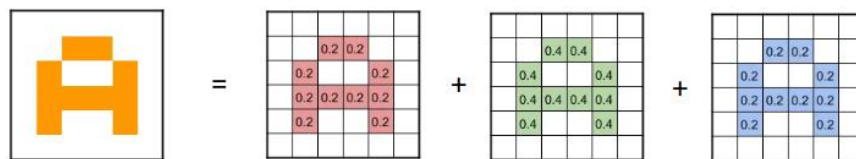


Figure 12: Color Image with three channels (RGB)

Obtaining image characteristics

This section will explain the different layers and operations within the CNN to obtain the most important features of an input image.

- **Convolution:** In convolution, the scalar product of the input image is performed with another smaller matrix called kernel. The output matrix generated will be a new layer of hidden neurons. It should be noted that, if an image is a colour image, the scalar product between each channel of the input image and three kernels (one for each channel) will be made, producing a processed version of each channel. Each of the processed versions of the channel are added together to form a new channel, to which

the scalar bias is added to produce the final output channel, as shown in figure 13. The set of kernels is called a filter, so, in short, the image is processed through one or several filters until we arrive at as many output channels as we have filters. The aim of this process is to obtain very simple features which will contribute to the analysis of the input image.

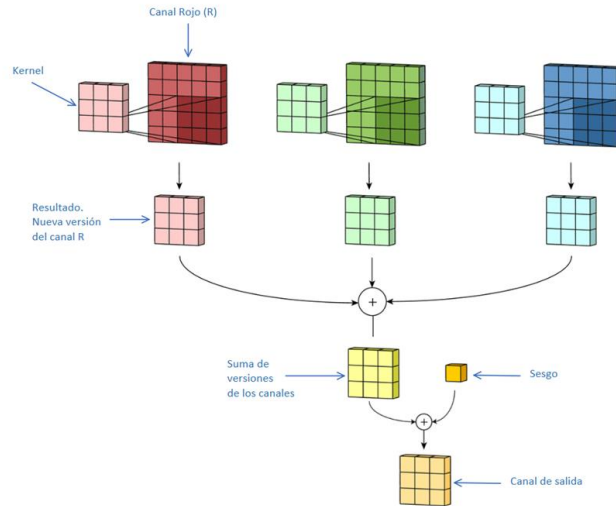


Figure 13: Convolution Process Schema

By applying the ReLU activation function, a matrix is obtained in which a series of image characteristics are more relevant than those obtained in the convolution.

- Pooling:** After applying the activation function we will normally have a large number of neurons, for example, in the case of a colour image of $n \times m$ pixels with 3 colour channels, we would have $n \times m \times 3$ neurons, when applying ' h ' filters, we would have $n \times m \times 3 \times h$, as we can see, many neurons, so if we apply the convolution process again to this last layer directly, we would end up having more neurons and as a consequence we would need a lot of computational capacity. To solve this problem, we sample to obtain the values of the matrix of the last most important convolution that each filter has detected. The most used sampling is Max-Pooling, which consists of grouping the maximum value of each region through which the sampling filter passes. Let us say we have a 4×4 matrix that represents our initial input, and we pass it through a 2×2 filter. This filter, when run on the image, will take the maximum value of the regions that the filter passes through, in this case no, the regions will not be assumed. We can see the example in figure 14, where each coloured region represents the region through which the filter passes.

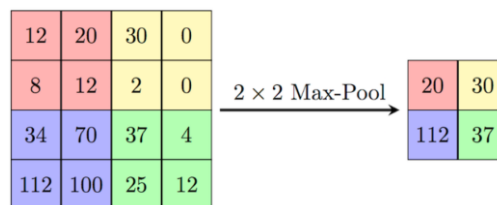


Figure 14: Max-Pooling

An example for a real case can be seen in figure 15, where the z-dimension does not vary.

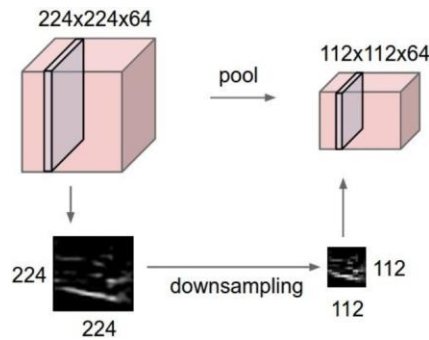


Figure 15: Max Pooling Real Case

At the end of the features stage, we will obtain those whose relevance has more weight, that is, we will be saving in the matrix the features that are more relevant for the identification of an object within an image. Several stages of convolutions can be concatenated in such a way that the matrix resulting from one convolution goes through the same stage, to obtain more precise features, as shown in figure 16.

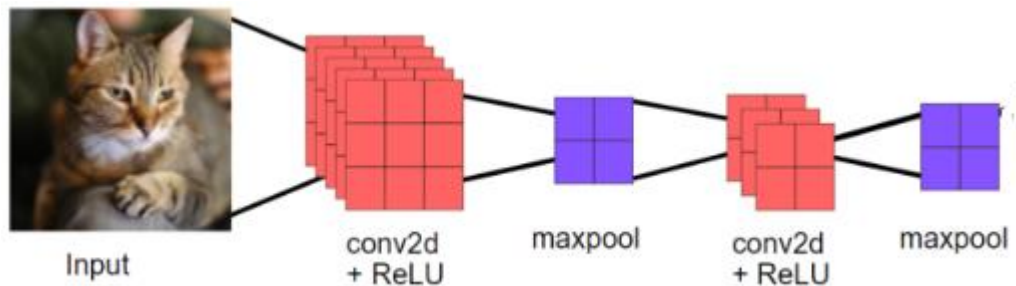


Figure 16: Concatenation of two convolution modules (Convolution + ReLu + pooling)

2.3.2. Classification of objects in the image

This section will present the last part of the CNNs where the features obtained will be processed to classify and distinguish the different objects that the input image may have.

- **Fully Connected (FC) Layer**

Once the relevant convolutions have been made for the image processing in order to have the most important features of the image, a three-dimensional matrix is obtained as the last layer, as the result of doing the grouping of figure 6, which must be flattened, forming a two-dimensional matrix, i.e. having an image with all the important features of the original image, becoming the form of inputs of a layer of a simple or fed back ANN as shown in figure 17.

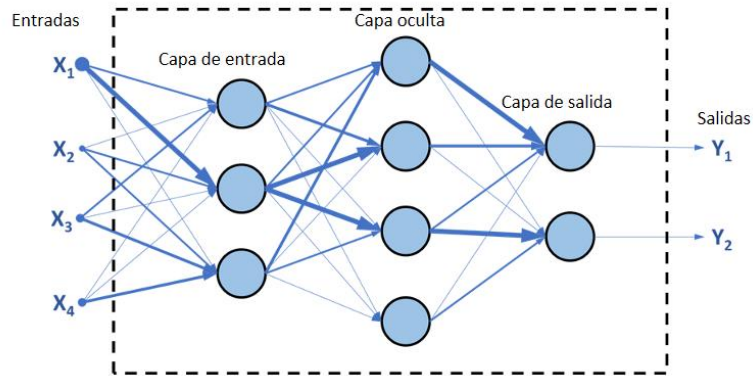


Figure 17: Pre-feed neural network model with only one hidden layer

- **Softmax Fucntion and Output**

To the hidden layer of this pre-fed ANN, Softmax is applied, which has the objective of passing to normalised probability between 0 and 1 and connects to the final output layer which will have a number of neurons equal to the number of different classes we are calcifying, for example, if we classify between car, tree or pedestrian, the number of output neurons will be three. The format of the outputs is known as "one-hot-encoding" which has the form of a vector, e.g. [0 0.2 0.8], indicating that there is a 20% chance that the identified object is a tree, and 80% chance that it is a pedestrian.

3. Generative Adversarial Networks (GANs)

A GAN is a deep neural network architecture that uses machine learning to generate data. The structure is mainly composed of two neural networks called the Generator network and the Discriminator network. Both networks train each other and try to outwit each other. The main goal of the Generator network is to generate data (image, video, audio, or text) from randomly generated data. In contrast, the Discriminator network tries to differentiate between the real data and the generated data, i.e., this network classifies the input data into predefined categories, which can be binary or multiclass. An example of binary would be the differentiation of what is real or fake. On the other hand, an example of multi-class classification would be the recognition of different objects from an image.

Each network can be any neural network such as Artificial Neural Network (ANN), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN) among others.

The term adversarial refers to the fact that these two networks compete with each other. Each network guides the other by providing feedback from successful changes in its own process at each iteration, i.e., the two networks are trained simultaneously.

The generator has no direct access to the actual data, so its only way to learn is by interacting with the discriminator. The discriminator has access to both real and fake data.

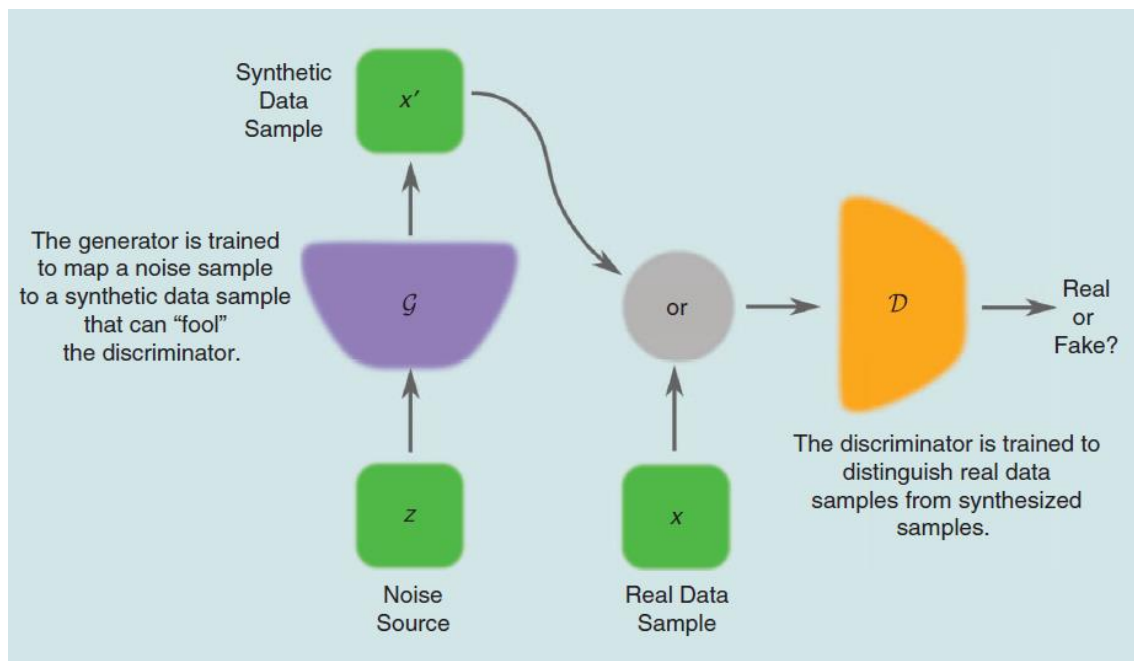


Figure 18: GAN Architecture

One of the most used ways to explain this structure is to assume that one wants to forge a work of art. In this scenario we would have a forger and an art critic. The forger, who will be in

charge of trying to replicate a piece, would refer to the Generator. On the other hand, the art critic would refer to the Discriminator.

1. The forger is trying to create an artwork that looks real, but with the particularity that he has never seen the real artwork.
2. The art critic is trying to distinguish whether a work of art is real or fake.
3. Again, the forger creates more realistic works of art in multiple attempts to try to fool the critic.
4. The critic refines his criteria for determining whether a work is fake by outwitting the forger.
5. They guide each other by providing feedback on the success stories they realise in their own process at each iteration. This point is the training of the GAN.
6. Finally, the critic trains the counterfeiter to the point where it is no longer possible to determine which work is real and which is fake, this state is known as Nash equilibrium. In other words, a Nash equilibrium occurs when none of the networks can improve.

3.1. Objective functions

As discussed above, in GANs, the aim is to increase the similarity of the generated data to the real data. To measure this similarity, objective functions are used. The two networks have their own objective function and during training they try to minimise their respective functions. Equation (18) shows the objective function of the GAN, where:

- D represents the probability of whether the data is true or false. Its output is a number between 0 and 1 interpreting 0 as false and 1 as real.
- G represents the Generator function, i.e. the generated sample.
- p_{data} is the distribution of the real data.
- p_z is the distribution of the data generated by G .
- x and z represent the samples in each space.
- E_x and E_z are the expected log likelihood of the different real and generated image outputs.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (18)$$

In the above expression, the value (V) represents a function for the Generator G and the Discriminator D . The objective of D is to maximise losses while that of G is to minimise losses, until there is a balance between the two networks. The value of V is the sum of the expected log-likelihood for the actual and generated data, i.e., the sum of the discriminator outputs for actual data or generated data. It is important to note that in the second logarithm to one, the response of the discriminator to a real data is subtracted. By applying the max function, the results lead to optimisation of the discriminator's parameters, allowing it to learn to better identify which data is real and which is fake.

During the training of both networks, errors in their outputs are propagated back to the models as gradients of the following loss functions.

3.2. Types of GANs

There are many types of GAN architectures, especially in applications where images are involved. This section will explain the most popular types of GANs.

The original GAN architecture used fully connected neural networks in both networks [11] and were implemented on simple image datasets such as MNIST (handwritten numbers).

3.2.1. Convolutional Generative Adversarial Networks (CGANs)

This architecture is based on convolutional layers, which are very efficient at processing images, as they can learn about image features such as edges or shapes.

Initially, CGANs were more difficult to train compared to conventional convolutional neural networks (CNNs) using the same level of capacity and representation, as they were two networks.

Deep Convolutional Generative Adversarial Networks (DCGANs) [16] were the first GANs to be able to create images with good resolution.

In this architecture, the generator and the discriminator have a similar architecture based on convolutional layers but inverted with respect to each other (see figure 19).

In the generator there is an increase in the dimension of the original noise vector, while in the discriminator the aim is to reduce the output of the generator and the real data to a single value in the range $[0, 1]$ which refers to the probability of the data to be real or false.

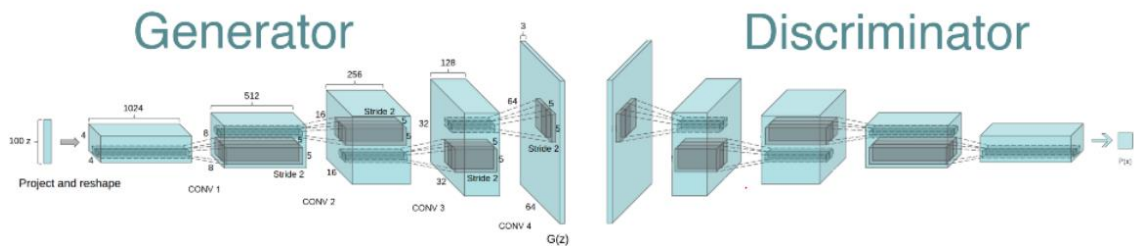


Figure 19: DCGAN Architecture

3.2.2. Conditional Generative Adversarial Networks

These are GANs in which the generator G and the discriminator D are conditional on certain additional information. This conditional information refers in general to labels in the data [17].

The generator uses the random noise vector z and the additional information y to generate dummy samples $G(z, c) = x|y$. Its goal is to produce samples as close to the real ones as possible given a certain condition. The discriminator receives the labelled real samples $(x|y)$ and the false samples produced by the generator $(x|y, y)$ and learns to discriminate real data. In this case, the loss function is defined as follows:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x, y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z, y)))] \quad (19)$$

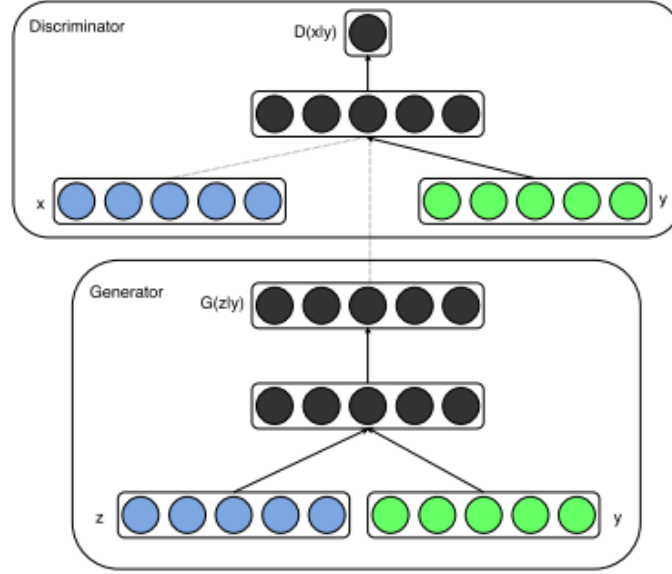


Figure 20: Conditional GAN Simple Architecture

3.2.3. Cycle GAN

CycleGANs emerged in 2017 with the publication of the paper Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks [18] by a group of researchers at the Berkeley AI Research (BAIR) laboratory.

This architecture makes it possible to transform images from one domain to another. To achieve this, they use an architecture that consists of two GANs, i.e., they have a total of two generators and two discriminators.

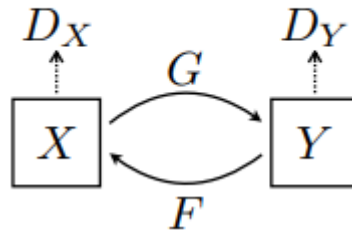


Figure 21: Cycle GAN Structure

The first generator G converts images from the X domain to the Y domain, while, conversely, the second generator F produces images from the Y domain to the X domain.

On the other hand, the first discriminator D_X is trained to be able to determine real Y-domain images from those produced by generator G. The second discriminator D_Y is trained to be able to distinguish real X-domain images from those produced by generator F.

3.2.3. Pix2Pix

The Pix2Pix model uses conditional generative adversarial networks to start from a first image and obtain a new image, thus transferring the pixels of the first image to a new domain. This model has a wide range of applications. It can be used both for converting a traced drawing into a real image and, for example, for restoring black and white photographs to a colour image.

To achieve these results, the Pix2Pix model, uses a decoder through which the information is introduced. By means of an inverted convolutional network, this information is increased layer by layer until the pixels of the image to be generated are predicted. But, in this case, the output image is conditioned to a first input image, which must also be analysed by a neural network. To achieve this, an encoder must be connected before the decoder which, as with convolutional networks, will process the image, eliminating the information that is not representative through layers of filters and max polling, and keeping the most representative characteristics of the image. This architecture in which the output of an encoder is connected to the input of a decoder is known as hourglass architecture, which is the shape shown in figure 22.

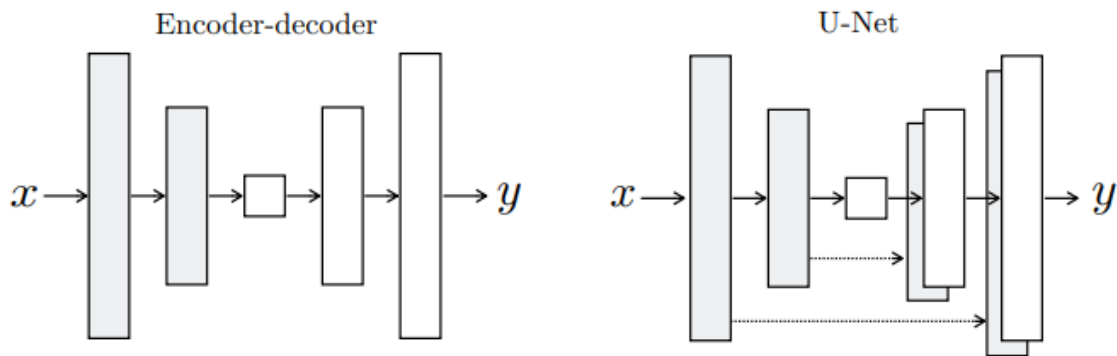


Figure 22: pix2pix Generator

The architecture of the pix2pix configuration discriminator is inspired by the architecture of a PatchGAN, which is a convolutional network that, instead of arriving at the final result of a single value, returns different patches of the original sample, thus obtaining the probability that an image is real or fake by patches, hence its name.

3.3. Applications of GANs

In this section we will look at some of the applications of GANs.

- Image generation: Generative networks can be used to generate realistic images after being trained on sample images. For example, if we want to generate new images of animals or flowers, we can train a GAN on thousands of sample images of animals or flowers. Once the training is finished, the generating network will be able to generate new images that are different from the images in the training set. Image generation is used in marketing, logo generation, entertainment, social networking, etc.
- Text-to-image synthesis: generating images from text descriptions is an interesting use case for GANs. This can be useful in the film industry, as a GAN is able to generate

new data based on some text you have invented, as well as in the comic industry, making it possible to automatically generate sequences of a story.

- Facial ageing: This can be very useful for both the entertainment and security industries. It is particularly useful for facial verification because it means that a company does not need to change its security systems as people age. An age-cGAN network can generate images at different ages, which can then be used to train a robust model for facial verification. In the case of entertainment, ageing an actor saves a lot of make-up time.
- Image-to-image translation: Image-to-image translation can be used to convert images taken during the day into images taken at night, to convert landscapes at different times of the year, to convert sketches into a final product, to copy image styles or to convert satellite images into different types of maps.
- Generating high-resolution images: If you have images taken with a low-resolution and/or old camera, GANs can help you generate high-resolution images without losing any essential detail.

In addition to the applications mentioned above, there are many others of great interest that have been and continue to be studied, such as video synthesis [19], which could be used to generate content in less time than would be required if the task were done manually, or video frame prediction [20-22].

3.4. Disadvantages of the GANs

GANs present several challenges to circumvent during training [13] and are currently the subject of research. Among them, the most usual problems that arise in the training of a GAN are:

- Non-convergence: the generator and the discriminator fail to reach an equilibrium. The loss function of the generator and discriminator start to oscillate without being able to achieve long-term stability. While it is common in GANs for the loss functions to oscillate initially, as training progresses, the goal is to achieve stability. When this does not occur, samples are produced by the generator, but their quality does not improve.
- Modal collapse: This occurs when the generator produces similar samples, even though the inputs are of very different characteristics. The generator finds that a small set of samples misleads the discriminator and is then unable to produce others. In these cases, the gradient of the loss function is stuck at a value close to 0.
- Uninformative loss: Although it may seem natural to think that the lower the loss of the generator, the higher the quality of the samples it produces, this is not so immediate. The loss of the generator must be compared to that of the discriminator, which is constantly improving. Therefore, it is not so straightforward to assess the improvement of the model. The generator could be producing higher and higher quality samples, even if the loss function is increasing.

For example, the Wasserstein GAN (WGAN) or the Wasserstein GAN-Gradient Penalty (WGAN-GP) [14, 15] are two approaches to improve the instability of GANs and partially deal with the problem of modal collapse. However, this is currently an open research topic.

4. Dataset

This section will give information about the datasets used to perform the GAN training. The datasets used for the project are the MIT-BIH Arrhythmia Database and the MIT-BIH Noise Stress Test Database. Both databases have been established by the Massachusetts Institute of Technology and Beth Israel Hospital.

Both databases are hosted by the Research Resource for Complex Physiologic Signals (PhysioNet) repository. This repository was established in 1999 under a grant from the National Institutes of Health (NIH). The main goal of PhysioNet Resource is to conduct and catalyse biomedical research and education by providing free access to large collections of physiological and clinical data and related open-source software.

The PhysioNet platform is managed by members of the MIT Computational Physiology Laboratory. The other core lab of PhysioNet Resource is the Margret and H.A. King Institute for Nonlinear Dynamics at Beth Israel Deaconess Medical Center.

4.1. MIT-BIH Arrhythmia Database

The MIT-BIH Arrhythmia Database dataset contains 48 half-hourly extracts of two-channel ambulatory ECG recordings. The recordings were digitised at 360 Hz per channel with 11-bit resolution over a 10 mV range.

It should be noted that the recordings are accompanied by annotations in .txt format, which contain information on each beat, the most relevant of which are the type of beat, the starting point of the beat in time format (mm:ss.000, where 000 indicates thousandths of a second) and the initial sample number of the beat (see table 1).

Time	Sample #	Type
0:00.058	21	+
0:00.736	265	N
0:01.597	575	N
0:02.433	876	N

Table 1: Extract of the annotations file

4.2. MIT-BIH Noise Stress Test Database

The MIT-BIH Noise Stress Test Database contains the three most common noise types in ECGs. Each noise type was added using two clean records from the previous database, specifically record 118 and 119. Recall that the three most common noise types are:

- Baseline wander (BW)

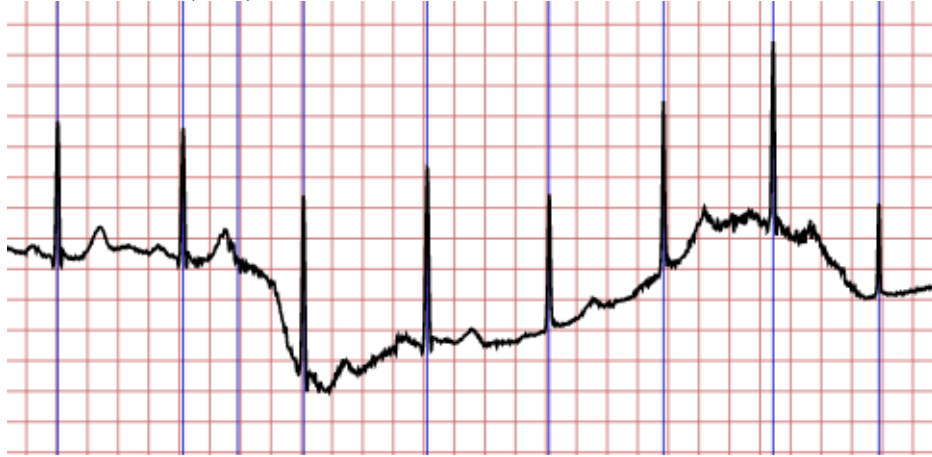


Figure 23: Signal with BW noise. Extract of the database.

- Muscle Artifact (MA)

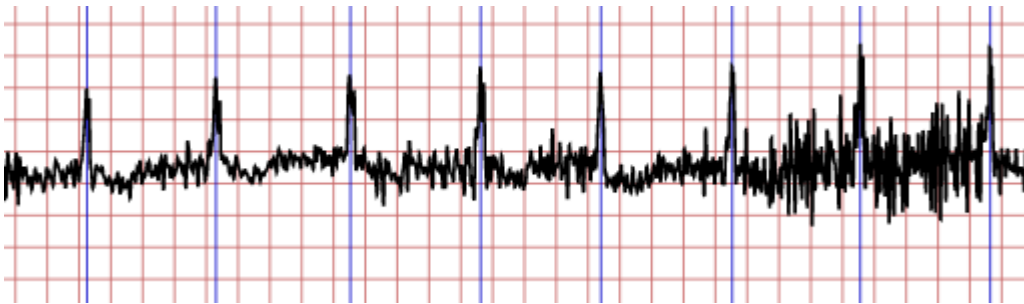


Figure 24: Signal with MA noise. Extract of the database

- Electrode motion artifact (EM)

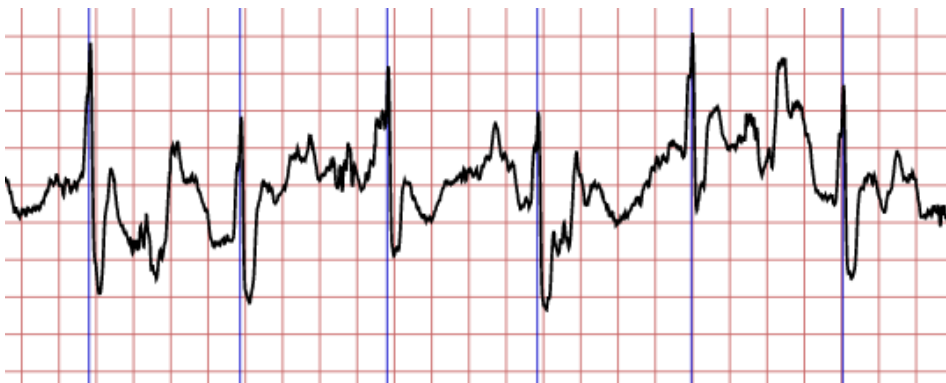


Figure 25: Signal with EM noise. Extract of the database.

It should be noted that, apart from having records 118 and 119 with the noise added, the three types of noise are available individually, without having been added to any record, i.e., the different types of EM, MA and BW noise are provided individually in the database.

4.3. Database processing

Concerning the first analysis of the data, clean fragments were manually selected from the records available in the MIT-BIH Arrhythmia Database. Fragments without noise were detected and separated from those containing noise. This process has been possible thanks to the graphical representation tool of the PhysioNet platform and the annotation file that comes with each signal, and has been carried out in the following way:

1. Identify the usable fragments from the Physionet graphical platform.
2. Identify the value of the time axis (mm:ss.000).
3. Cut out the fragment with noise from the annotation file by looking at the Time data.

Once the fragments without noise have been identified, they have been loaded into the Google Colab environment to start working with them.

In order to obtain the heartbeat, a 320-sample size window has been applied starting from the point indicated in the annotation file. Samples like the one in figure 26 have been obtained.

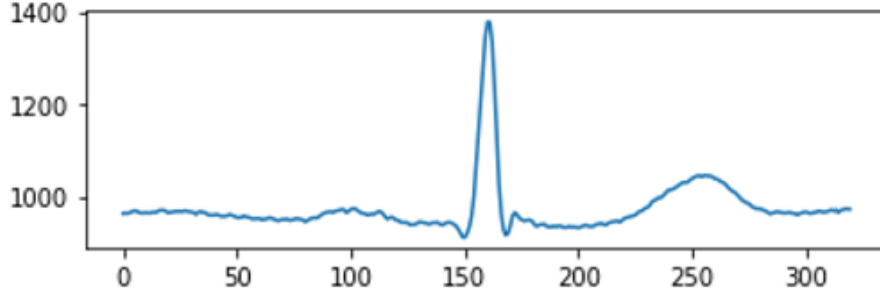


Figure 26: Heartbeat sample

These ECG fragments have been normalised taking into account the learning characteristics of the implemented model. The ECG signals have been normalised according to equation (20):

$$X_{i_{normalized}} = \frac{X_i - X_{min}}{X_{max} - X_{min}} \quad (20)$$

Where X_{max} and X_{min} represent the maximum and minimum of the signal, respectively.

Next, 5 contiguous beats have been joined together to obtain a more realistic ECG signal as it is shown in figure 27.

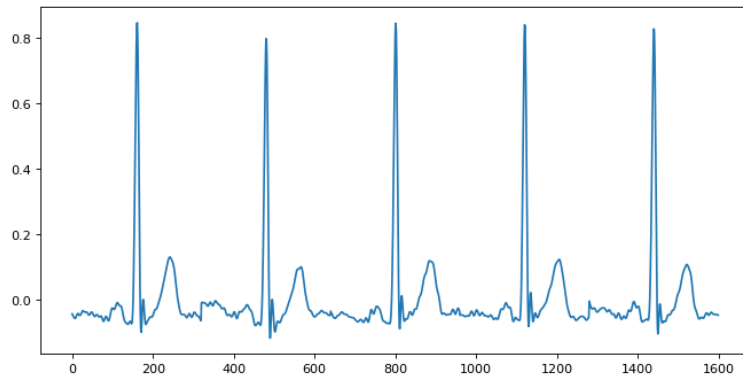


Figure 27: Sample ECG signal

Finally, the noise records from the MIT-BIH Noise Stress Test Database have been processed to add them to the clean signals obtained in the previous process. At this stage, the data had to be transformed into .csv files using the WFDB library recommended by the data provider in order to be able to process them more comfortably in Python.

In order for the model to be efficient and robust with the three types of noise, each of the noises has been added individually and also the possible combinations between them, as will be seen in the images below.

- Signal corrupted with MA noise

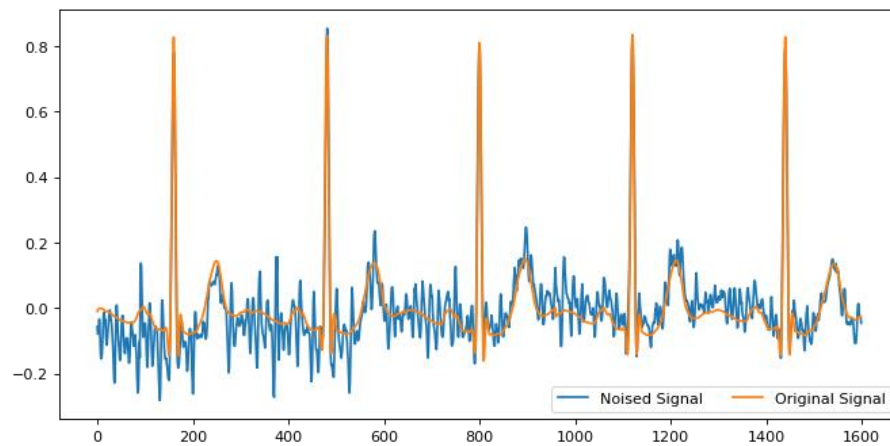


Figure 28: Signal corrupted with MA noise

- Signal corrupted with EM noise

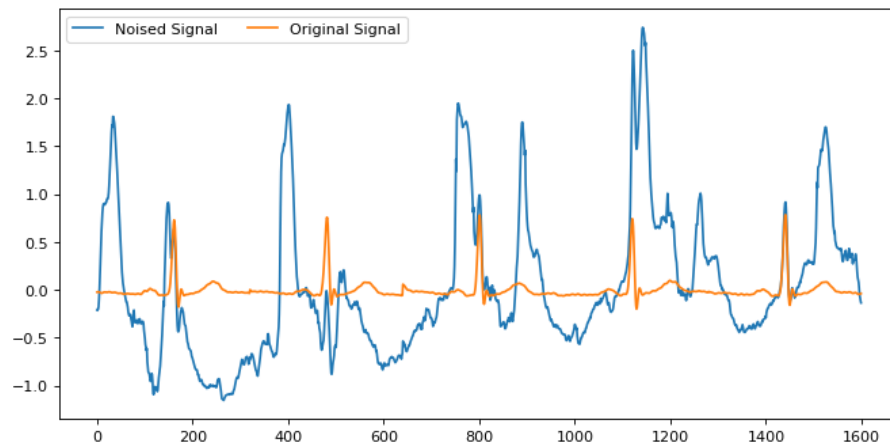


Figure 29: Signal corrupted with EM noise

- Signal corrupted with BW noise

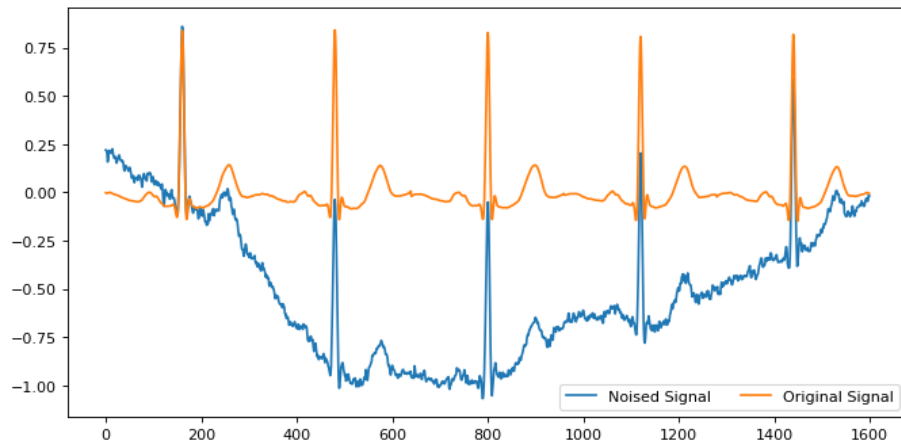


Figure 30: Signal corrupted with BW noise

- Signal corrupted with MA + EM noises

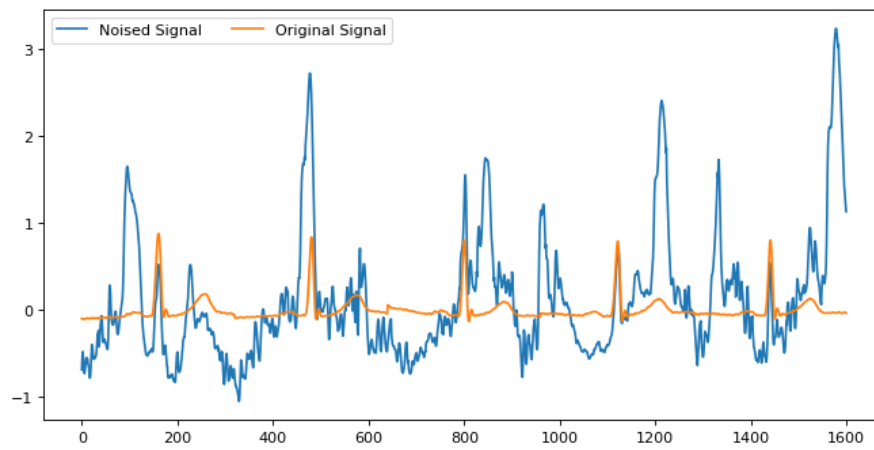


Figure 31: Signal corrupted with MA + EM noises

- Signal corrupted with MA + BW noises

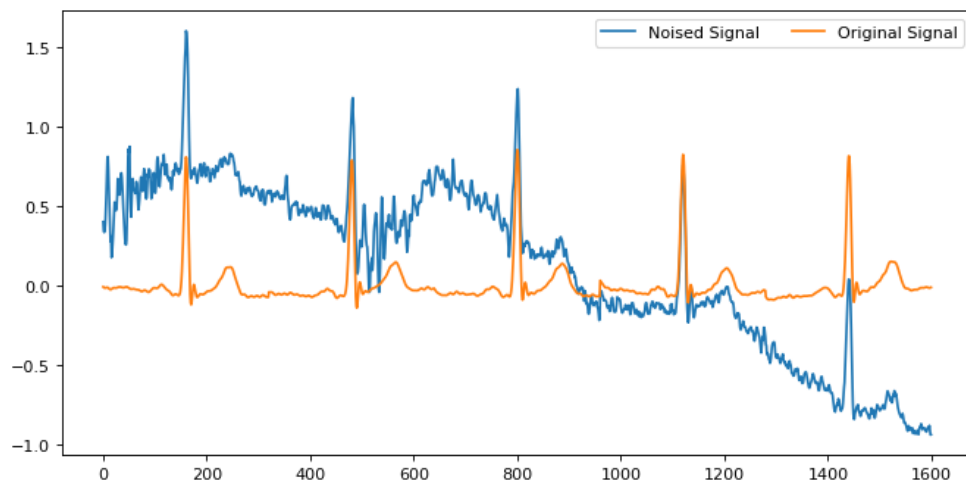


Figure 32: Signal corrupted with MA + BW noises

- Signal corrupted with BW + EM noises

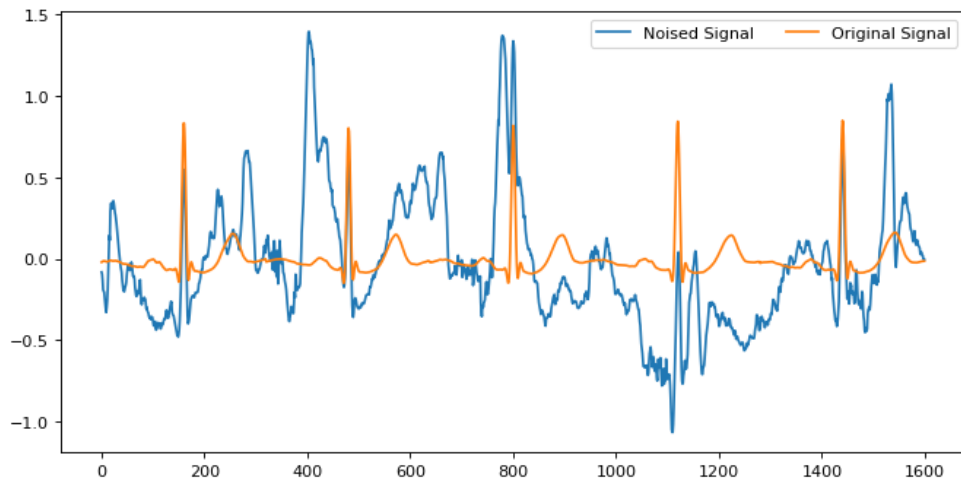


Figure 33: Signal corrupted with BW + EM noises

- Signal corrupted with MA + EM + BW noises

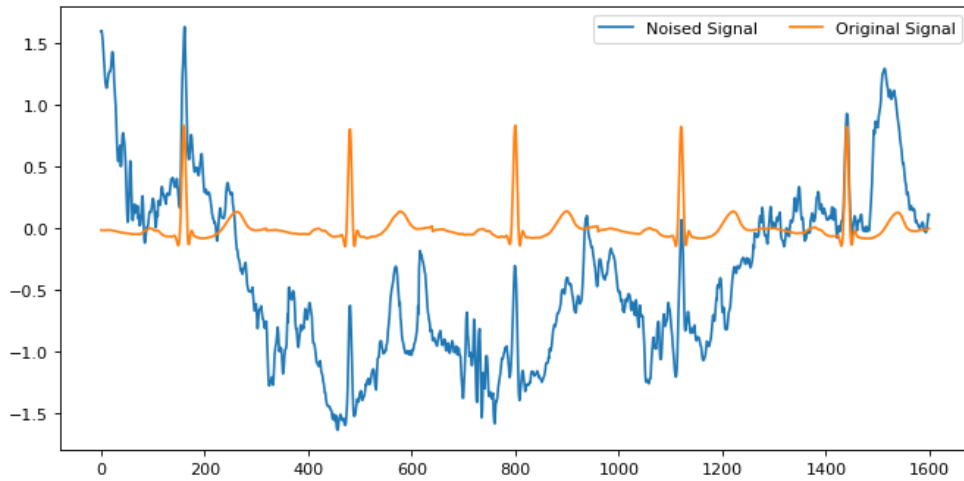


Figure 34: Signal corrupted with MA + EM + BW noises

Comparing the ECG signals with noise obtained in this section with those of section 4.2. MIT-BIH Noise Stress Test Database, it can be concluded that we have reliable data to proceed with the design and implementation of the GAN model.

At the end of the data processing, around 25,000 ECG signals with 5 heartbeats each have been obtained. For training, 90% of the samples have been used while for testing the remaining 10%. The signals from the MIT-BIH Arrhythmia Database from which samples have been extracted are: 103, 105, 111, 116, 122, 205, 213, 219, 223, 230, which are the ones with the least noise and therefore we could get more clean samples.

5. GAN Implementation

In this section, the implementation of the system and the technologies used will be explained in detail.

5.1. Implemented GAN structure

The model on which the GAN configuration of this thesis has been based is the pix2pix configuration, in which modifications have been made to deal with the desired data, ECG signals. As mentioned above, this configuration is a type of Conditional GAN network. The reason why the pix2pix configuration has been chosen is because of its wide use in image restoration applications and it uses Convolutional Neural Networks. While in the original model 2D convolutional layers are used when dealing with images, in our application we have used 1D convolutional layers due to the nature of the data, since they are signals stored in $n \times 1$ vectors, where n is the size of the vector. For beats $n = 320$ while for signals with five beats $n = 1600$, the dimension of 5 beats has been used to work with cases more similar to a real ECG signal.

As discussed above, the main architecture of the GAN requires a generator network (Generator) and a discriminator network (Discriminator), the overall structure of the GAN can be shown in figure 35. In the following, it will be shown how these components of the model implemented for this thesis look like.

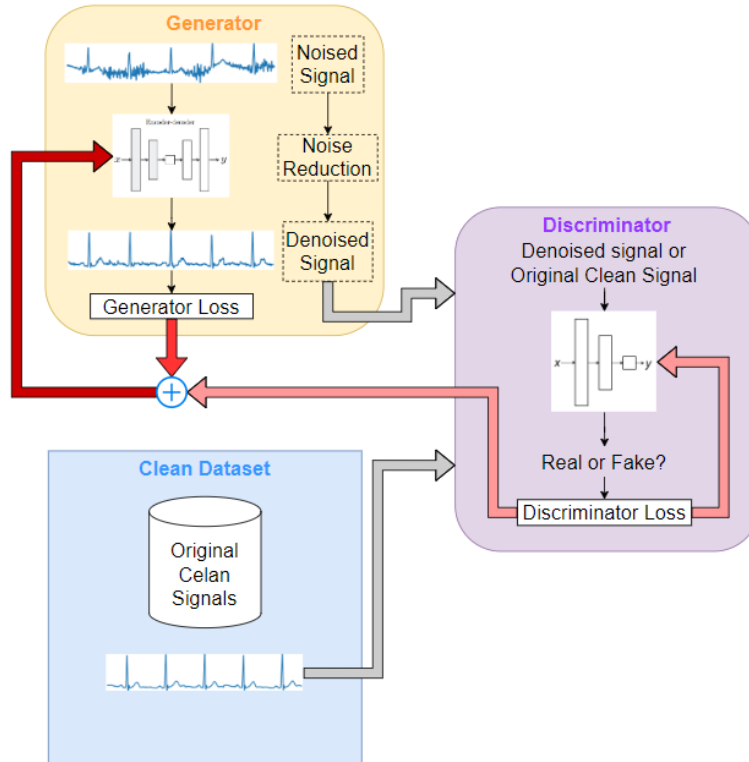


Figure 35: Overall GAN Structure

5.1.1. Generator

The generator network is inspired by the architecture of an Encoder-decoder network, which is very similar to the U-Net architecture, but without hop connections. It is important to note that both architectures have been tested with better results using the Encoder-decoder configuration.

The generator is composed of two parts, the encoder, which compresses the information, and the decoder, which decompresses the information.

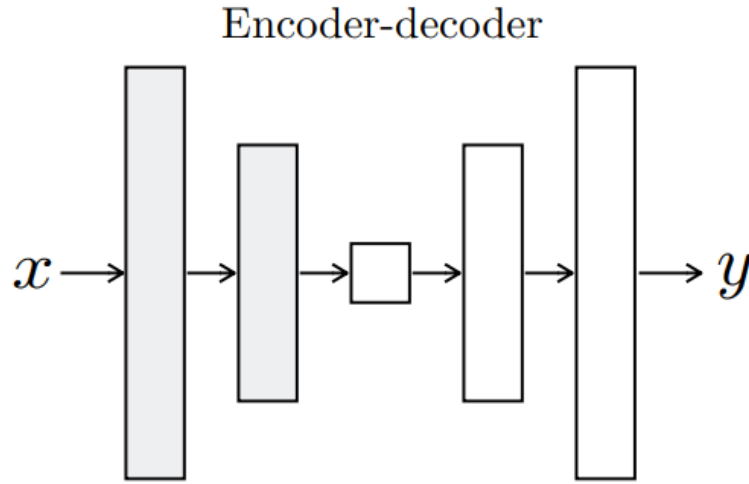


Figure 36: Encoder-decoder architecture

5.1.1.1. Encoder

This stage consists of 6 downsample blocks. Each downsample block has a 1D convolutional layer, a Batch Normalization layer and a LeakyReLU activation function (see image 37). Note that the first block is the only one without a Batch Normalization layer as the input data is already normalised.

The input x at this stage is the signal with size noise ($BatchSize \times 1600 \times 1$). The output y must have the size ($BatchSize \times 25 \times 512$), being a feature tridimensional vector of the signal, where the *BatchSize* dimension represents the number of signals treated at the same time.

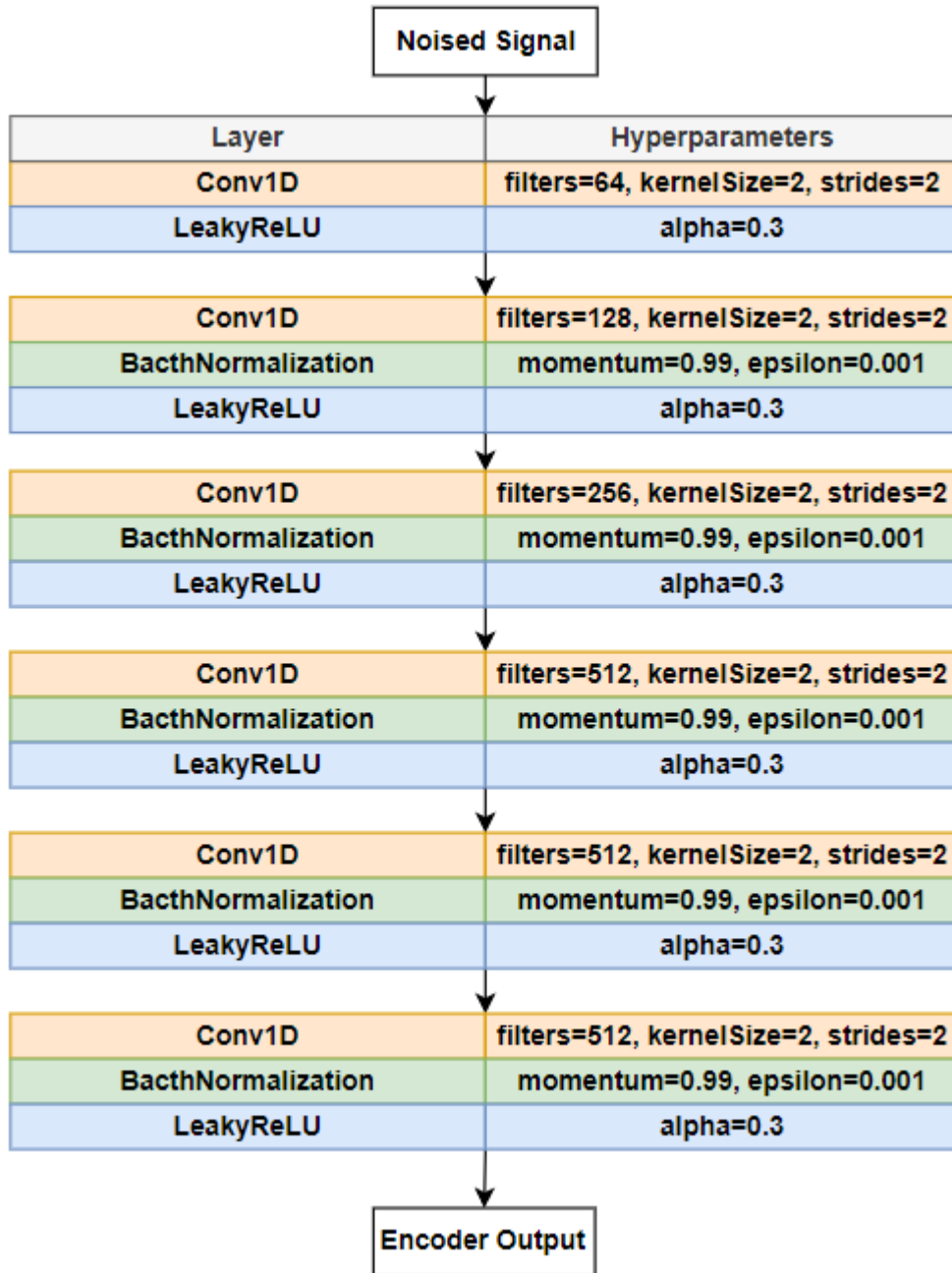


Figure 37: Encoder

5.1.1.2. Decoder

This stage is composed of five upsample blocks. Each block contains a 1D transposed convolutional layer, a Batch Normalization layer, a Dropout layer (for the first two blocks) and the ReLU activation function. The Dropout layer helps to reduce overfitting as nearby neurons tend to learn patterns that are related, and these relationships can form a very specific pattern with the training data, with Dropout this dependency between neurons is less throughout the neural network, so neurons need to work better alone and not rely so much on relationships with neighbouring neurons. In this case, the Dropout has a ratio of 0.5, indicating that half of the neurons will be deactivated.

In this case, the input size will be ($BatchSize \times 25 \times 512$) and the output will have a size of ($BatchSize \times 800 \times 128$). The output of this stage is not yet that of the generator.

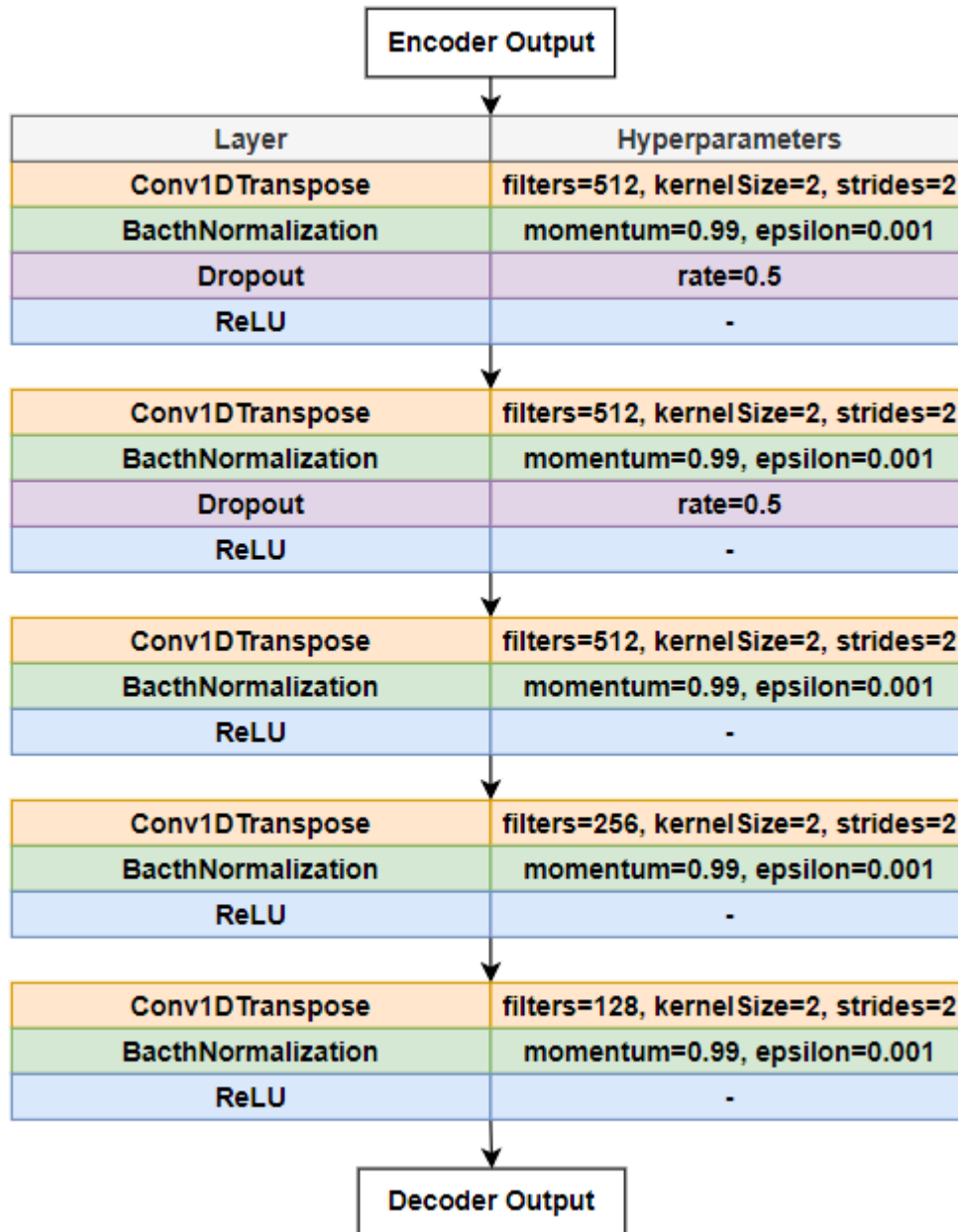


Figure 38: Decoder

The output of the generator would already be the prediction of the generator with respect to the input signal, which has a size of ($BatchSize \times 1600 \times 1$). To achieve this output, a final layer has been added to the decoder output, see picture 38.

As can be seen, the network is composed of convolutional layers, this is because this type of layer helps to obtain the most important characteristics of the signal.

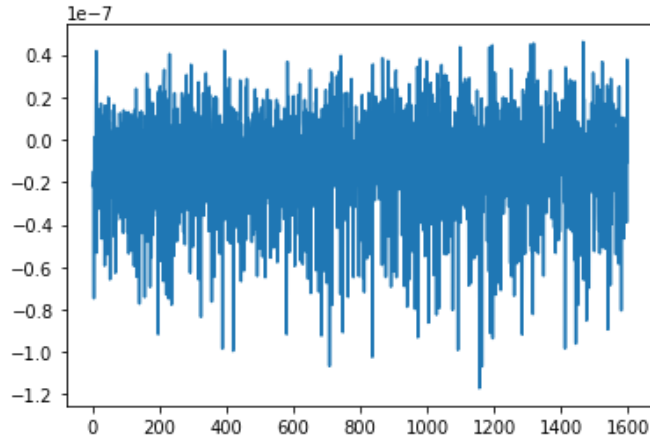


Figure 39: Generated output without training

5.1.2. Discriminator

The architecture of the original pix2pix configuration discriminator is inspired by the architecture of a PatchGAN, which is a convolutional network that, instead of arriving at the final result of a single value, returns different patches of the original sample. In this case, a convolutional network has been used, obtaining a single final value that indicates the probability that the value is true or false. As it can be shown, the last layer is a Sigmoid, in order to obtain the probability.

In this case the input has the size of the signal ($BatchSize \times 1600 \times 1$) while the output ($BatchSize \times 1 \times 1$).

As mentioned, the output contains the Discriminator's prediction of whether the input signal is true or false, i.e. if the input signal comes from the original data set of original ECG signals without noise, the predicted probability should be 1 or approximately 1. On the other hand, if the signal is the one generated by the Generator, which would be the one with the noise removed, it should be 0 or approximately 0. This will be explained in more detail in the next section.

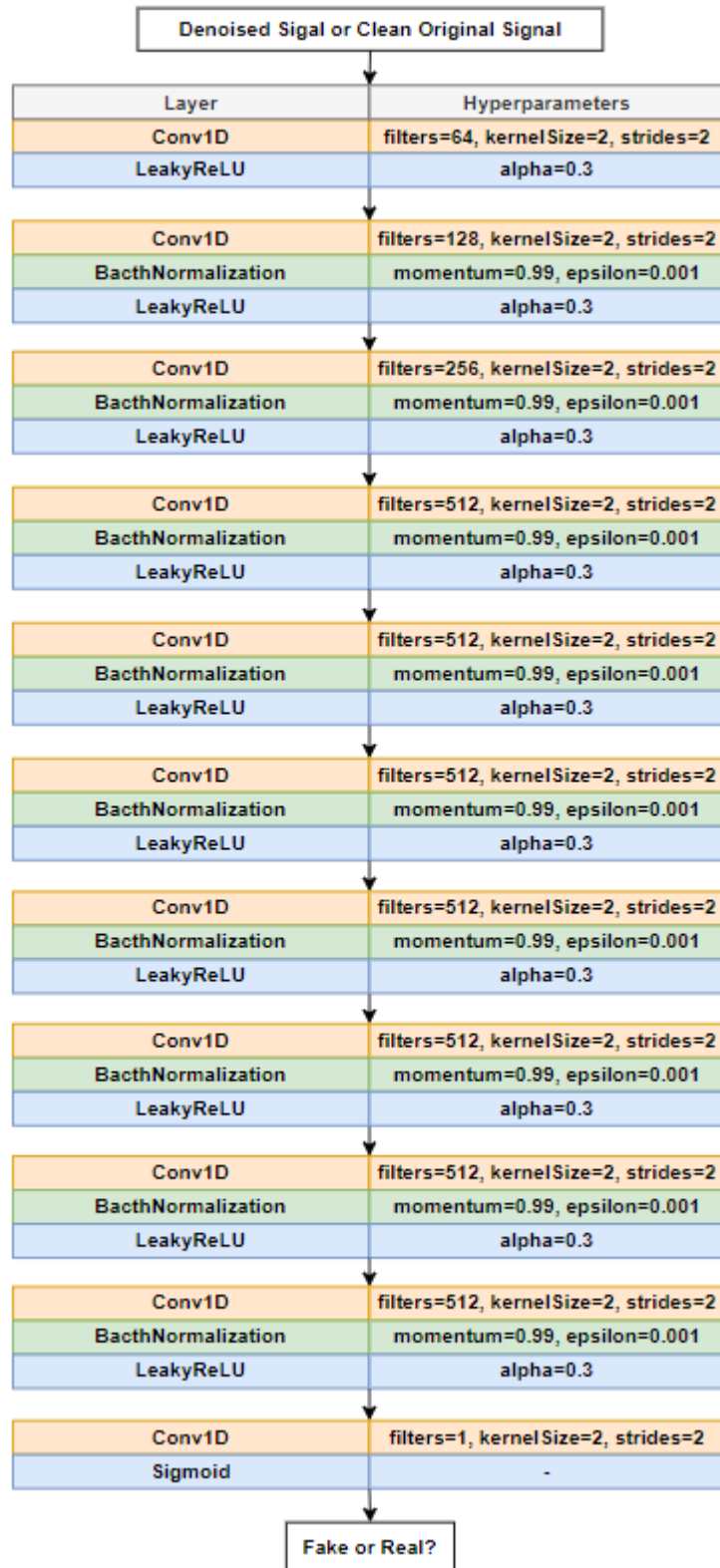


Figure 40: Discriminator Structure

5.2. Training Objective Functions

In this section we will present the loss functions used for both the generator and the discriminator.

The main objective is for the Discriminator to output a value close to 1 when evaluating the signal generated by the Generator, as this would indicate that the Generator has been able to "fool" the Discriminator and its signals are of good quality and resemble those of the original dataset. On the other hand, we also want the Discriminator to be able to learn more and more from its results in order not to be fooled by the Generator, so it will know when it has been "fooled" and will reconfigure its weights so that this does not happen again. To achieve this effect in our network, the following loss functions are calculated.

5.2.1. Generator Loss

The error of the Generator is obtained from three error functions. These three functions are the original Generator Loss, the difference between the original signal and the generated signal (the one with the noise removed) and finally the maximum error.

5.2.1.1. Generator original Loss

Concerning the original Generator loss, the error is calculated as described in (21), which is the Generator part of the original GAN loss function.

$$l_G = \log(1 - D(G(s))) \quad (21)$$

This error refers to the adversarial error, i.e., the BCE between the signal generated by the Generator and the result we expect from the Discriminator when it evaluates this signal. In order to fool the Discriminator, we expect it to be 1, which represents the probability of a real signal. So, if the signal generated by the Generator looks real, the error should be low.

5.2.1.2. Difference error

The aim of this error function is to calculate the difference between the generated (denoised signal) and the original signal, which is a useful metric to add to the total Generator loss. Equation (22) describes its calculation.

$$l_{diff} = \sqrt{\sum_{n=1}^N |s'_k - s_k|^2} \quad (22)$$

Where N is the number of samples of the signal and k is the specific sample of each signal being evaluated. The more similar the signals are, the lower the error should be.

5.2.1.3. Max Error

This loss function refers to the maximum difference between the generated signal and the original signal. When the error is low, it means that the generated signal resembles the original signal, since at the point of maximum signal error, it does not differ much from the original signal. The equation described is (23).

$$l_{max} = \max(|s'_1 - s_1|, |s'_2 - s_2|, \dots, |s'_k - s_k|) \quad (23)$$

Taking into account these loss functions, the total error for the Generator is as shown in equation 23.

$$L_{Gen} = l_G + l_{diff} + l_{max} \quad (24)$$

5.2.2. Discriminator Loss

The Discriminator error is obtained from the loss function of the original GAN, which for each sample the calculation can be obtained as described in (25). In this we have two error components.

$$L_{Disc} = - \left[\log(D(s)) + \log(1 - D(G(s))) \right] \quad (25)$$

For the first loss component, $D(s)$ represents the probability evaluated by the discriminator that s is a denoised ECG signal. If the Discriminator has predicted that the signal s is real ($D(s) \sim 1$), then the error will be low.

For the second component of the loss function, $G(s)$ represents the signal generated by the Generator (the denoised signal), therefore, $D(G(s))$ represents the probability evaluated by the discriminator that $G(s)$ is a denoised ECG signal. In this case, if the Discriminator has predicted that the signal was false, i.e., an ECG signal with noise or generated by the Generator, ($D(G(s)) \sim 0$) then the error should be low.

5.3. Training step

The neural network was trained for 500 epochs with a batch size of 128. The training time was 5 hours.

The optimiser used to update the weights according to the loss functions is Adam, whose parameters are $\beta_1 = 0.0001$ and $\beta_2 = 0.5$. On the other hand, the learning ratio is 0.0001. The learning ratio controls the updating of the gradient coefficients. If the learning ratio is too large, the changes in the weights will also be too large and it will be difficult to find the coefficients that minimise the cost function.

The following steps describe how the GAN is trained for each batch, i.e. these steps are performed during every batch the 500 epochs of training:

1. The Generator makes its prediction by taking as input a signal with. The first attempts to eliminate the noise will be unfavourable.
2. The Discriminator checks whether the signal predicted by the Generator looks real or not. It does the same for the input signal to the Generator.
3. The losses of the Generator are calculated.
4. The gradients are accessed, and the weights are optimised, i.e. there is backpropagation.

The evolution of the losses for both networks can be seen in figure 40, in which it can be observed that approximately at epoch 175, the losses of both networks undergo a drastic change, giving us an idea of up to which epoch the network is able to learn

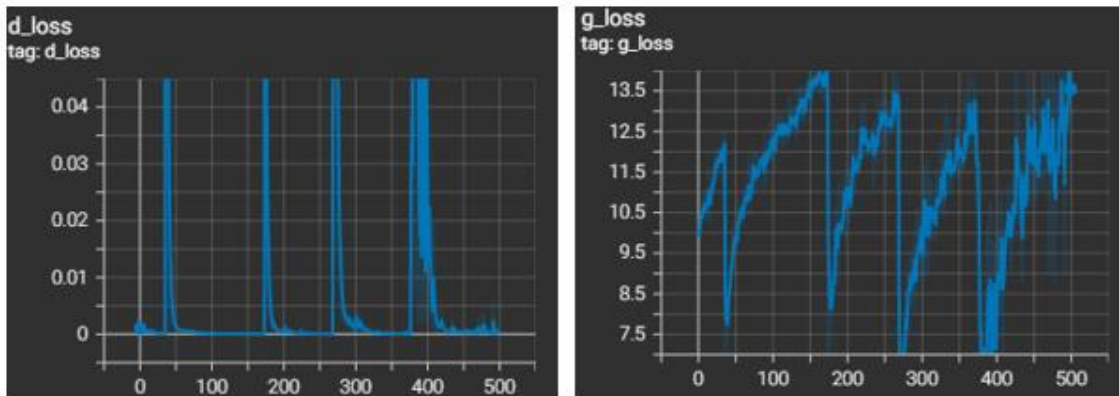


Figure 41: Evolution of the Generator and Discriminator Losses

5.4. Results

This section shows the results obtained for each type of noise.

Regarding the MA and BW noises, the results are quite good, since the signals from which the noise has been removed come to resemble the original signal, while for the EM type of noise the results have not been optimal. Neither for those noises containing the EM type. In the literature we saw that researchers had difficulties in dealing with this type of noise.

In the following, we will show the tests performed for different noise levels and different types of noise. In these tests the magnitude of the noise has been modified so that we will see the results for 0.2, 0.4, 0.6, 0.8 and 1 of the original noise. In these tests it can be seen that as the noise increases, the noise suppression loses efficiency. Only the worst case, when the noise has its normal magnitude, and the best case, when the noise is multiplied by 0.2, will be shown. The rest can be seen in Annex 1.

- Denoising signal corrupted by MA noise

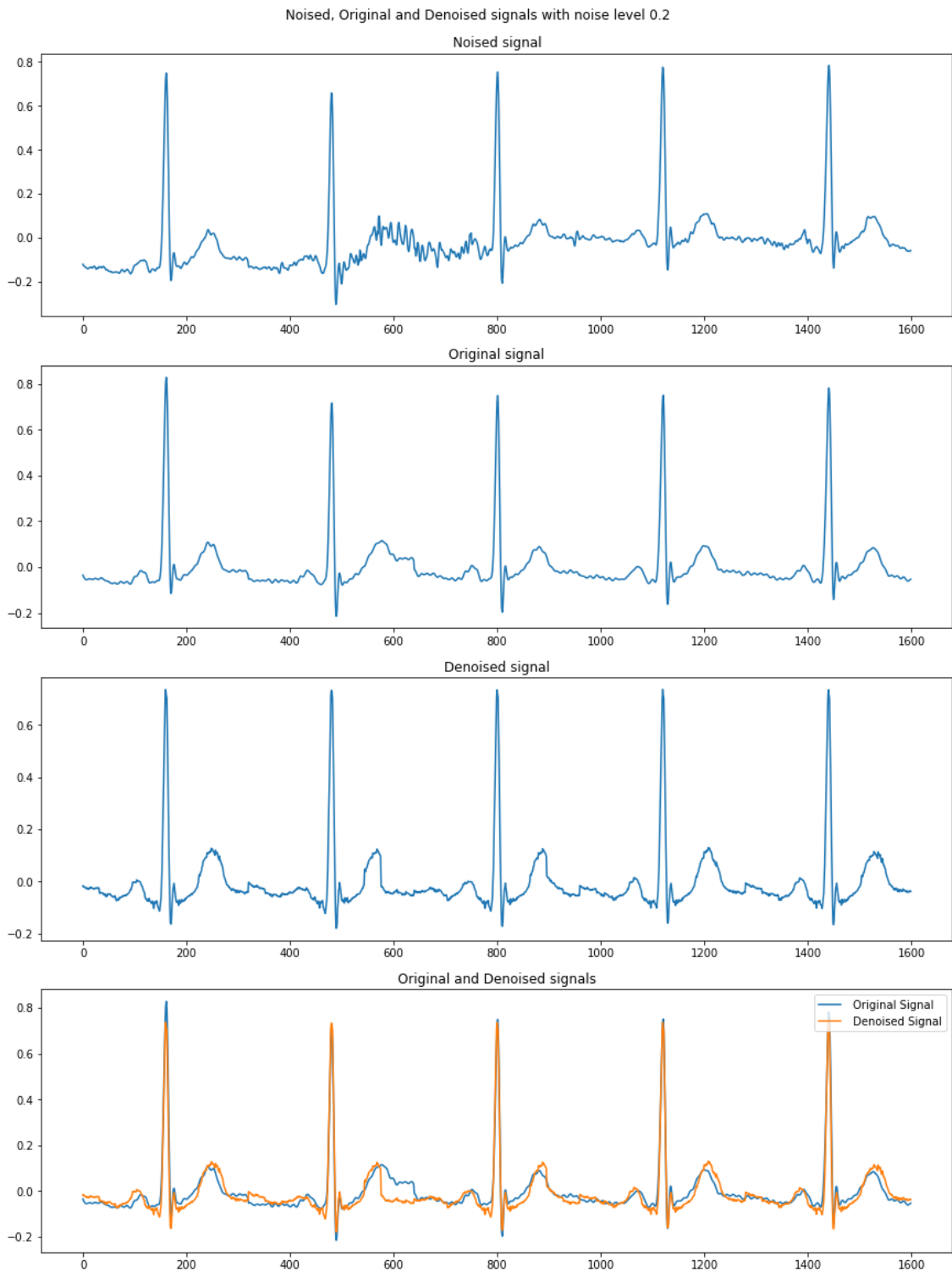


Figure 42: Denoising signal corrupted by MA noise at level 0.2

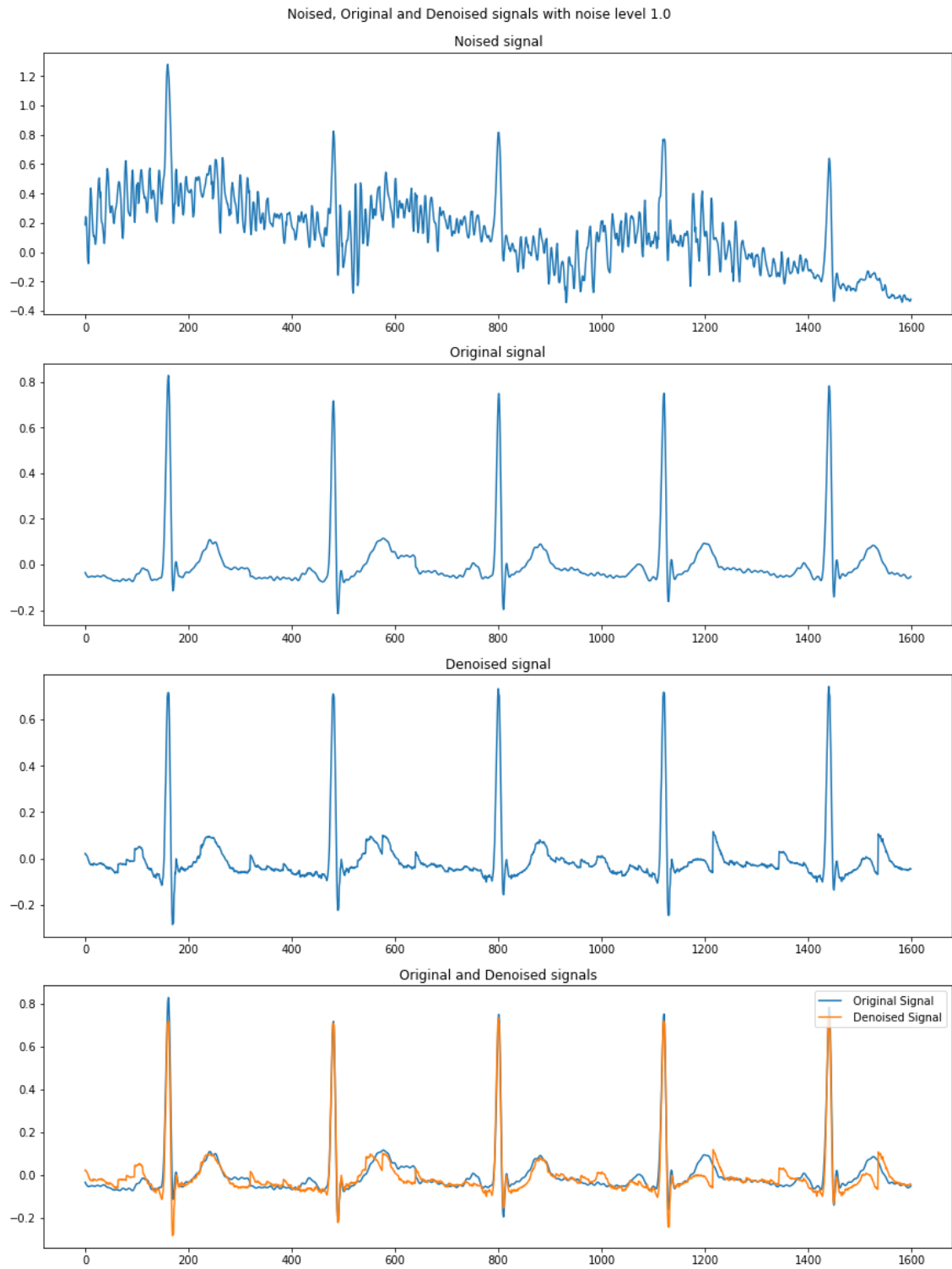


Figure 43: Denoising signal corrupted by a real sample of MA noise

- Denoising signal corrupted by BW noise

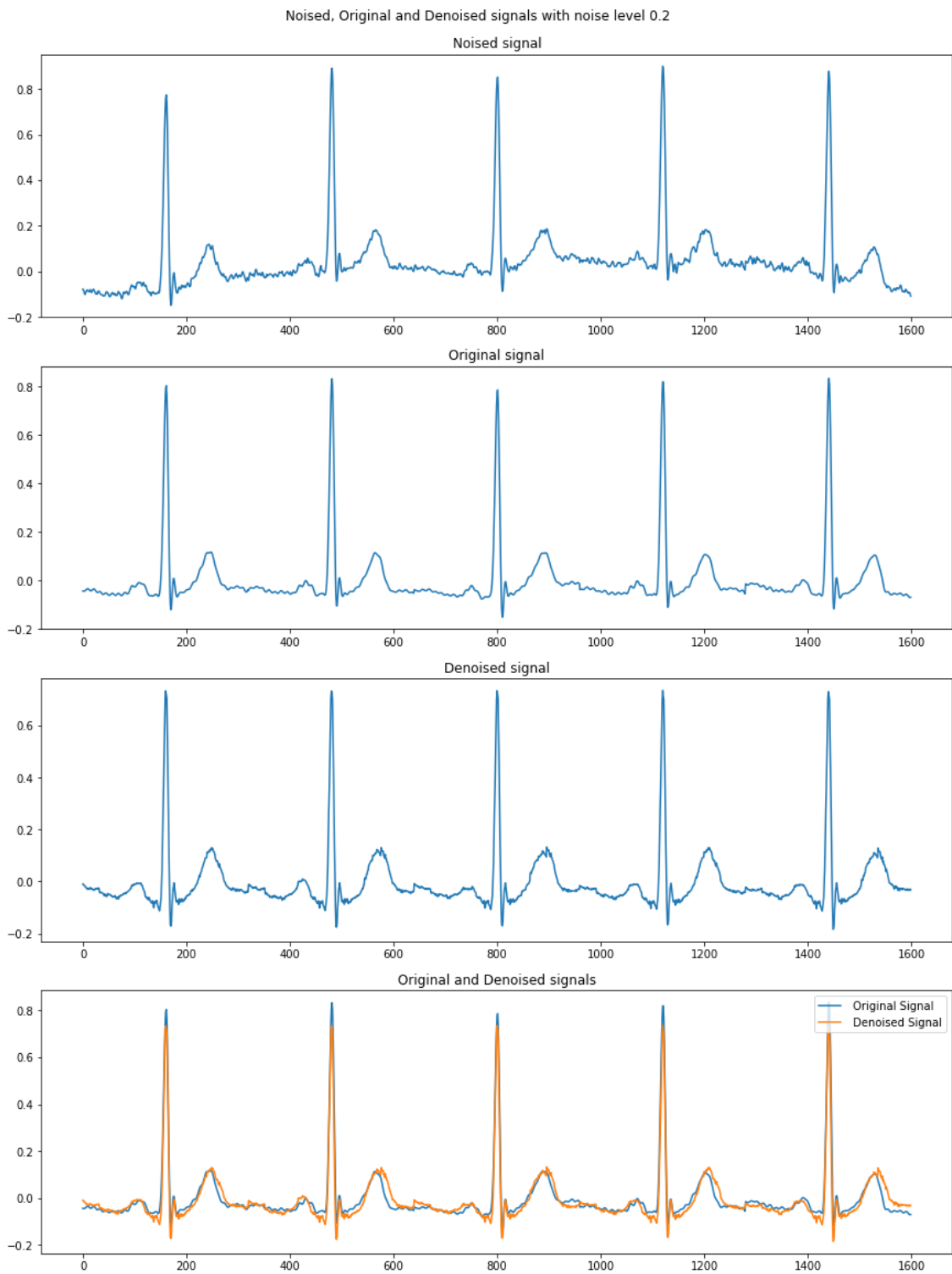


Figure 44: Denoising of signal corrupted by BW noise at 0.2 level of noise

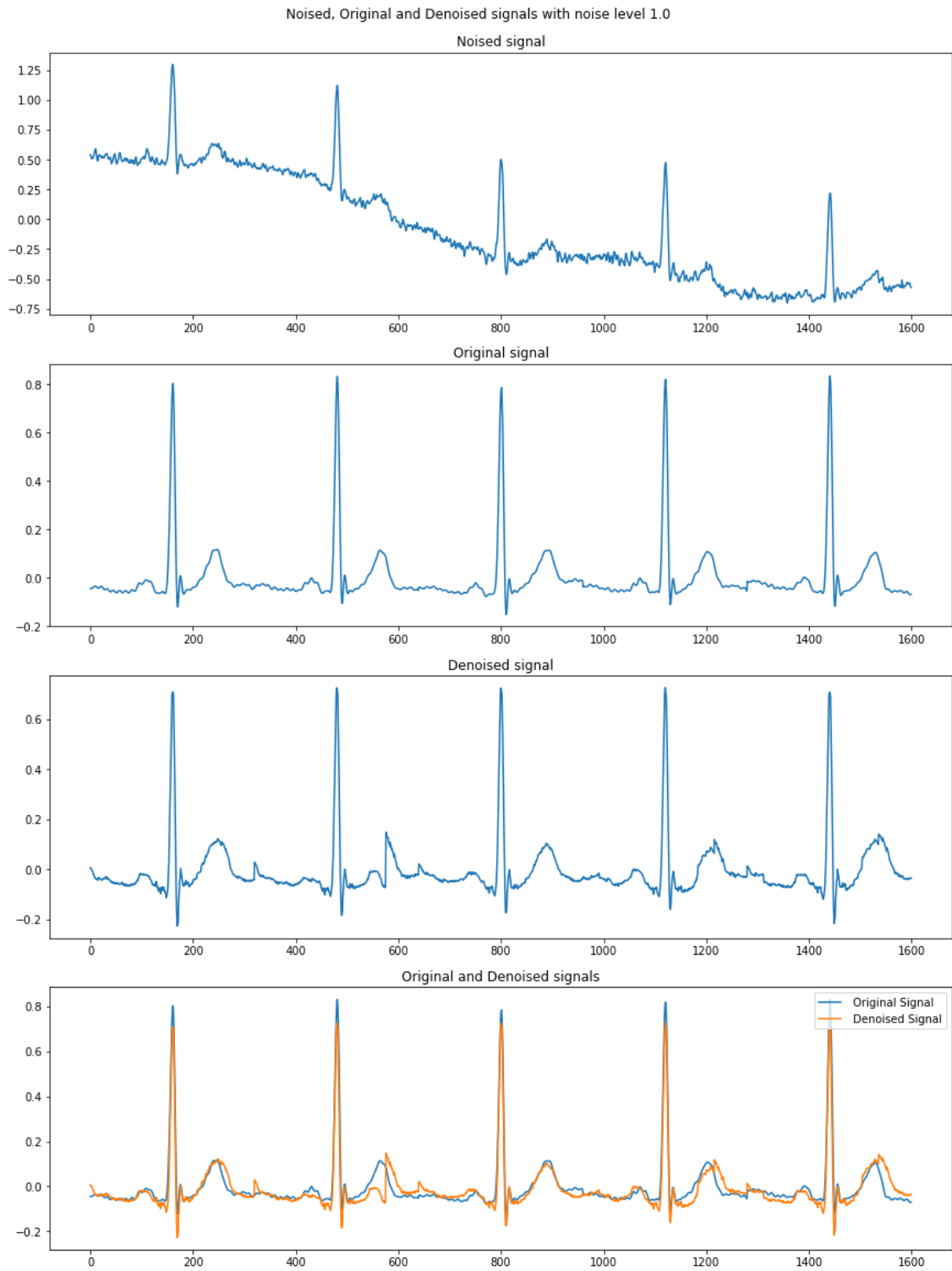


Figure 45: Denoising signal corrupted by a real BW noise sample

- Denoising signal corrupted by EM noise

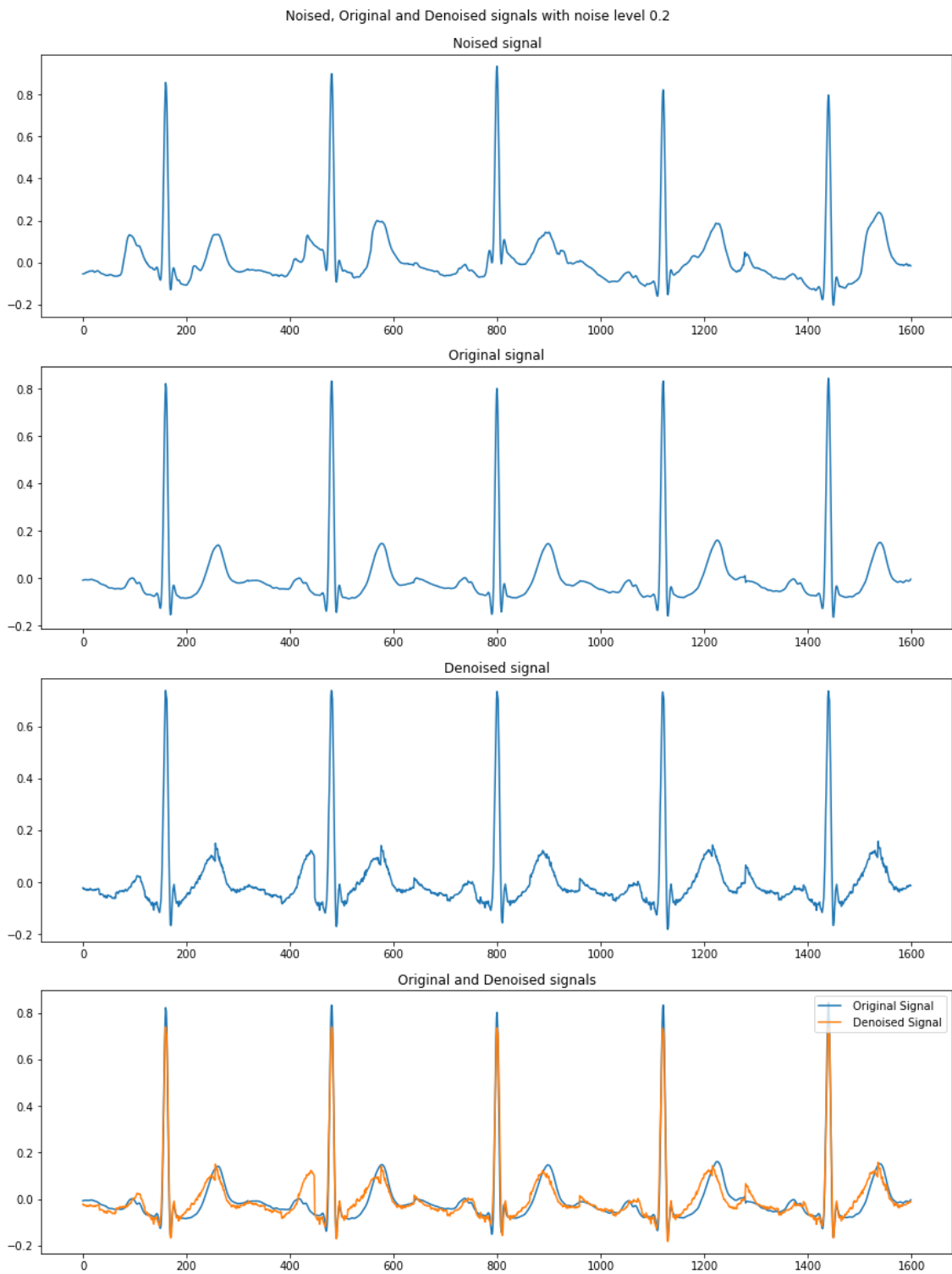


Figure 46: Denoising signal corrupted by EM noise at 0.2 level

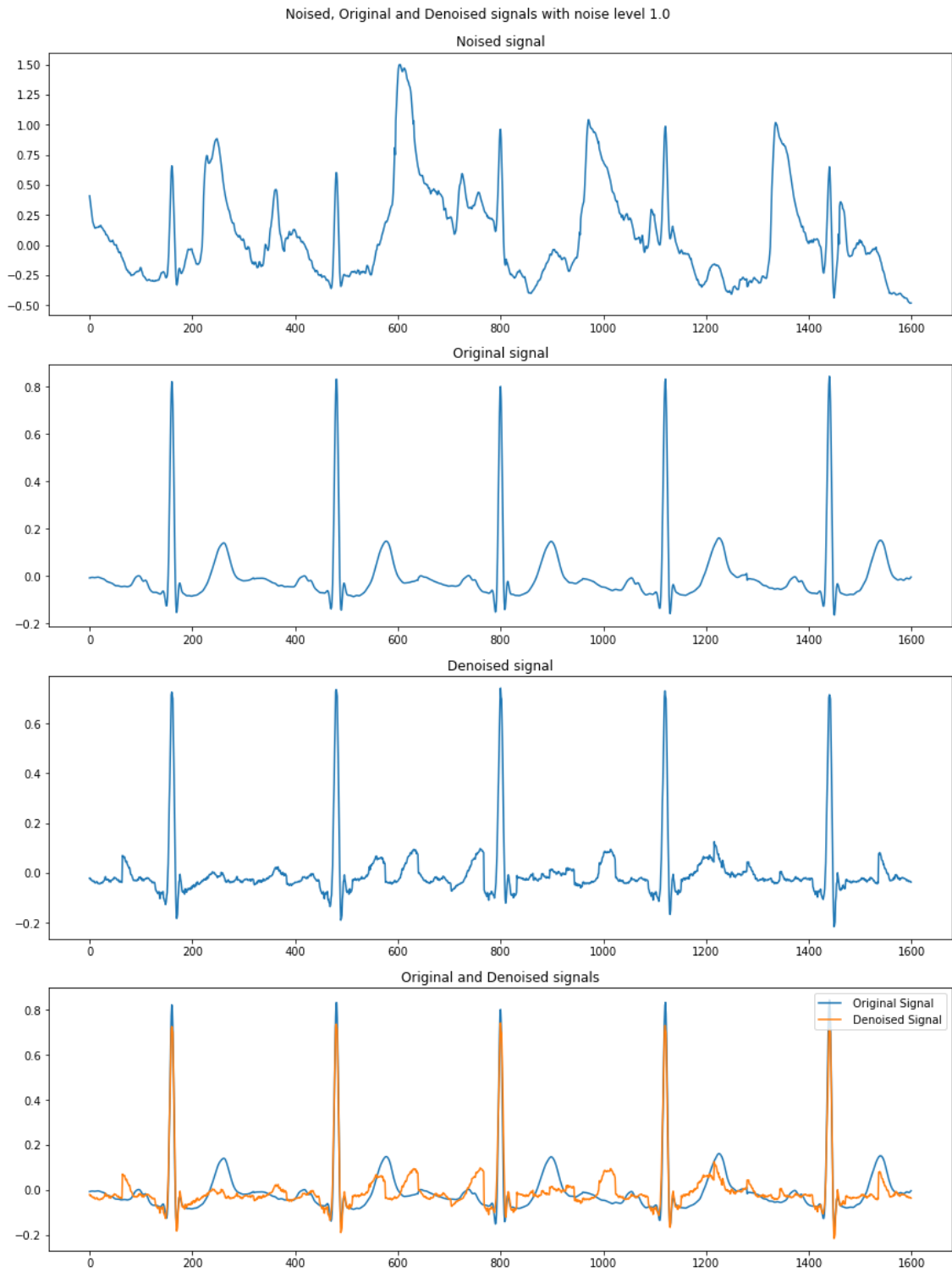


Figure 47: Denoising signal corrupted by a real sample of EM noise

- Denoising signal corrupted by MA + BW noises

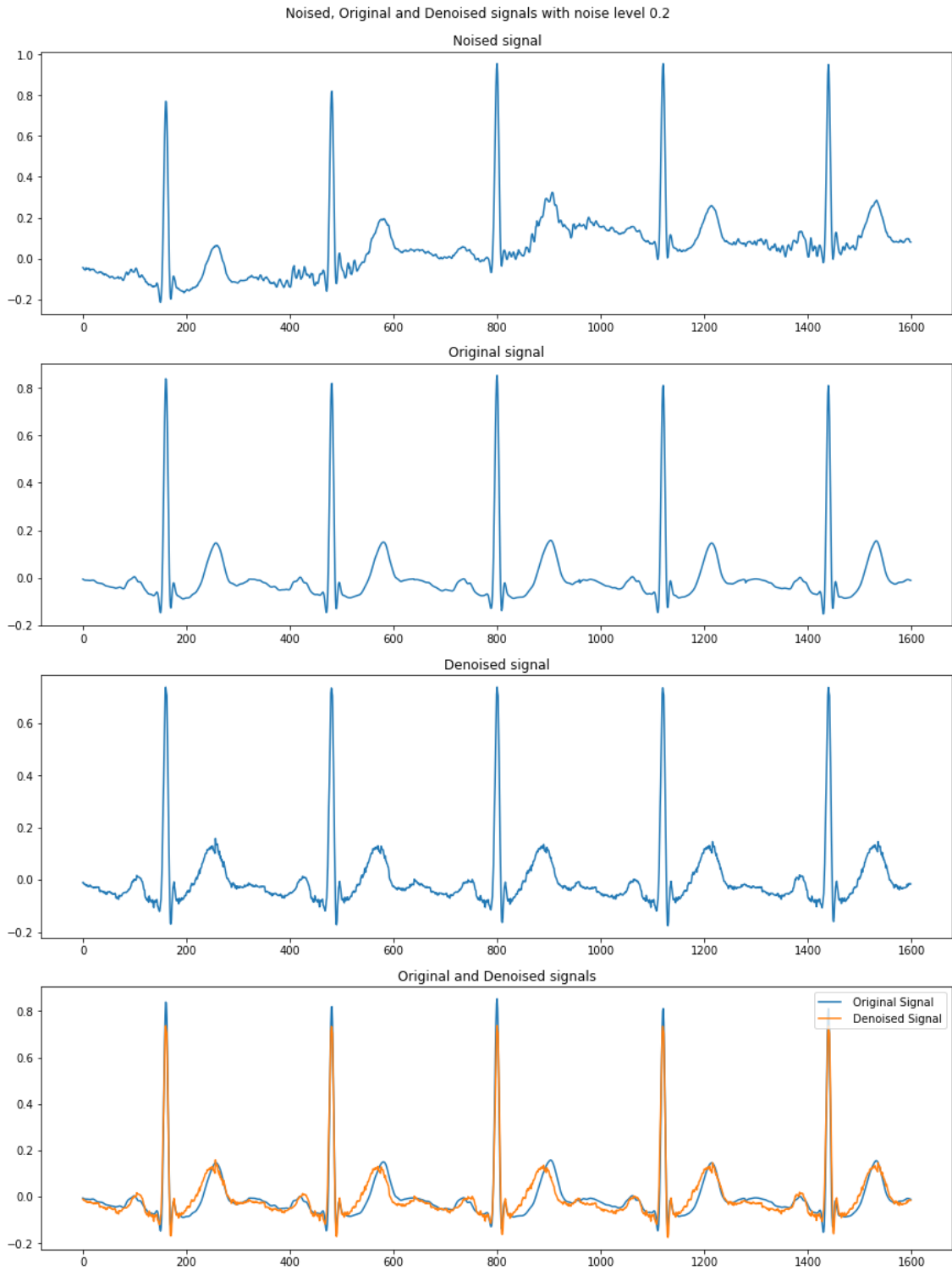


Figure 48: Denoising signal corrupted by MA + BW at 0.2 level of noises

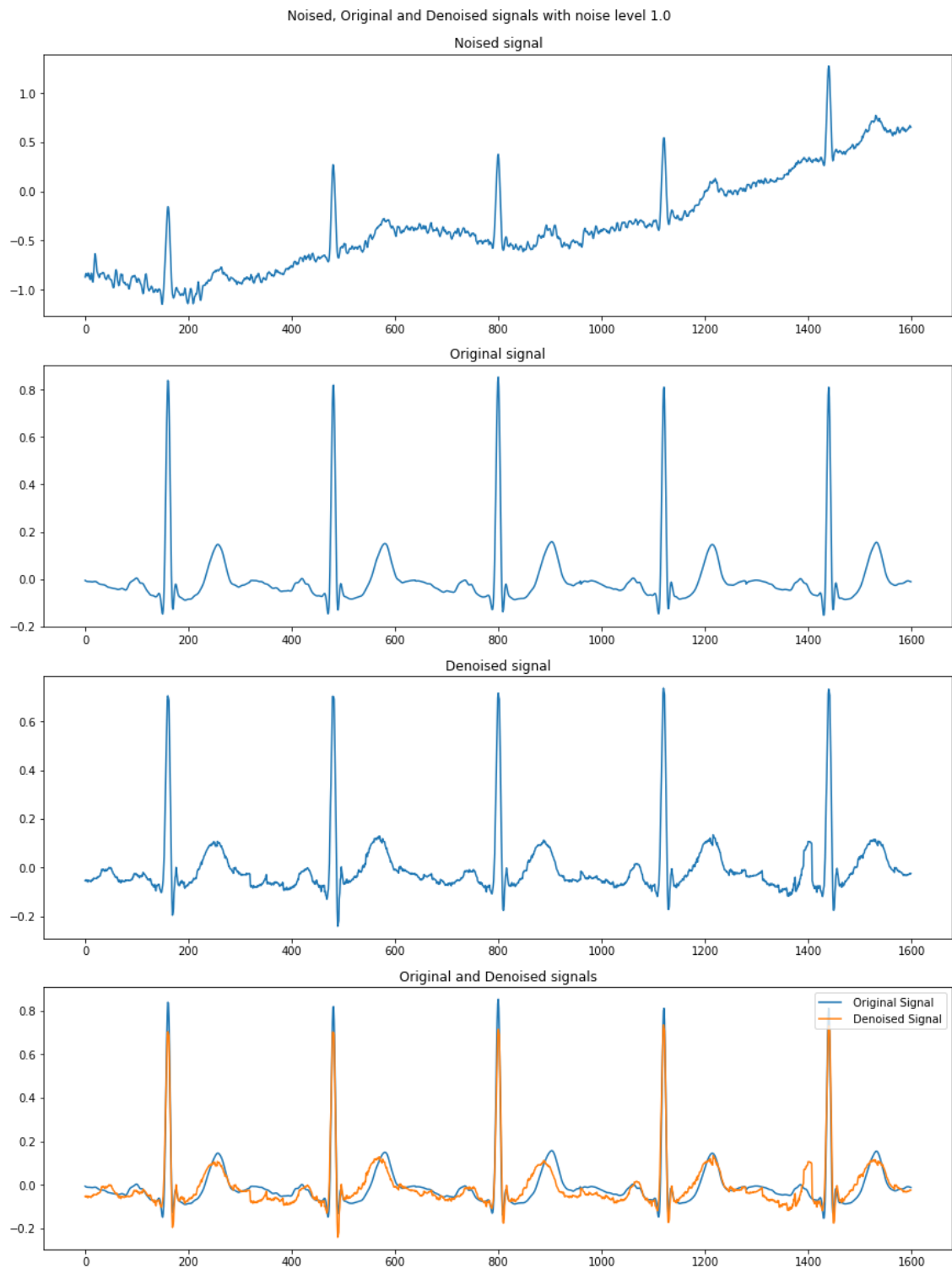


Figure 49: Denoising signal corrupted by a real sample of MA + BW noises

- Denoising signal corrupted by EM + MA noises

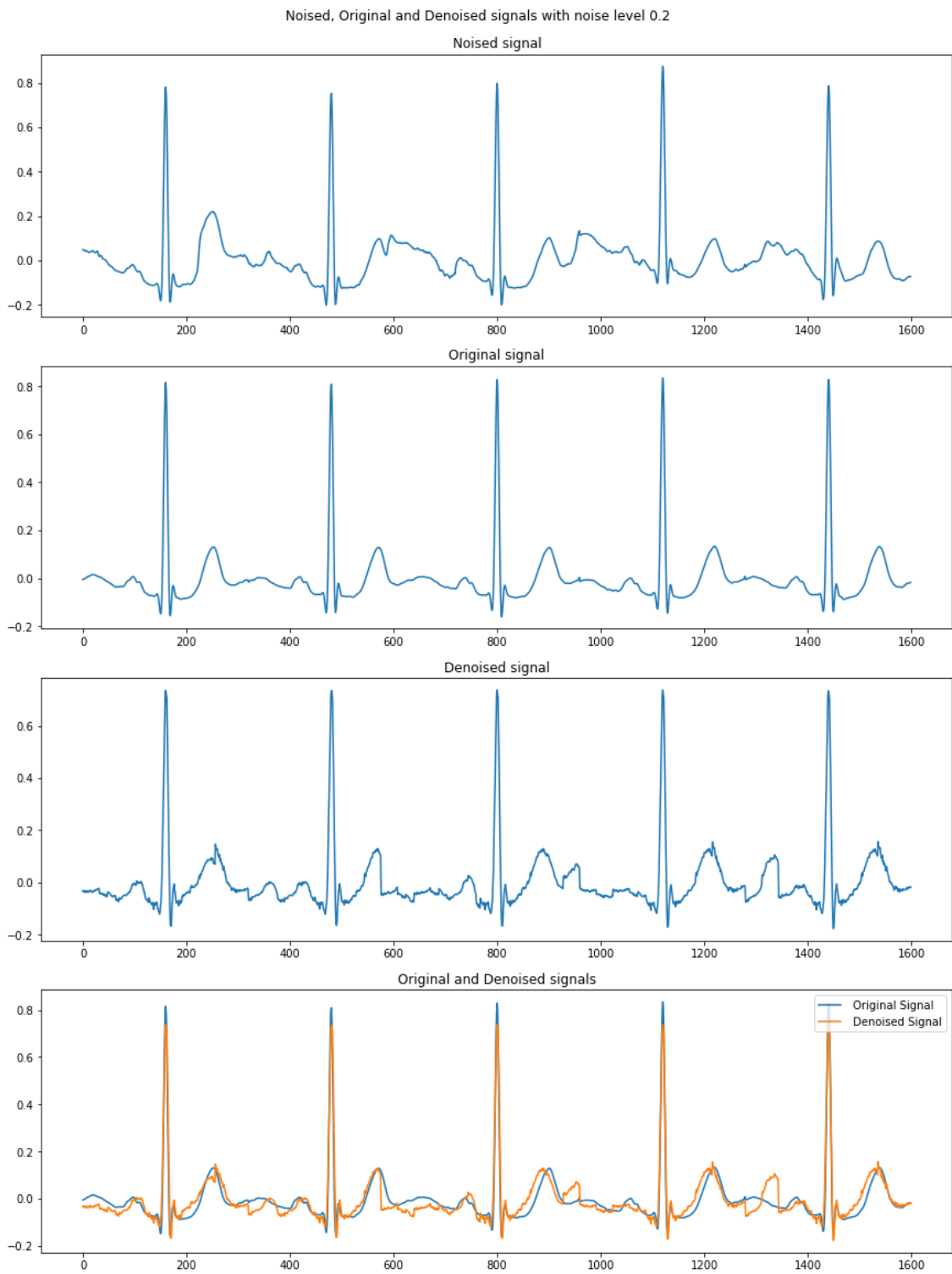


Figure 50: Denoising signal corrupted by EM + MA at 0.2 level of noises

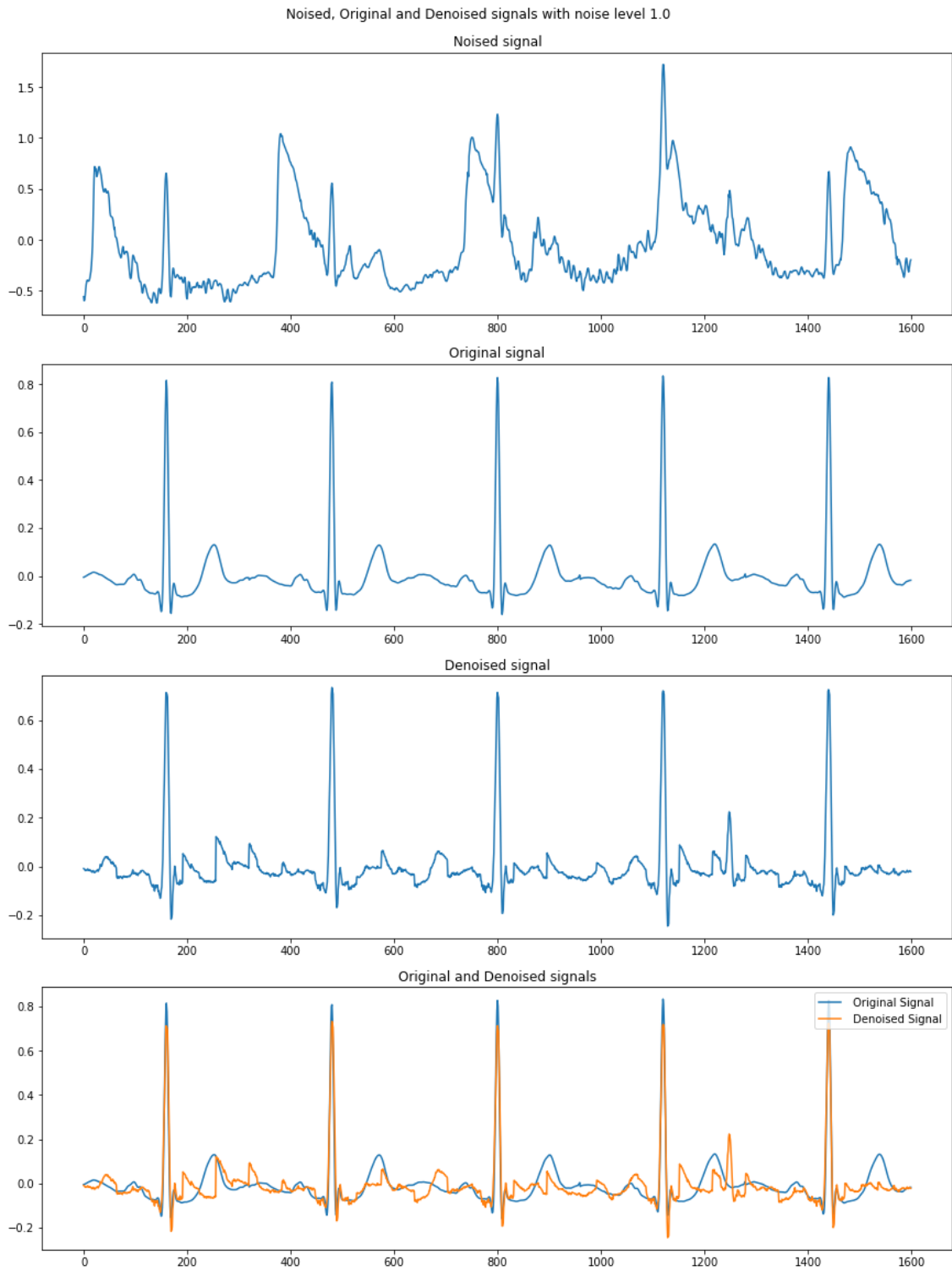


Figure 51: Denoising signal corrupted by a real sample of EM + MA noises

- Denoising signal corrupted by EM + BW noises

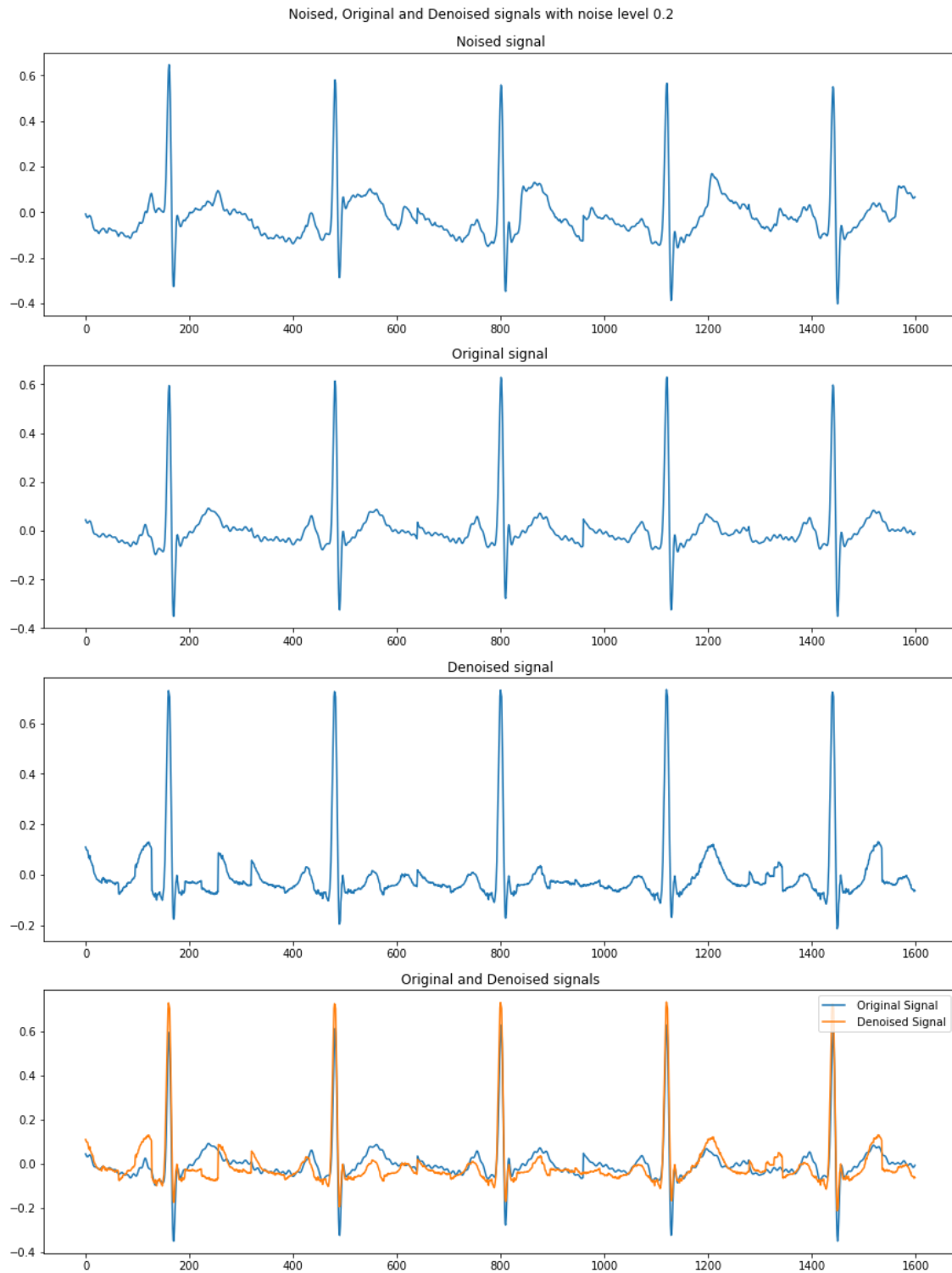


Figure 52: Denoising signal corrupted by EM + BW at 0.2 level of noises

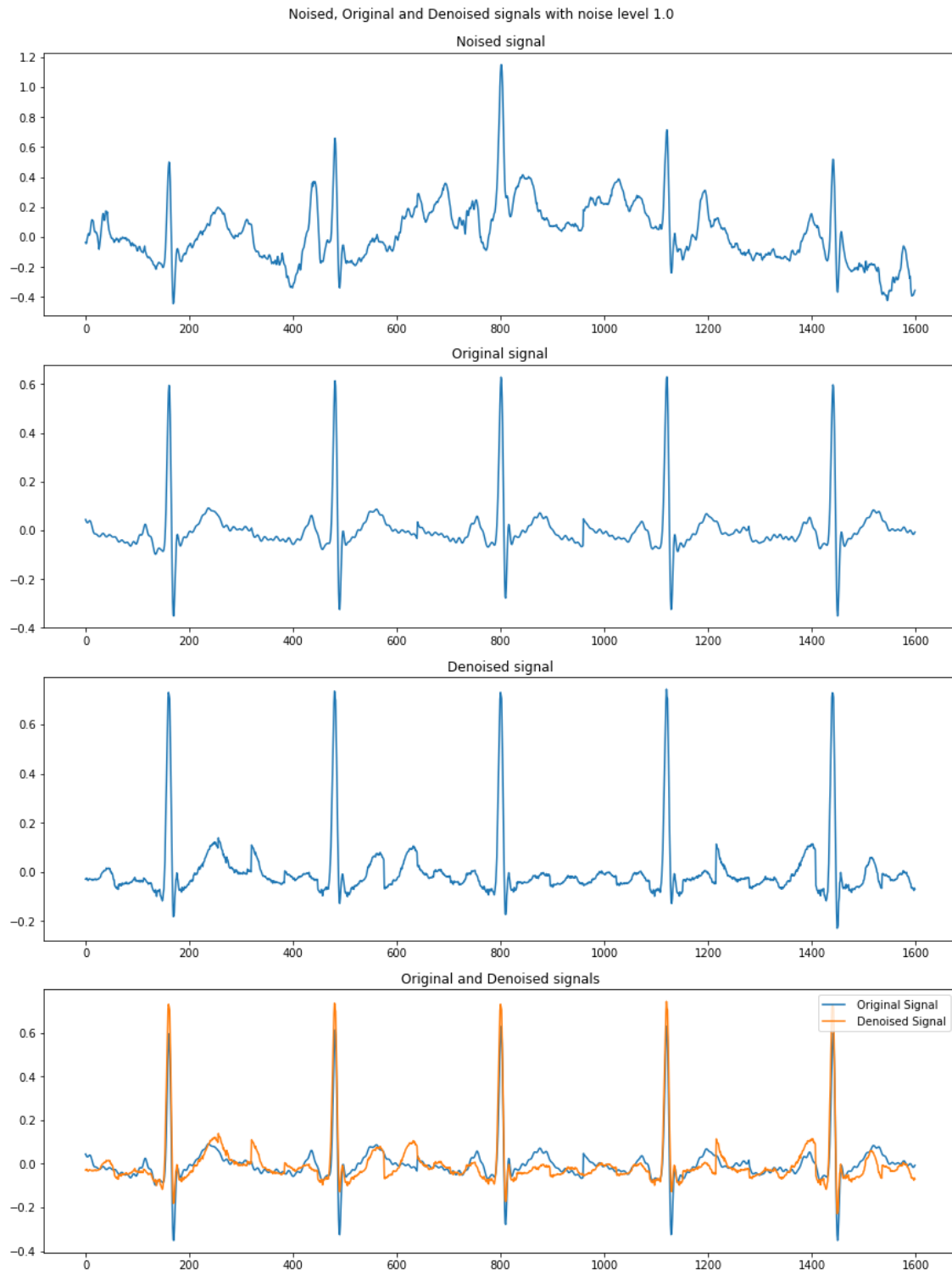


Figure 53: Denoising signal corrupted by a real sample of EM + BW noises

- Denoising signal corrupted by MA + EM + BW noises

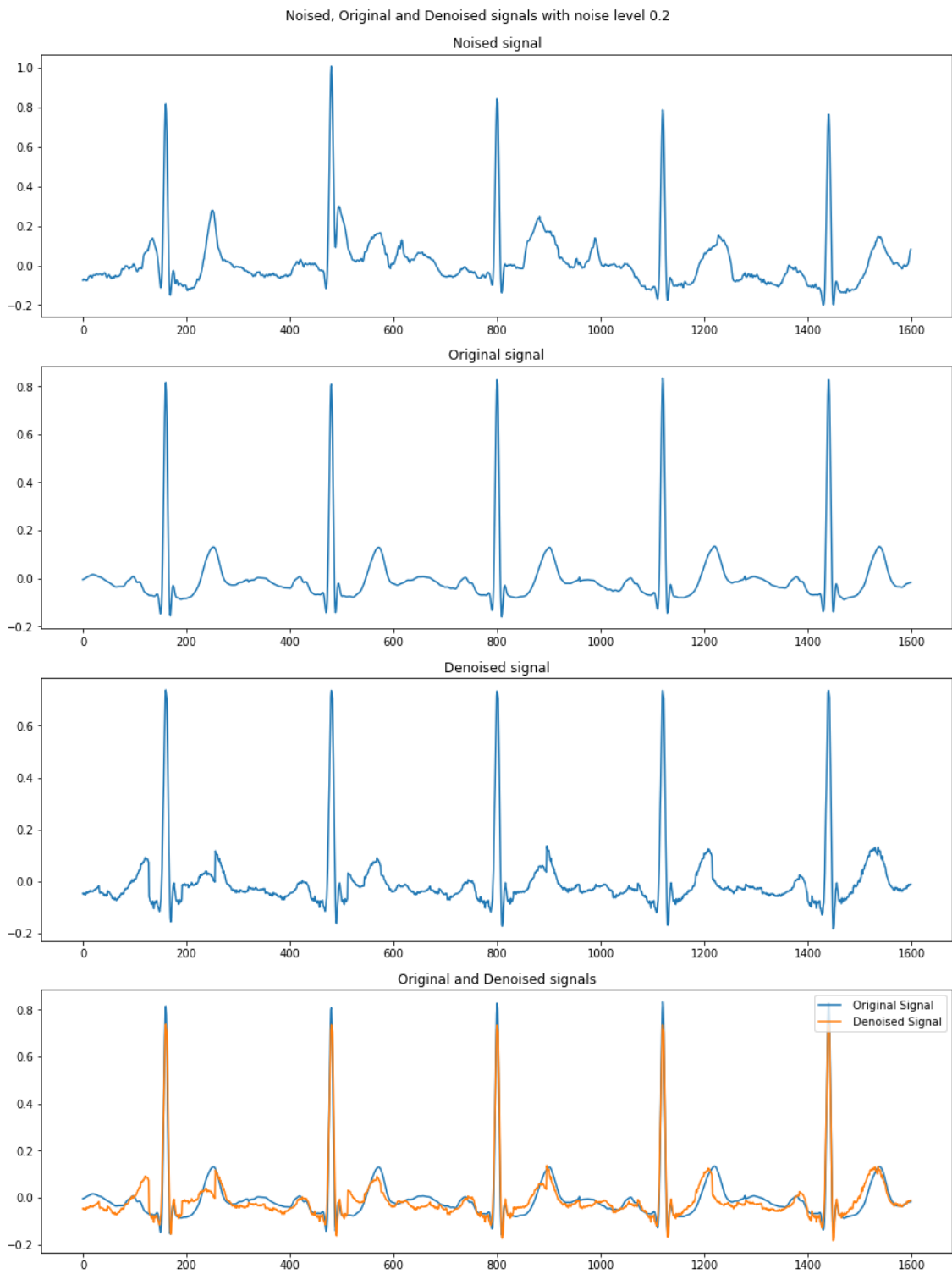


Figure 54: Denoising signal corrupted by MA + EM + BW at 0.2 level noises

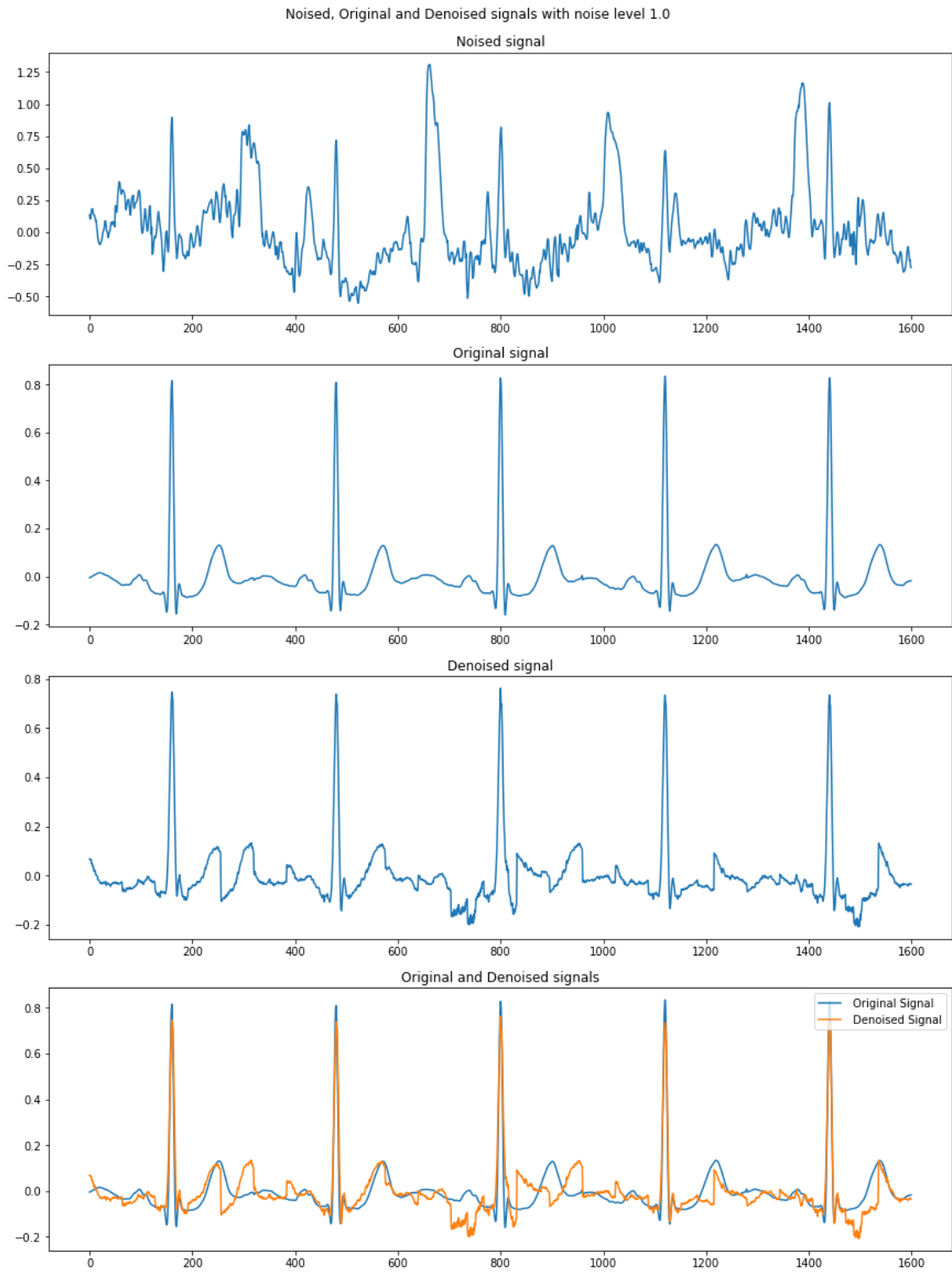


Figure 55: Denoising signal corrupted by a real sample of MA + EM + BW noises

6. Classifier Network

The initial idea of the thesis was also to classify the beats of the ECG signals with the noise removed, but this could not be done due to the time required to design the noise removal stage because of the long run times required to train the GAN. The complete structure of the system to be achieved is shown in figure 56.



Figure 56: Structure of the system to be achieved

For this reason, this section will explain the heartbeat type classifier network where its input is beats directly extracted from the MIT-BIH Arrhythmia Database. The beat type information can be retrieved from the annotations accompanying the signals in the MIT-BIH Arrhythmia Database, where they are labelled as shown in the following table 2.

Symbol	Meaning
L	Left bundle branch block beat
R	Right bundle branch block beat
A	Atrial premature beat
V	Premature ventricular contraction
N	Normal sinus rhythm
\	Paced Beat

Table 2: Types of Heartbeats and representing symbols

It should be noted that on this occasion around 40,000 beats were obtained, as those with noise were not discarded; all the samples that had the informative label of the type of beat were taken, with a maximum of 10,000 samples for each type. This has been done because there are many more samples of type N and they have been limited. From which we can differentiate the number of samples according to their type as shown in table 3 and figure 57.

Heartbeat Type	Samples
L	8071
R	7255
A	7129
V	7023
N	9997
\	2546

Table 3: Number of samples per heratbeat type

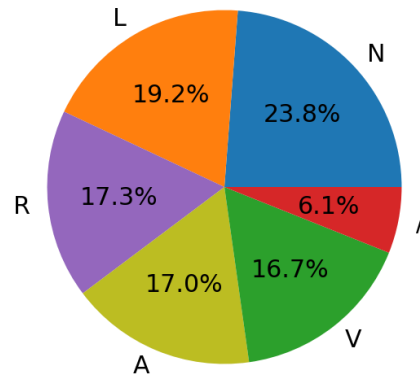


Figure 57: Samples initial distribution

In order to have a balanced dataset, each beat type has been resampled to 5000 samples, so that we avoid adding too many samples to the "/" type which did not reach 5000 samples originally. Thus, we will have 30,000 total samples of which 80% are reserved for training and 20% for testing. The distribution of the samples by type after resampling is shown in the figure 58.

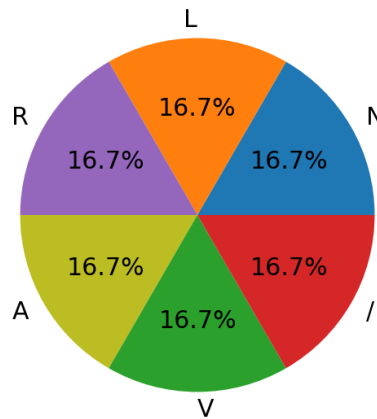


Figure 58: Samples dsitribution after resample

The shape of the different heartbeats can be seen in figure 59.

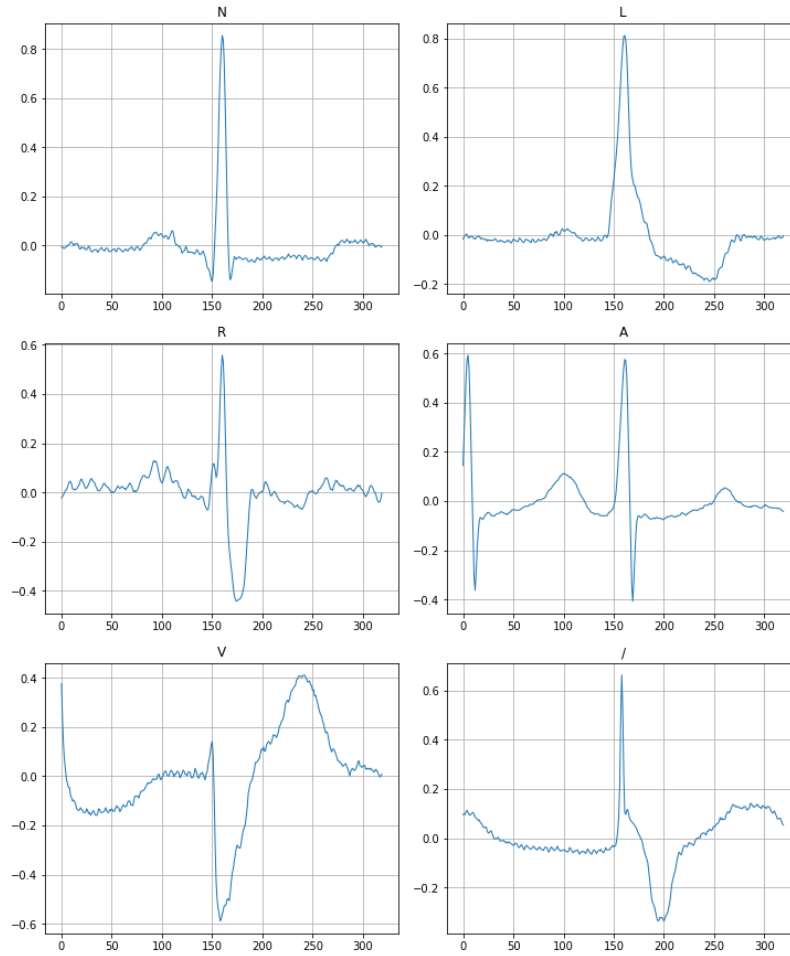


Figure 59: Shape of the Heartbeats by types

6.1. Network architecture

A convolutional neural network has been chosen, since, as previously mentioned, these networks are able to learn the signal features.

After each convolution, the ReLU activation function has been applied and the Average Pooling has been calculated, which returns the average of the features through which the filter passes, i.e. a portion of the signal.

In the last stage, a Flatten layer has been applied, since during the different convolution processes the dimension of the signal changes, and this layer allows us to resize it to a one-dimensional vector. The Dropout so that the neurons of the main characteristics of the signal do not connect to each other during the training, and finally the softmax function that will be in one-hot-encoding format, that is, for each prediction we will have 6 values that represent the probability for each type of heartbeat. The complete architecture can be seen in figure 60.

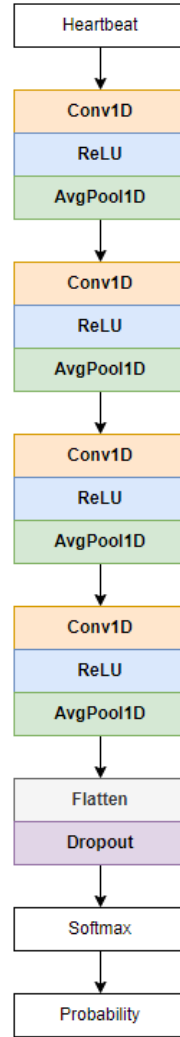


Figure 60: Classifier Network Structure

6.2. Optimiser and loss function

To train this network we have chosen the cross entropy or categorical cross entropy error function, since we have six categories. Regarding the optimiser, we have chosen the Adam optimiser with a learning rate of 0.0001 and the parameters $\beta_1 = 0.99$ and $\beta_2 = 0.5$.

6.3. Training

The network was trained for 60 epochs with a *Bacthsize* = 16. The training execution time was 15 minutes.

The training process has followed a positive evolution according to the figures 61 and 62, where we can see that almost 100% of successes have been reached (specifically 99.83% of accuracy).

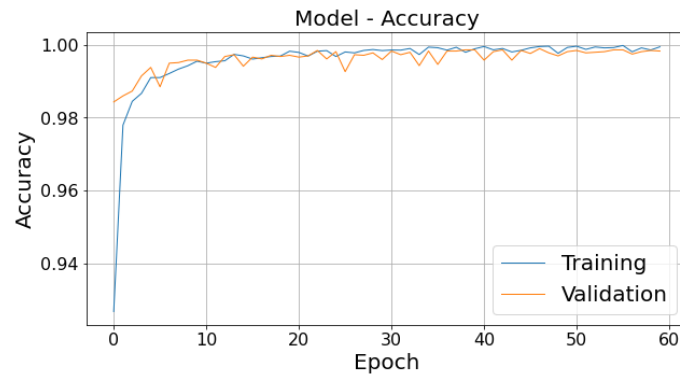


Figure 61: Model accuracy evolution

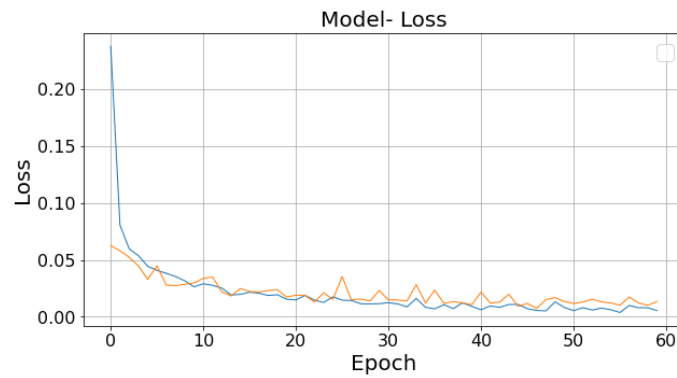


Figure 62: Model Loss evolution

6.4. Results

This section will present some test results for the network, where it can be seen that it works correctly. Below are six heartbeats and their network predictions to see how they perform.

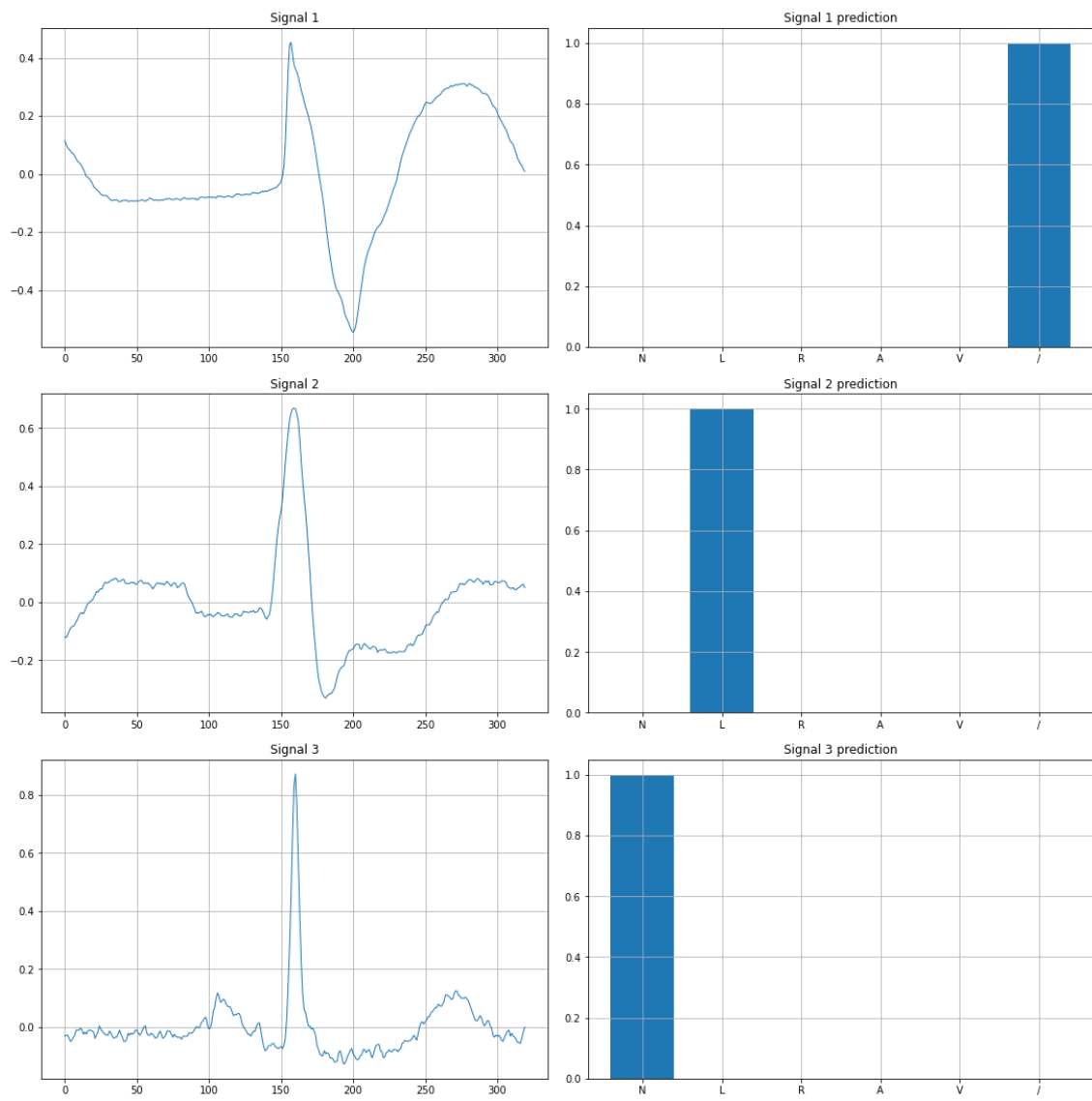


Figure 63: Model Predictions I

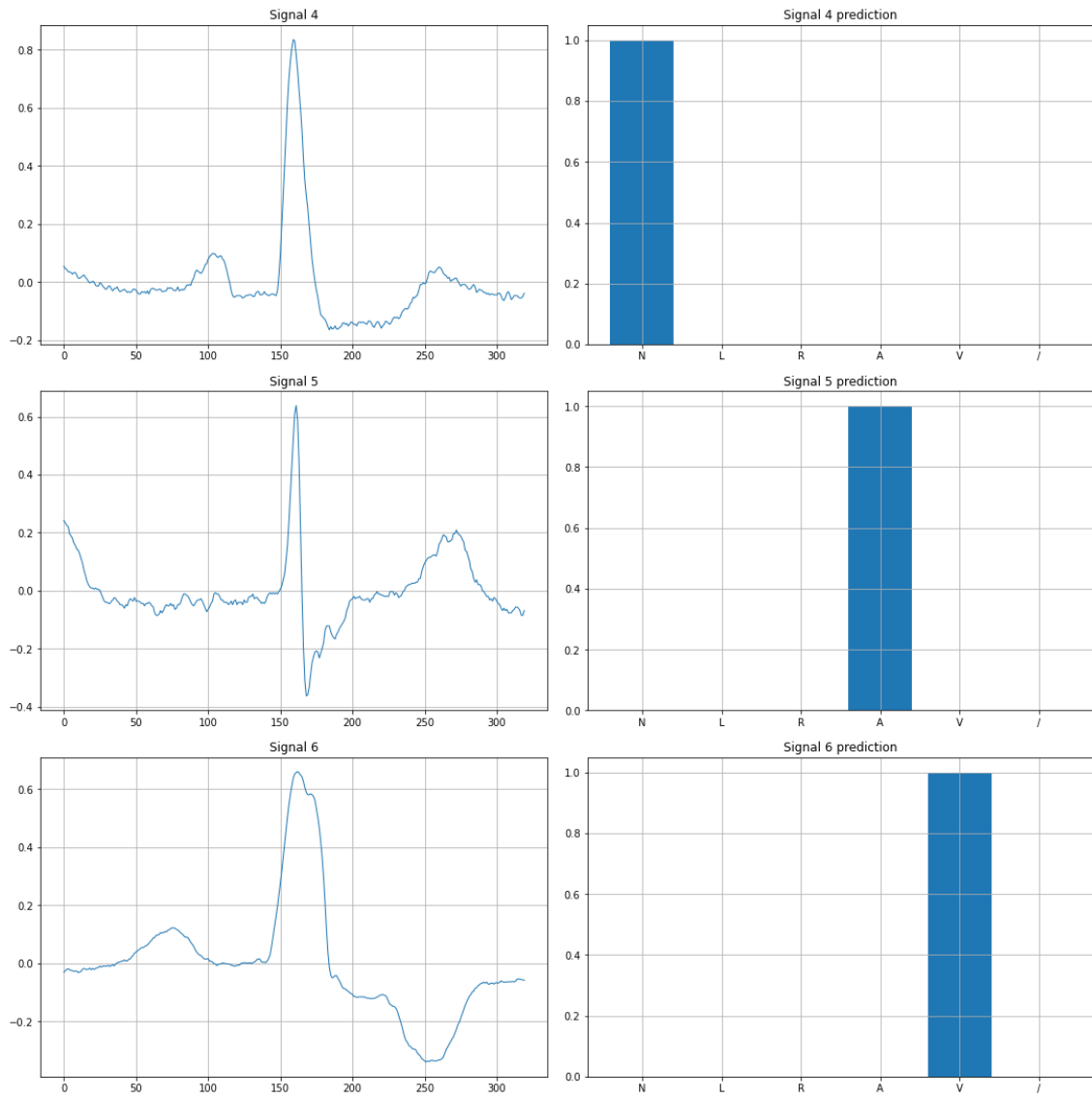


Figure 64: Model Predictions II

7. Conclusions

The main objective of the thesis has been partially achieved. The GAN has been designed and shows a quite correct behaviour for MA and BW noises and their combinations, in which the wavelets of the denoised ECG signals can be distinguished. However, as the noise increases, the neural network loses efficiency in restoring the signal.

For EM noise, it has already been seen in the literature that many researchers have had problems dealing with this type of noise, in our case this has also happened. The problem with this type of noise is that it masks too much of the signal, leading the neural network to invent the peaks of the representative waves of the ECGs.

A pix2pix GAN capable of de-noising some ECG signals has been implemented, adding value to this type of neural networks, as they have a lot of potential in fields other than image processing.

Although it has not been possible to tune the output of the GAN to be able to classify ECG signals without noise, a CNN that can classify different types of heartbeats has been designed for proof-of-concept purposes.

It is important to note that, as we have seen, it is important to have powerful machines available to facilitate the training of networks in terms of time. With the evolution of GPUs in terms of power and energy consumption, more and more complex problems can be tackled. Also, with the emergence of new services such as the one used in this thesis, Google Colaboratory.

Bibliography

1. Organización Mundial de la Salud, “El tabaco rompe corazones. Elija salud, no tabaco. Organización Mundial de la Salud, (WHO/NMH/PND/18.4), Ginebra. 2018.[Online]. Available: <https://www.who.int/>
2. Z. Wang, C. M. Wong, J. N. da Cruz et al., “Muscle and electrode motion artifacts reduction in ECG using adaptive Fourier decomposition,” in 2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 1456–1461, San Diego, CA, USA, 2014.
3. P.R.B. Barbosa , J. Barbosa-Filho , C.A.M. de Sa , E.C. Barbosa , J. Nadal , Reduction of electromyographic noise in the signal-averaged electrocardiogram by spectral decomposition, IEEE Trans. Biomed. Eng. 50 (1) (2013) 114–117.
4. M.A. Kabir , C. Shahnaz , Denoising of ECG signals based on noise reduction algorithms in EMD and wavelet domains, Biomed. Signal Process. Control 7 (5) (2012) 4 81–4 89.
5. M.A. Awal , S.S. Mostafa , M. Ahmad , et al. , An adaptive level dependent wavelet thresholding for ECG denoising, Biocybern. Biomed. Eng. 34 (4) (2014) 238–249.
6. Z. Shang , Low power FIR filter design for wearable devices using frequency response masking technique, in: Proceedings of the 2017 IEEE 12th International Conference on ASIC, 2017 .
7. S.S. Dhillon , S. Chakrabarti , Power line interference removal from electrocardiogram using a simplified lattice based adaptive IIR notch filter, engineering in medicine and biology society, in: Proceedings of the 23rd Annual International Conference of the IEEE, 2001, pp. 3407–3412
8. N. Dey, T. P. Dash, and S. Dash, “ECG signal denoising by functional link artificial neural network (FLANN),” International Journal of Biomedical Engineering & Technology, vol. 7, no. 4, pp. 377–389, 2011.
9. S. Pongpon Sri and X. Yu, “Electrocardiogram (ECG) signal modeling and noise reduction using wavelet neural networks,” in 2009 IEEE International Conference on Automation and Logistics, pp. 394–398, Shenyang, China, 2009.
10. P. Xiong, H. Wang, M. Liu, S. Zhou, Z. Hou, and X. Liu, “ECG signal enhancement based on improved denoising autoencoder,” Engineering Applications of Artificial Intelligence, vol. 52, pp. 194–202, 2016.
11. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in Proc. Advances Neural Information Processing Systems Conf., 2014, pp. 2672–2680
12. D. Rumelhart, G. Hinton y R. Williams «Learning representations by back-propagating errors». Universidad de California, 1986.
13. A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath. Generative adversarial networks: an overview. IEEE Signal Process. Mag., 35(1):53–65, 2018.
14. M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein gan. eprint arXiv:1701.07875, 2017.
15. I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A.C. Courville. Improved training of wasserstein gans. In NIPS, 2017.
16. Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. 2015
17. Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. CoRR, abs/1411.1784, 2014.

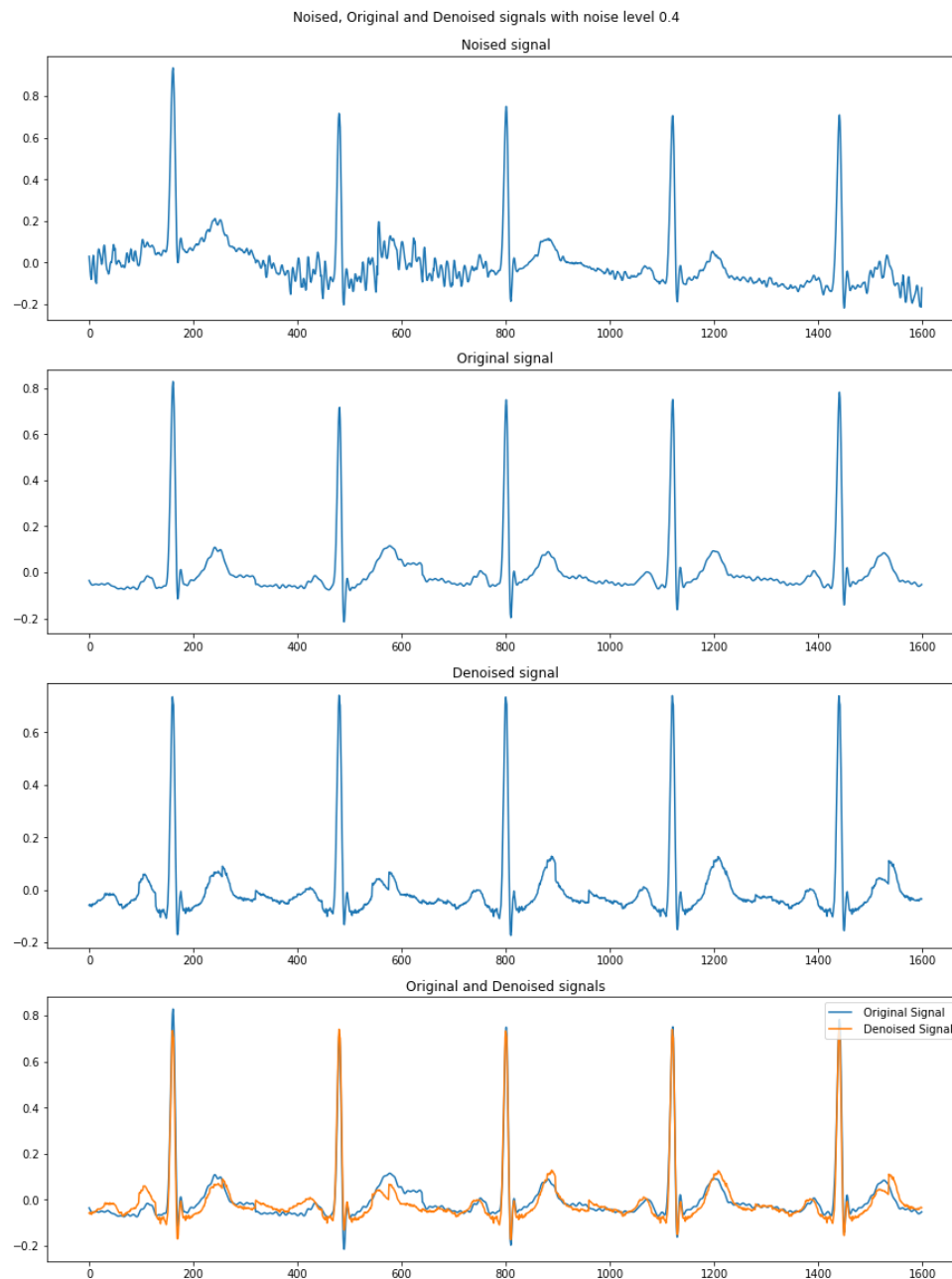
18. Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. CoRR, abs/1703.10593, 2017.
19. A. Clark, J. Donahue, and K. Simonyan. Efficient video generation on complex datasets. CoRR, abs/1907.06571, 2019.
20. S. Aigner and M. Körner. Futuregan: Anticipating the future frames of video sequences using spatio-temporal 3d convolutions in progressively growing gans. ISPRS - International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, XLII-2/W16:3–11, 09 2019.
21. Y. Kwon and M. Park. Predicting future frames using retrospective cycle gan. In 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 1811–1820, 2019.
22. William Lotter, Gabriel Kreiman, and David Cox. Unsupervised learning of visual structure using predictive generative networks, 2015.

Appendices

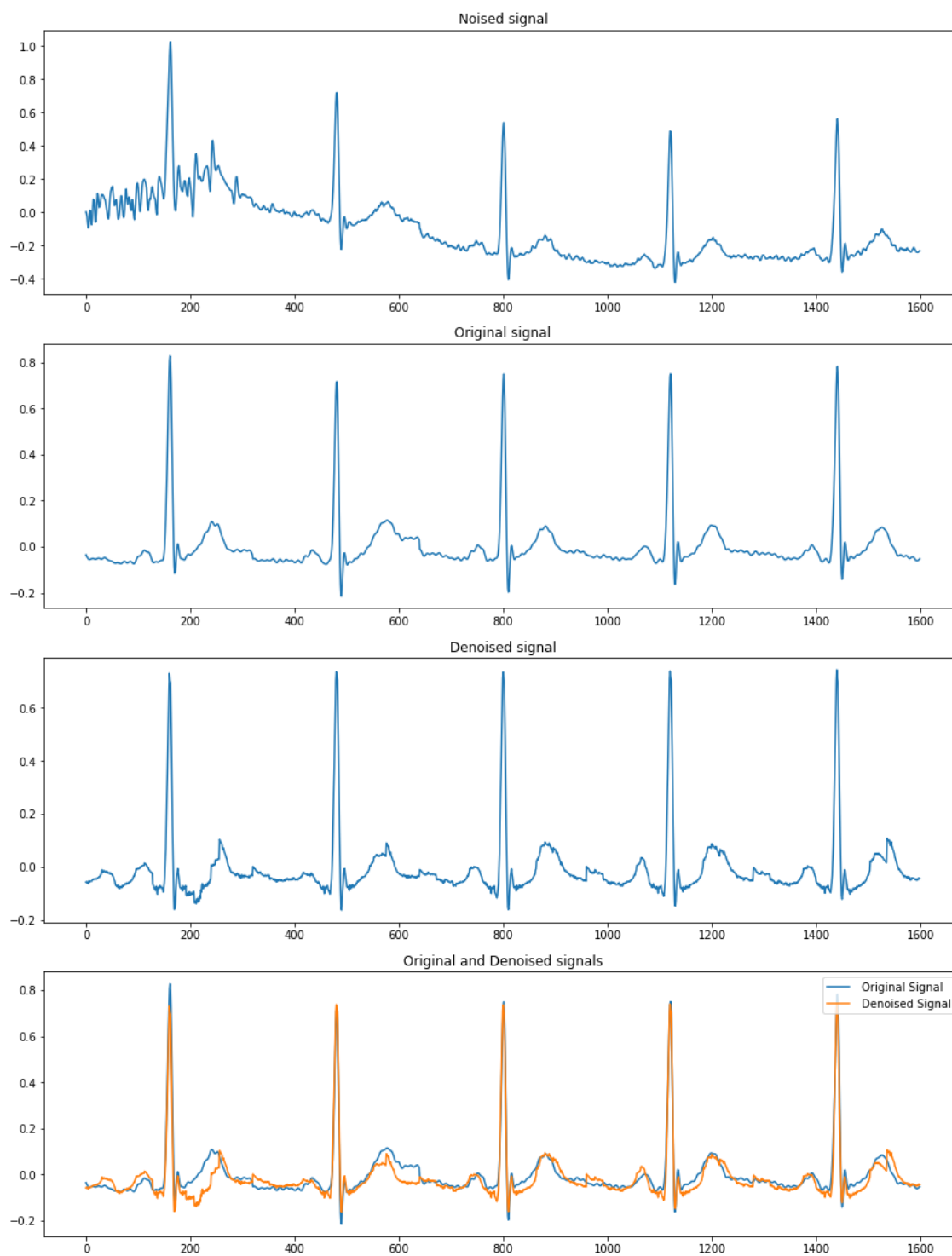
Appendix 1

Denoising for all other noise levels are available in this appendix.

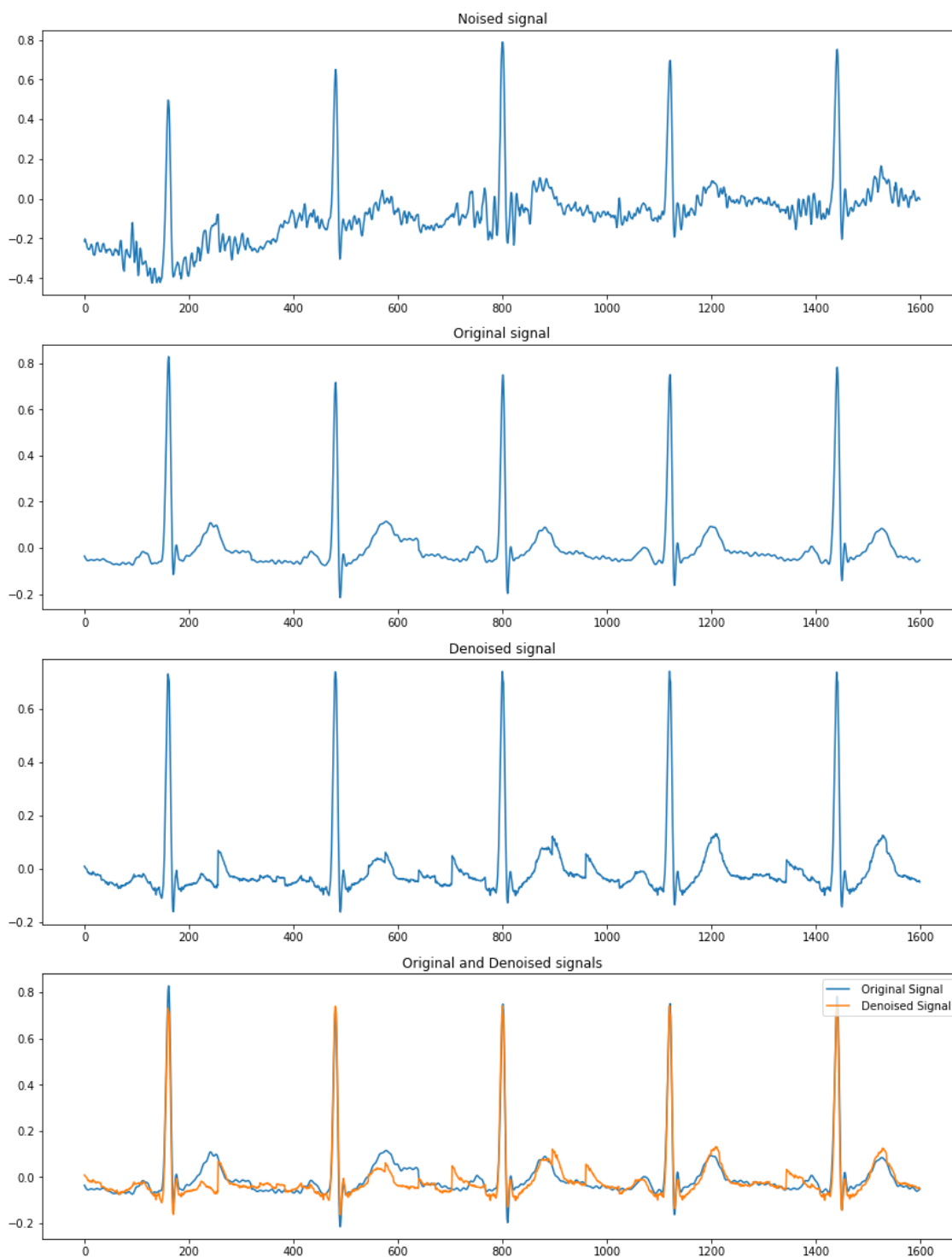
Denoising MA noise



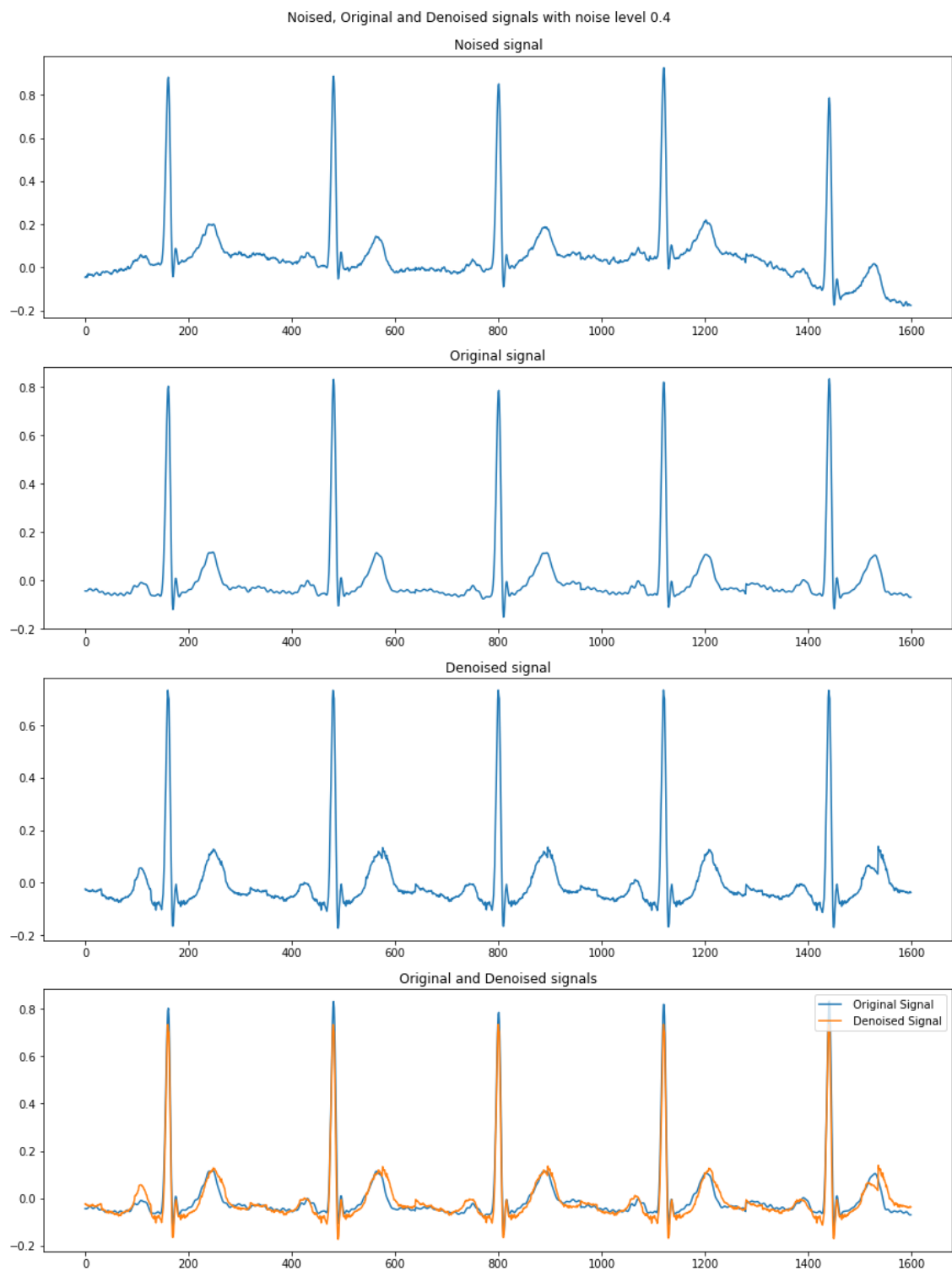
Noised, Original and Denoised signals with noise level 0.6



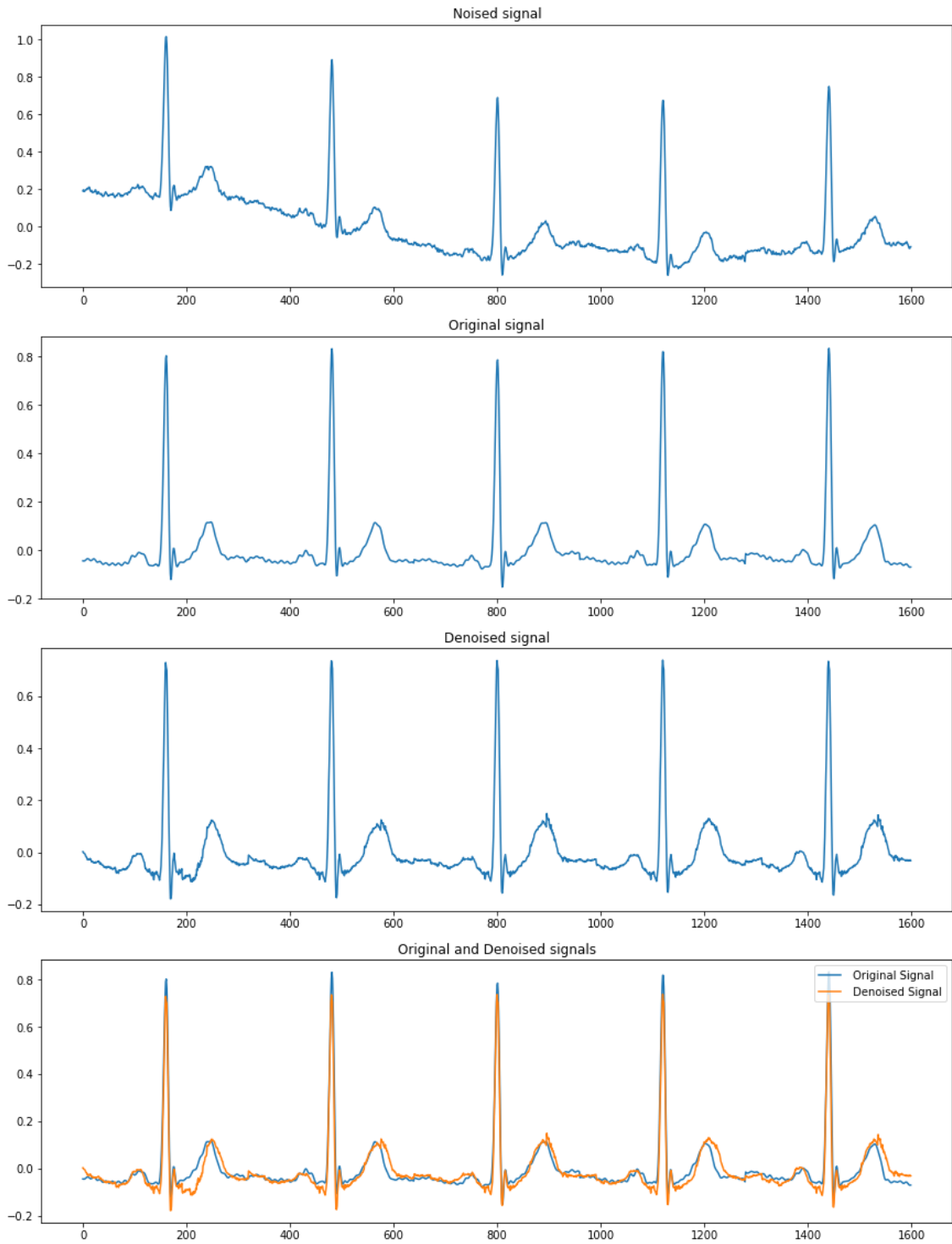
Noised, Original and Denoised signals with noise level 0.8



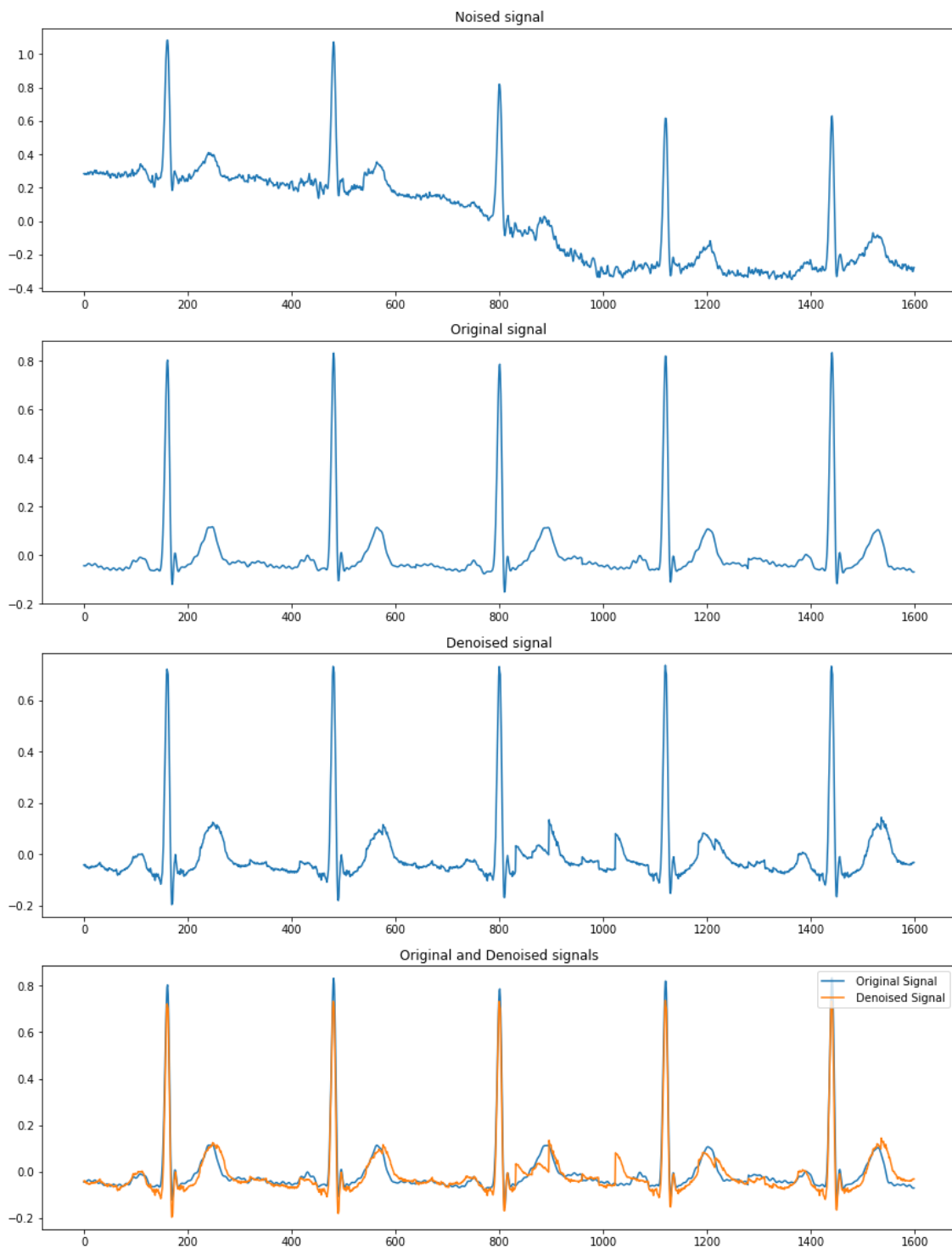
Denoising BW noise



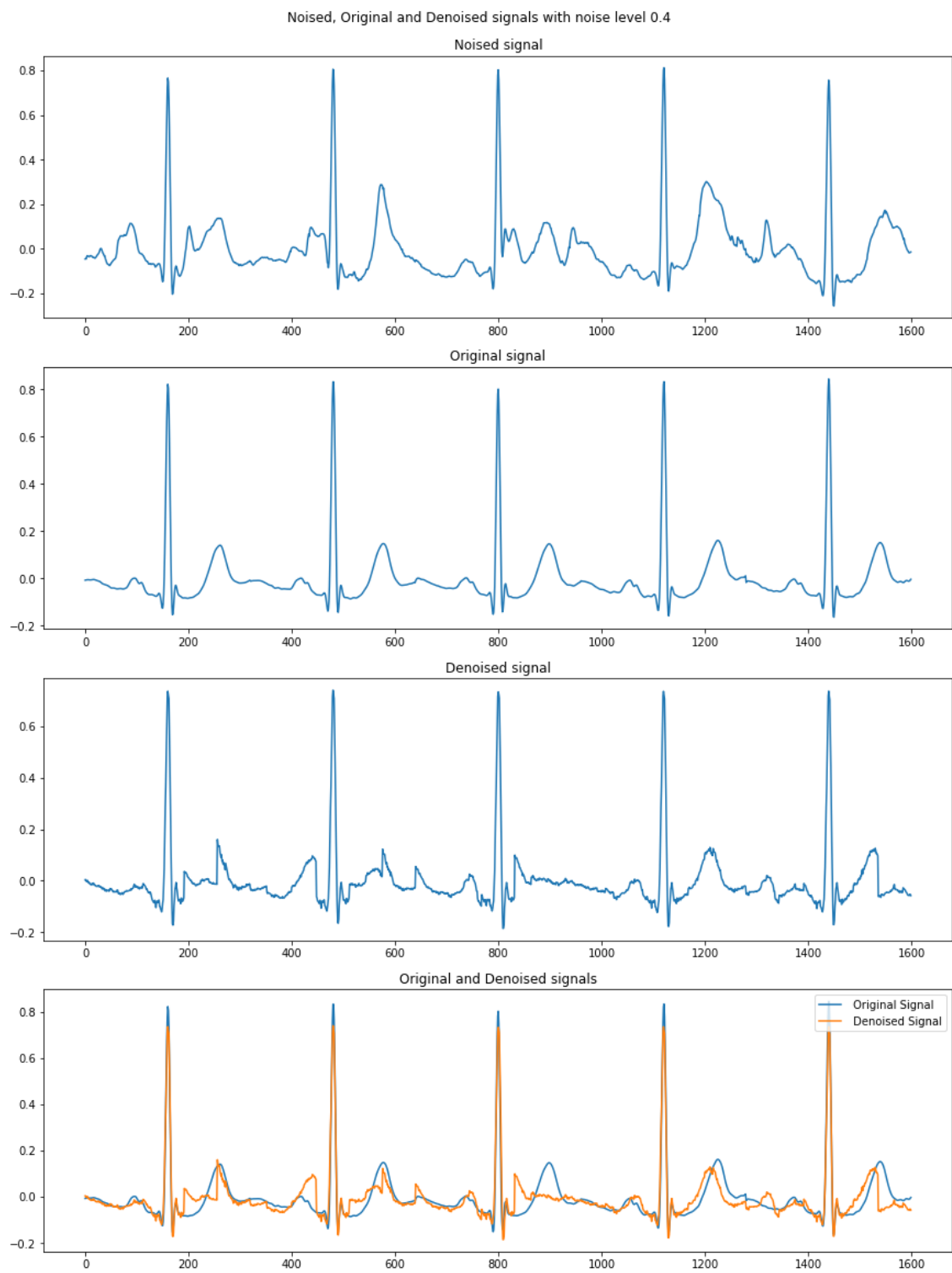
Noised, Original and Denoised signals with noise level 0.6



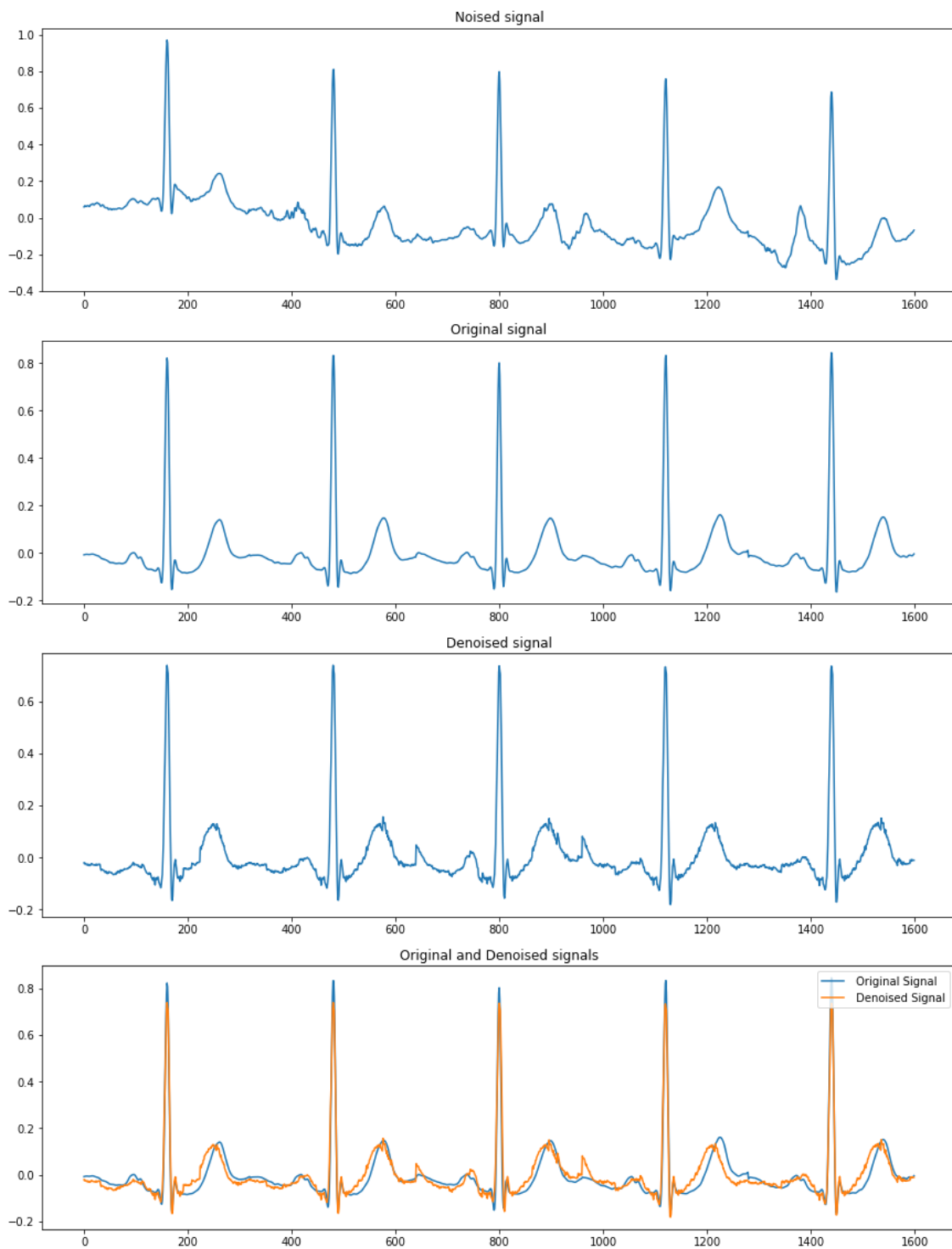
Noised, Original and Denoised signals with noise level 0.8



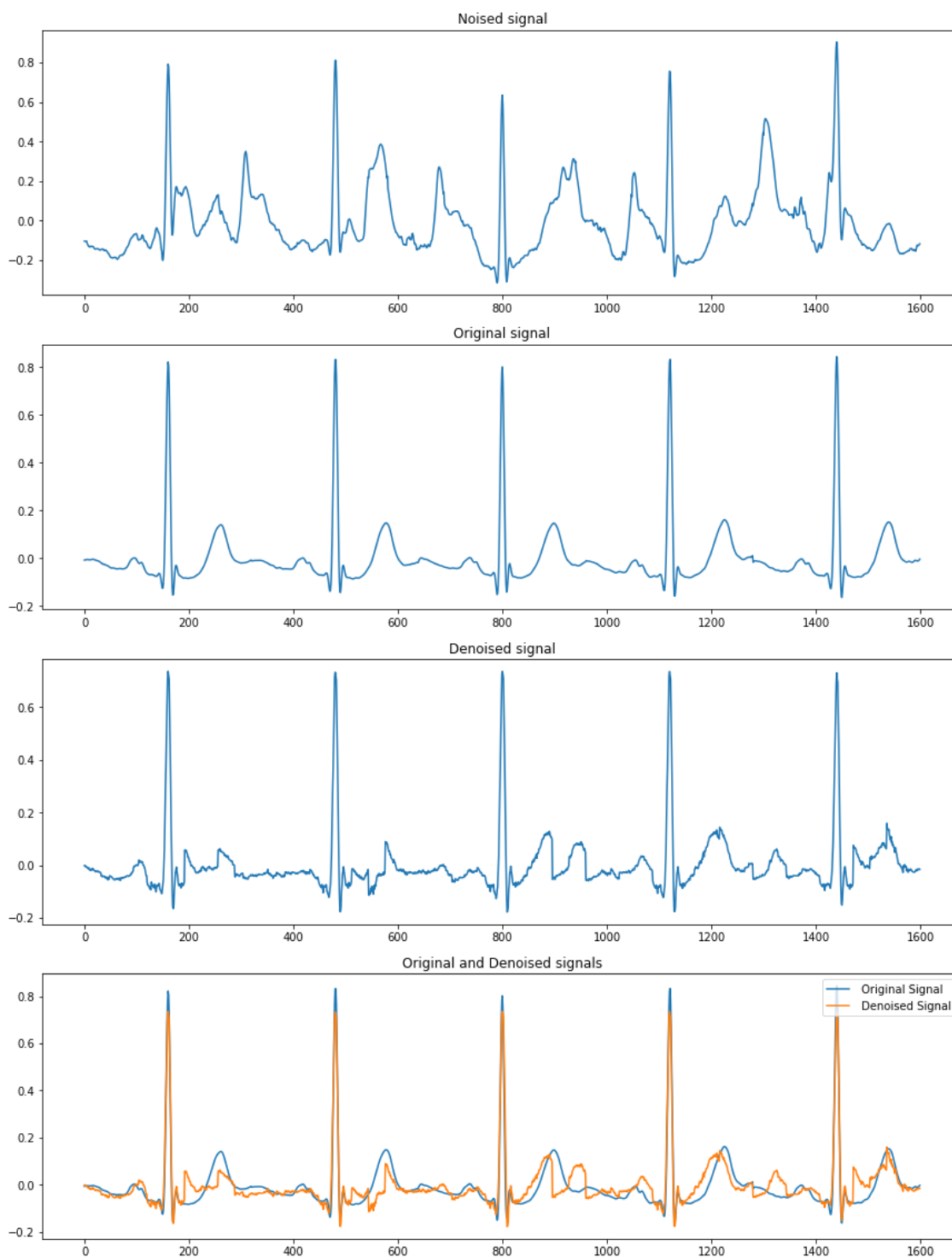
Denoising EM noise



Noised, Original and Denoised signals with noise level 0.6



Noised, Original and Denoised signals with noise level 0.8

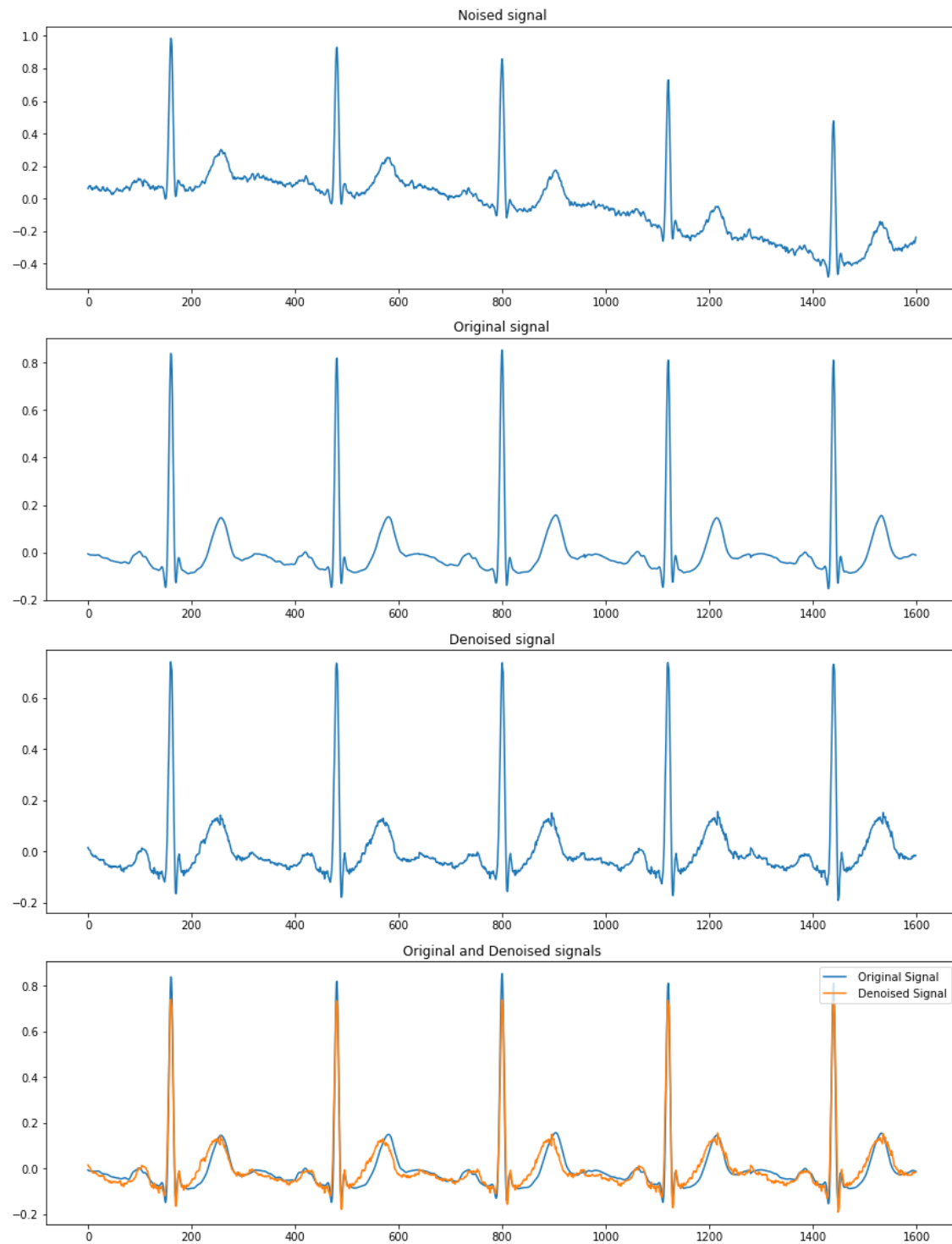


Denoising MA + BW noises

Noised, Original and Denoised signals with noise level 0.4



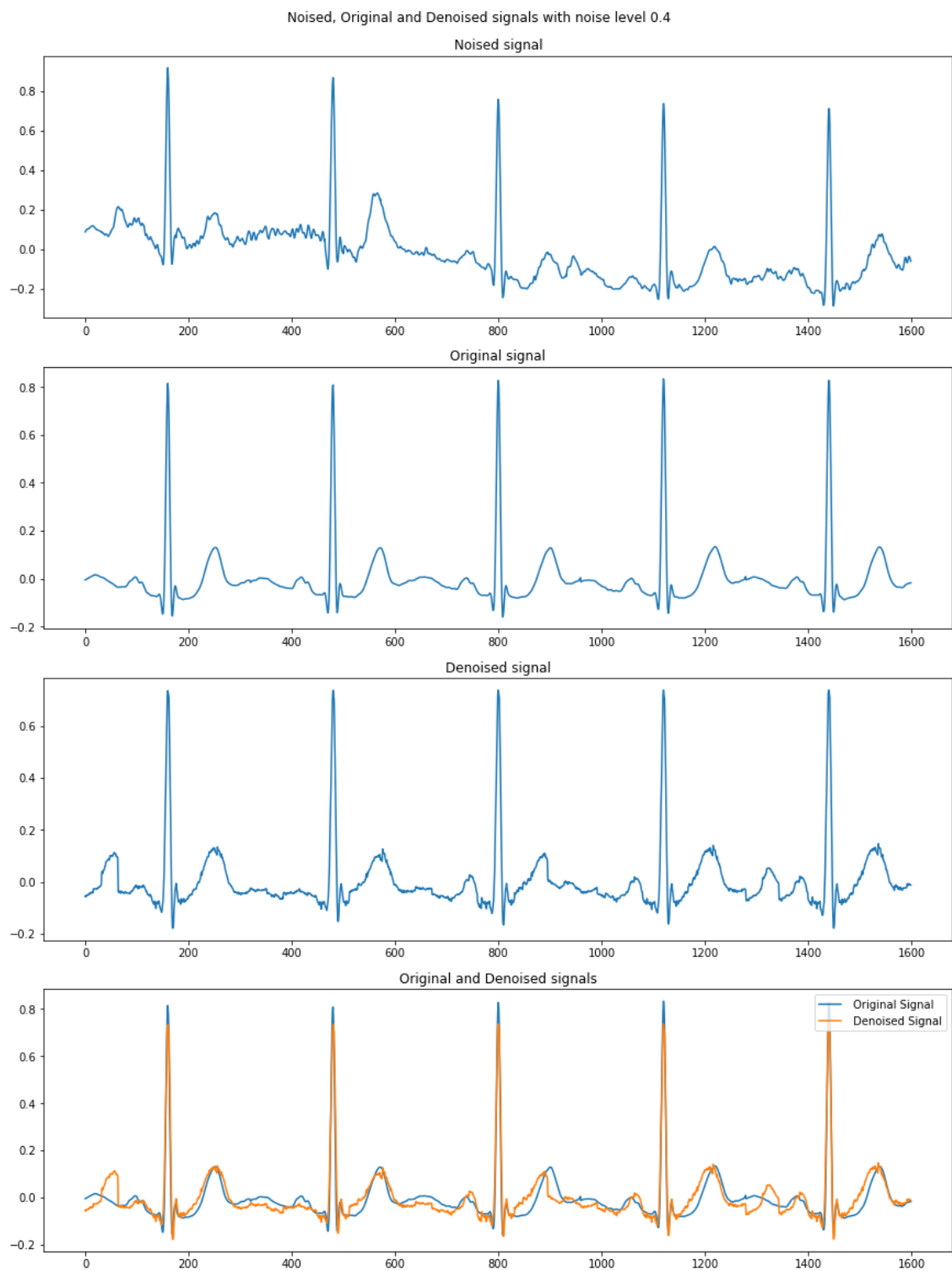
Noised, Original and Denoised signals with noise level 0.6



Noised, Original and Denoised signals with noise level 0.8



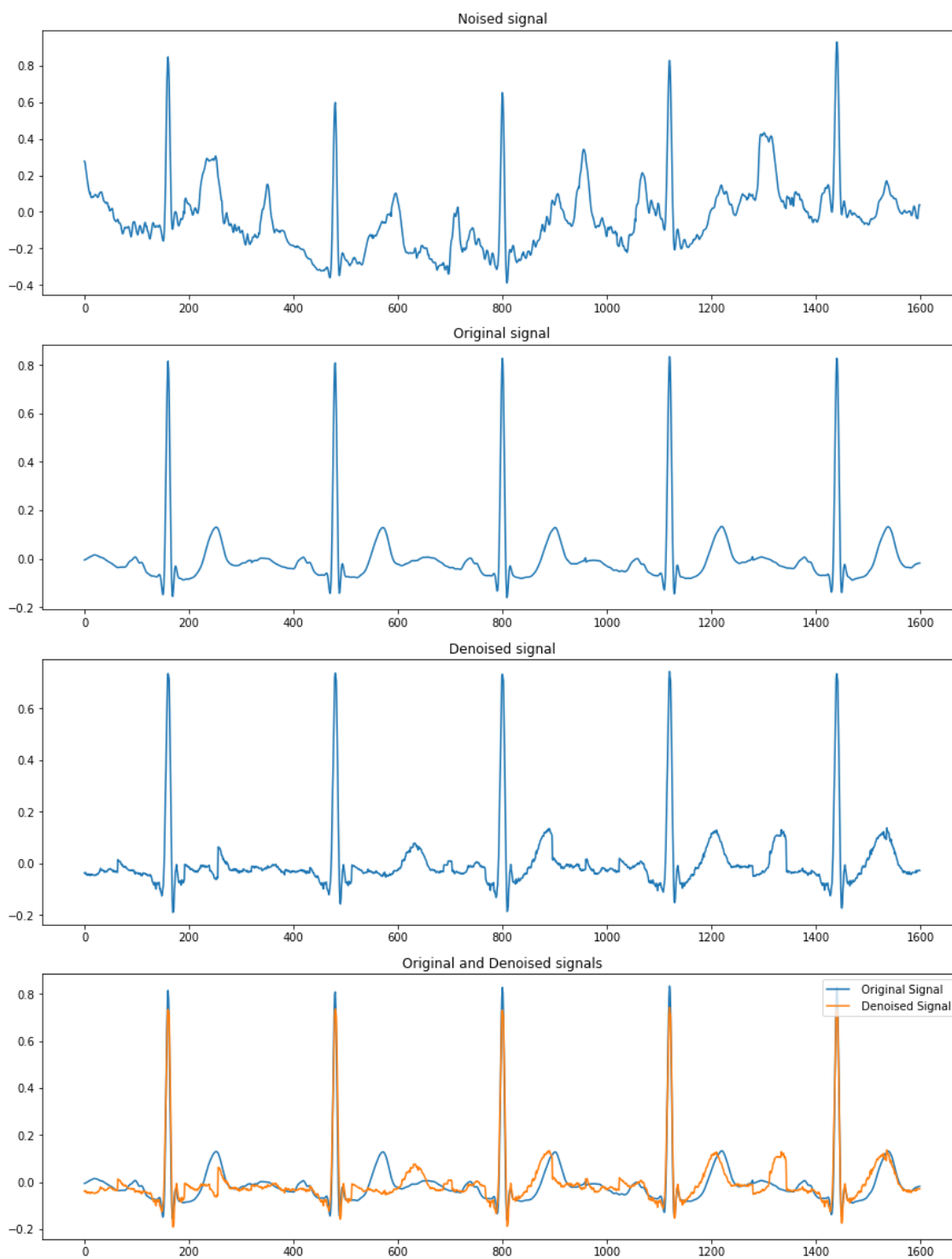
Denoising MA + EM noises



Noised, Original and Denoised signals with noise level 0.6

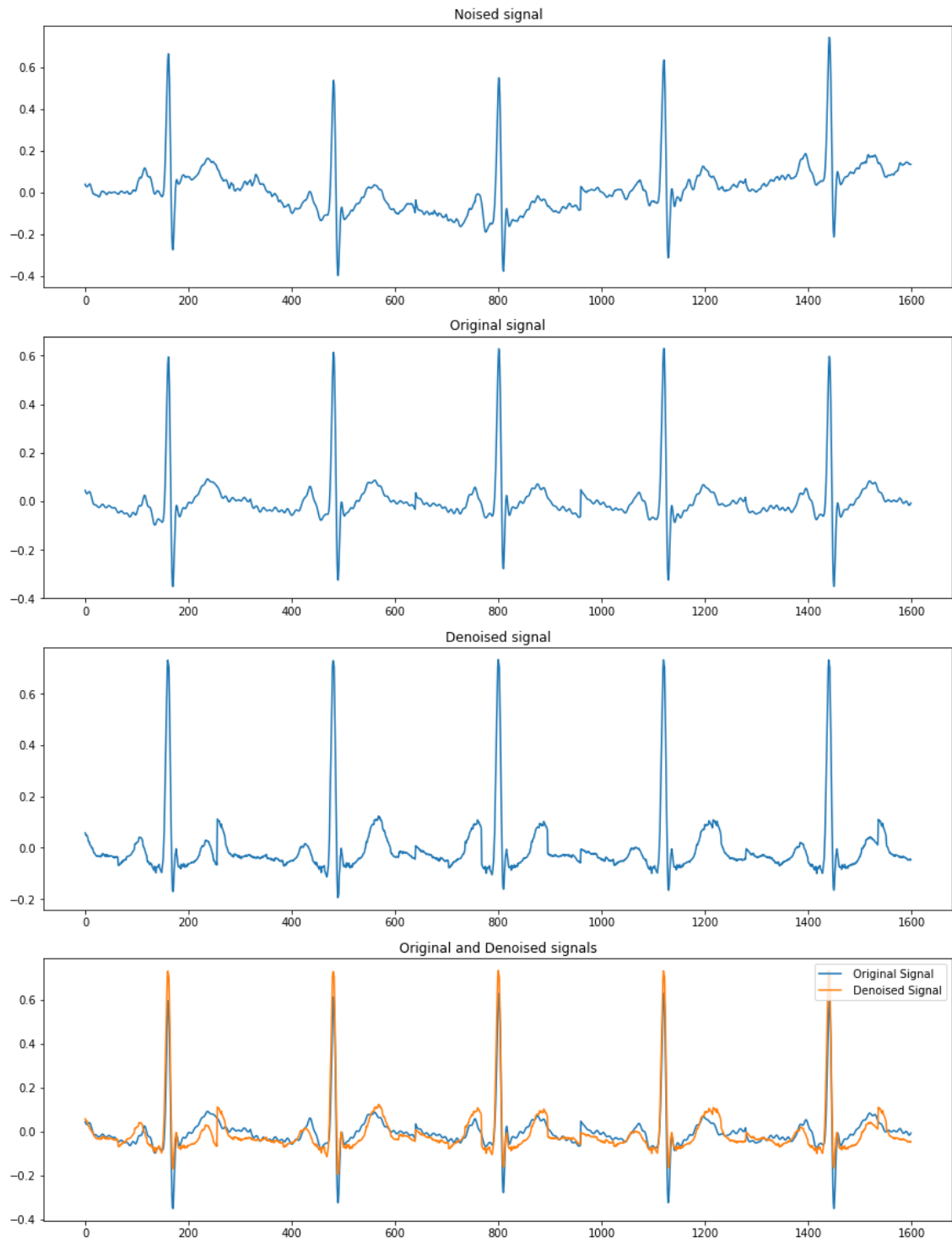


Noised, Original and Denoised signals with noise level 0.8

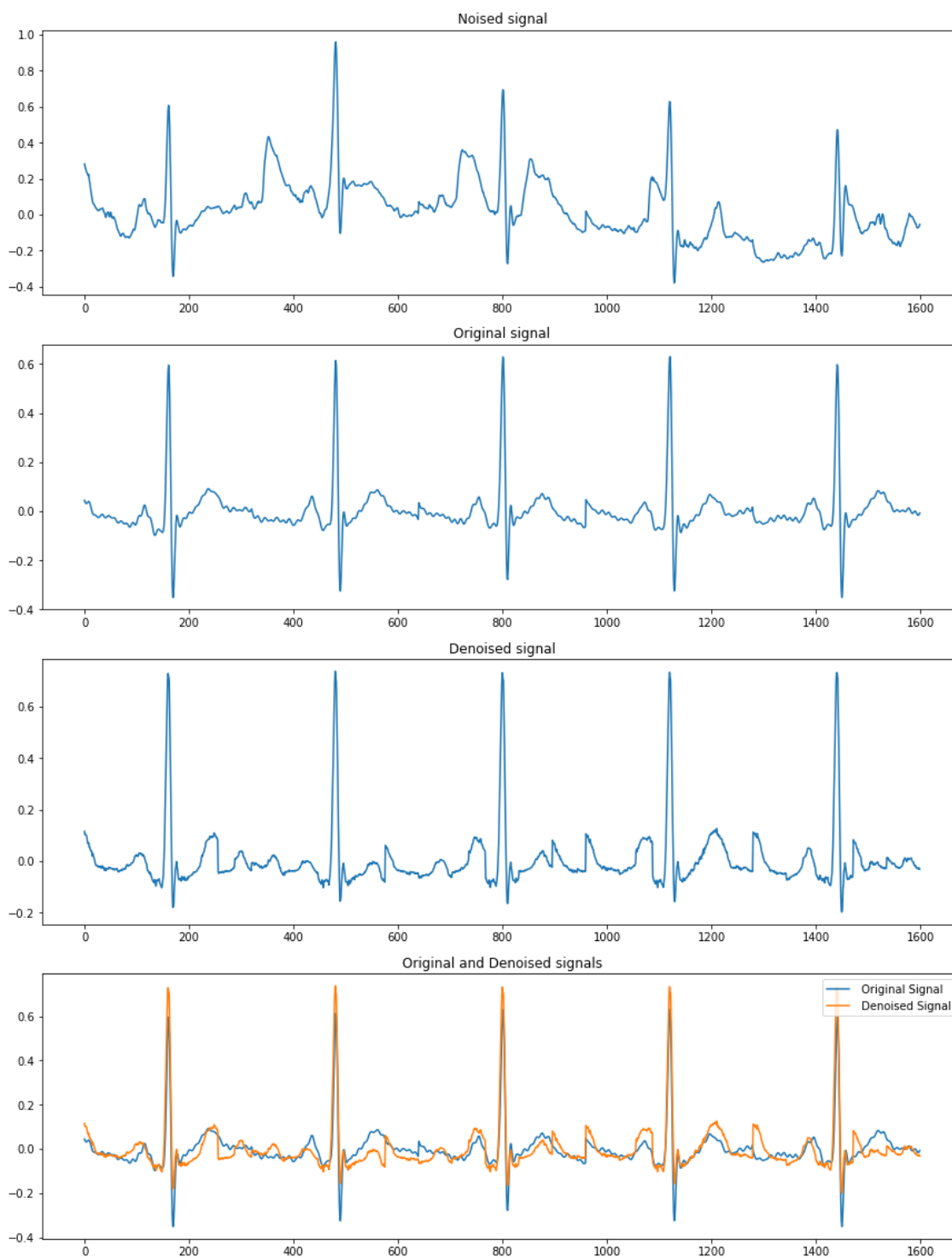


Denoising EM + BW noises

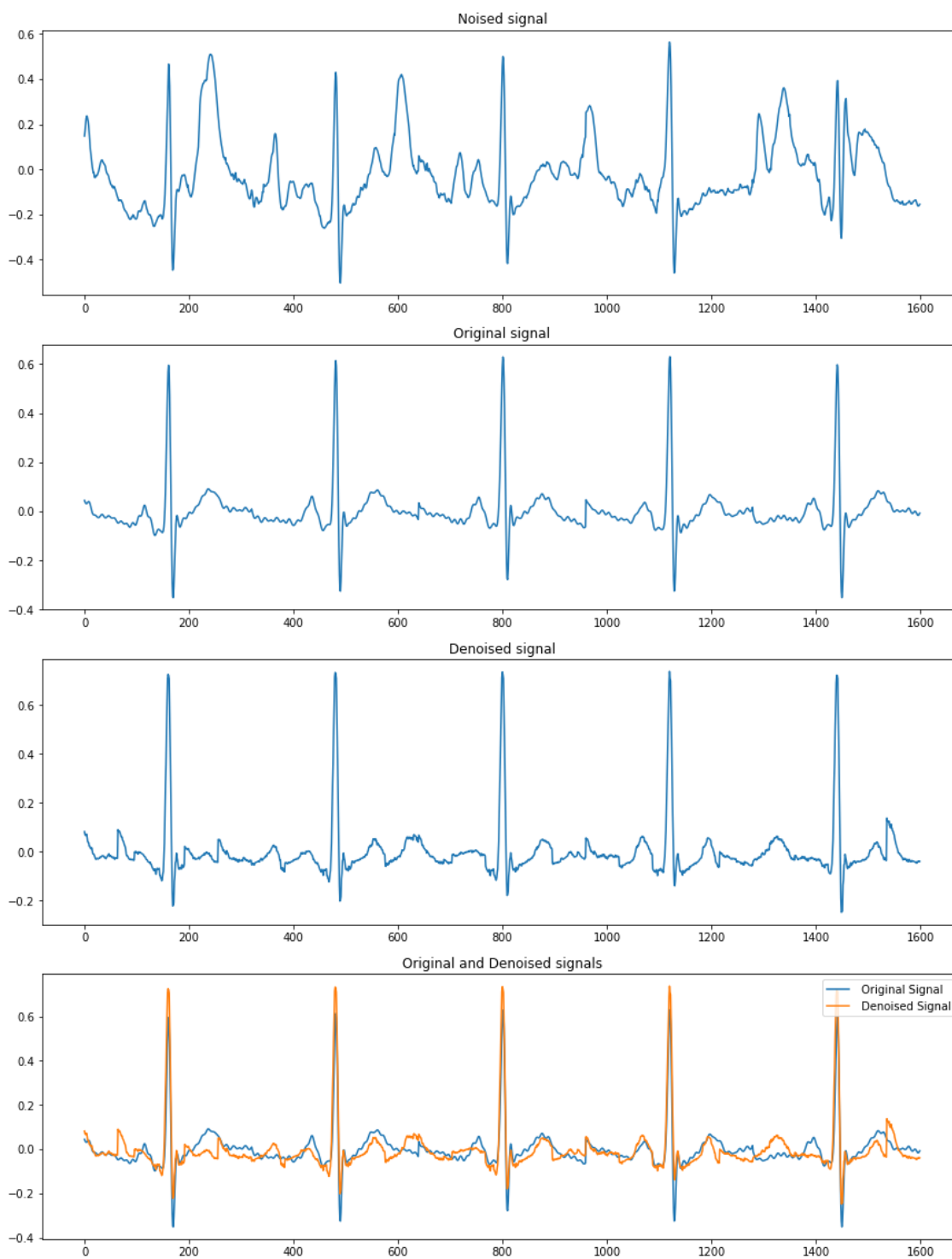
Noised, Original and Denoised signals with noise level 0.4



Noised, Original and Denoised signals with noise level 0.6



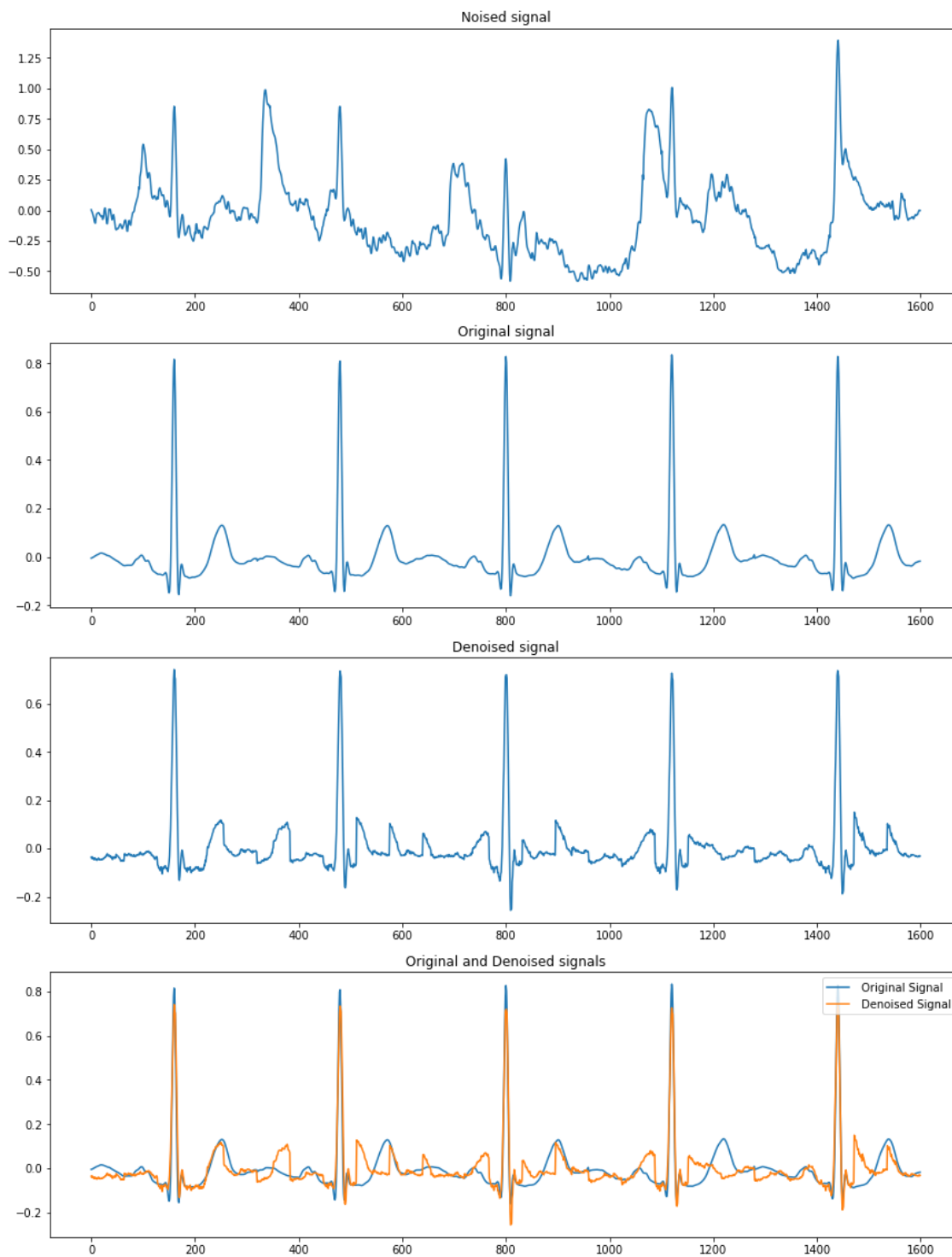
Noised, Original and Denoised signals with noise level 0.8



Denoising EM + MA + BW noises



Noised, Original and Denoised signals with noise level 0.6



Noised, Original and Denoised signals with noise level 0.8

